



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO
GRADO EN INGENIERÍA INFORMÁTICA

Utilización de Técnicas de Machine Learning para la Hibridación de Metaheurísticas

Autor

Marino Fernández Pérez

Directores

Daniel Molina Cabrera



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Junio de 2026

Utilización de Técnicas de Machine Learning para la Hibridación de Metaheurísticas

Marino Fernández Pérez

Palabras clave: metaheurísticas, modelos subrogados, aprendizaje automático, hibridación, optimización continua, optimización costosa.

Resumen

La optimización mediante metaheurísticas permite abordar problemas complejos para los que los métodos exactos no resultan viables. Sin embargo, estos algoritmos suelen requerir numerosas evaluaciones de la función objetivo. Cuando estas implican ejecutar simulaciones o procesos costosos, su aplicabilidad práctica queda limitada.

Los modelos subrogados tratan de reducir este coste aproximando la función objetivo a partir de soluciones ya evaluadas. Su integración no es inmediata, ya que la distribución de estas soluciones cambia conforme evoluciona la población y puede verse condicionada por fases de estancamiento. Antes de incorporar un modelo al bucle de optimización, resulta necesario analizar qué datos generan las metaheurísticas y cuándo conservan información útil para orientar la búsqueda.

Este trabajo estudia el uso de algoritmos de aprendizaje automático como alternativa a otras técnicas subrogadas para su integración en metaheurísticas evolutivas. Se analiza el histórico de evaluaciones generado por AGE, DE y SHADE sobre CEC2017 y se evalúa un mecanismo de reinicio destinado a reactivar la exploración. A partir de las muestras obtenidas, se comparan diferentes familias de modelos mediante estrategias de evaluación que respetan la estructura temporal de los datos generados durante la búsqueda. El estudio considera su capacidad para preservar el orden relativo entre soluciones, con el fin de seleccionar una configuración subrogada adecuada para su integración en las metaheurísticas y evaluar si mejora su comportamiento frente a las versiones originales.

Use of Machine Learning Techniques for the Hybridization of Metaheuristics

Marino Fernández Pérez

Keywords: metaheuristics, surrogate models, machine learning, hybridization, continuous optimization, expensive optimization.

Abstract

Metaheuristic optimization makes it possible to address complex problems for which exact methods are not feasible. However, these algorithms usually require a large number of objective function evaluations. When such evaluations involve simulations or costly processes, their practical applicability becomes limited.

Surrogate models aim to reduce this cost by approximating the objective function from previously evaluated solutions. Their integration is not straightforward, since the distribution of these solutions changes as the population evolves and may be affected by stagnation phases. Before incorporating a model into the optimization loop, it is necessary to analyze what data the metaheuristics generate and when these data retain useful information to guide the search.

This work studies the use of machine learning algorithms as an alternative to other surrogate techniques for their integration into evolutionary metaheuristics. The history of evaluations generated by AGE, DE, and SHADE on CEC2017 is analyzed, and a restart mechanism intended to reactivate exploration is evaluated. From the samples obtained, different families of models are compared using evaluation strategies that preserve the temporal structure of the data generated during the search. The study considers their ability to preserve the relative ordering of solutions, in order to select a suitable surrogate configuration for integration into the metaheuristics and assess whether it improves their behavior compared with the original versions.

Yo, **Marino Fernández Pérez**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 75941131F, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Marino Fernández Pérez

Granada, a 22 de junio de 2026.

D. Daniel Molina Cabrera, Profesor del Departamento de Ciencias de la Computación e Inteligencia Artificial de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado *Utilización de Técnicas de Machine Learning para la Hibridación de Metaheurísticas*, ha sido realizado bajo su supervisión por Marino Fernández Pérez, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada, 22 de junio de 2026.

El director:

Daniel Molina Cabrera

Agradecimientos

Índice general

Glosario	x
1. Introducción	1
1.1. Motivación	2
1.2. Objetivos	3
1.3. Estructura de la memoria	3
2. Planificación y presupuesto	5
2.1. Tareas realizadas y planificación temporal	5
2.2. Estimación del coste del proyecto	9
2.2.1. Coste de la mano de obra	9
2.2.2. Coste de equipamiento	9
2.2.3. Coste computacional equivalente	9
2.2.4. Presupuesto global	10
3. Fundamentos teóricos	11
3.1. Problemas de optimización continua de caja negra	11
3.2. Metaheurísticas poblacionales	12
3.2.1. Dinámica de búsqueda	12
3.2.2. Algoritmos evolutivos	13
3.3. Aprendizaje supervisado para aproximar la función objetivo .	14
3.4. Técnicas subrogadas en optimización	15
4. Revisión de la literatura: estado del arte	17
4.1. Optimización evolutiva asistida por subrogados	18
4.2. Gestión del modelo subrogado durante la búsqueda	19

4.3. Modelos subrogados y criterios de decisión	20
4.4. Integración de subrogados en metaheurísticas poblacionales	21
4.5. Posicionamiento del presente trabajo	22
5. Descripción de los algoritmos y técnicas usadas	24
5.1. Metaheurísticas consideradas	24
5.1.1. Algoritmo Genético Estacionario (AGE)	25
5.1.2. Mantenimiento del rango de búsqueda	27
5.1.3. Differential Evolution (DE)	28
5.1.4. Success-History based Adaptive Differential Evolution (SHADE)	30
5.2. Mecanismo de reinicio elitista	33
5.3. Algoritmos considerados como técnicas de subrogado	35
5.3.1. Least Absolute Shrinkage and Selection Operator (LAS- SO)	35
5.3.2. Response Surface Methodology (RSM)	35
5.3.3. Radial Basis Function (RBF)	36
5.3.4. Support Vector Regression (SVR)	36
5.3.5. Multilayer Perceptron (MLP)	37
5.3.6. Random Forest (RF)	37
5.3.7. Métodos de boosting basados en árboles	38
5.3.8. Kriging	39
5.4. Estrategias <i>offline</i> para la evaluación de modelos subrogados	39
5.4.1. Estrategia <i>offline</i> no acumulativa	40
5.4.2. Estrategia <i>offline</i> acumulativa	40
5.4.3. Validación sobre muestras futuras	41
5.5. Estrategia <i>online</i> para la integración del modelo subrogado	42
6. Diseño experimental	44
6.1. Benchmark utilizado	44
6.1.1. Suite CEC2017	44
6.2. Estrategias de resolución: Metaheurísticas	46
6.2.1. Criterio de reinicio elitista	47
6.3. Registro temporal de evaluaciones	48
6.4. Técnicas subrogadas: modelos candidatos y de referencia	49

6.4.1. Configuración experimental de los modelos	49
6.4.2. Preprocesamiento de los datos	49
6.5. Métricas empleadas	51
6.5.1. Error de predicción	52
6.5.2. Capacidad de ordenación	52
6.5.3. Coste computacional	53
6.5.4. Criterio de comparación de los modelos	54
6.6. Evaluación de modelos subrogados sobre históricos de eva- luaciones	54
6.7. Ajuste fino de hiperparámetros	56
6.8. Integración del modelo subrogado durante la búsqueda	57
6.9. Detalles de la implementación	59
6.9.1. Bibliotecas y herramientas empleadas	59
6.9.2. Registro y almacenamiento de resultados	61
6.9.3. Reproducibilidad de los experimentos	62
7. Experimentación y resultados	64
7.1. Estancamiento de las metaheurísticas y evaluación del reini- cio elitista	64
7.1.1. Motivación del reinicio	65
7.1.2. Evaluación de las configuraciones de reinicio	65
7.1.3. Conclusiones sobre la reactivación de la búsqueda	70
7.2. Estudio <i>offline</i> y ajuste de los modelos subrogados	72
7.2.1. Selección de la estrategia de entrenamiento <i>offline</i> . . .	72
7.2.2. Comparación de modelos por bloques temporales	73
7.2.3. Ajuste de Hiperparámetros	76
7.2.4. Resultados del modelo ajustado	79
7.3. Evaluación de la estrategia <i>online</i>	82
7.3.1. Calibración de los parámetros de integración	83
7.3.2. Comparación con la versión sin subrogado	84
7.3.3. Resumen de los resultados experimentales obtenidos . .	88
8. Conclusiones y trabajo futuro	89
Bibliografía	94

A. Repositorio del proyecto	95
B. Tablas completas de comparación TACOLAB	96
B.1. AGE	97
B.2. DE	98
B.3. SHADE	99
C. Tablas de configuraciones de hiperparámetros	100
C.1. Ranking global	100
C.2. AGE	101
C.3. DE	101
C.4. SHADE	102
D. Tablas de configuraciones de parámetros para modelar la hibridación <i>online</i>	103
D.1. AGE	104
D.2. DE	105
D.3. SHADE	105
E. Tablas de comparación de la hibridación respecto a la me- taheurística original	106
E.1. AGE	107
E.2. DE	108
E.3. SHADE	109
F. Parámetros de ejecución de los experimentos	110
F.1. Parámetros de ejecución de las metaheurísticas	110
F.2. Parámetros de ejecución de las técnicas subrogadas	112
F.3. Parámetros de ejecución de la hibridación	116

Índice de figuras

2.1. Diagrama de Gantt de la planificación temporal del proyecto.	8
4.1. Evolución anual del número de publicaciones recuperadas en Scopus sobre algoritmos evolutivos asistidos por subrogados.	17
5.1. Representación esquemática del mecanismo de reinicio elitista.	34
5.2. Representación esquemática de la estrategia <i>offline</i> no acumulativa.	40
5.3. Representación esquemática de la estrategia <i>offline</i> acumulativa.	41
5.4. Esquema de integración <i>online</i> del modelo subrogado como filtro previo a la evaluación real.	43
6.1. Representación ilustrativa de dos paisajes de búsqueda pertenecientes a la suite CEC2017.	45
6.2. Esquema de validación interna para el ajuste de hiperparámetros	57
7.1. Curvas de convergencia promedio sin reinicio en tres funciones representativas de CEC2017.	66
7.2. Distribución del error por bloques de evaluación: AGE sobre f_{12} sin reinicio (semilla 41).	67
7.3. Número medio de reinicios elitistas por ejecución según la ventana de estancamiento ρ	68
7.4. Evolución del rendimiento agregado en función del tamaño de la ventana de estancamiento ρ	70
7.5. Ejemplo del efecto de una ventana de reinicio corta en SHA-DE sobre f_{10} con $\rho = 0.01$	71

7.6. Evolución del tiempo medio de entrenamiento de SVR sobre f_4 para las estrategias acumulativa y no acumulativa.	72
7.7. Evolución de la correlación de Spearman media por bloque temporal para las estrategias acumulativa y no acumulativa. .	74
7.8. Spearman medio de RBF por función y metaheurística sobre CEC2017 completo.	81
7.9. Convergencia de AGE-1 frente a AGE-1-RBF.	85
7.10. Convergencia de DE-10 frente a DE-10-RBF.	86
7.11. Convergencia de SHADE-10 frente a SHADE-10-RBF.	87

Índice de Tablas

2.1. Desglose de mano de obra y tiempo de ejecución computacional del proyecto.	6
2.2. Resumen del presupuesto estimado del proyecto.	10
6.1. Familias de funciones de CEC2017 según la numeración empleada en este trabajo.	46
6.2. Parámetros comunes utilizados en la ejecución de las metaheurísticas.	46
6.3. Parámetros utilizados para AGE.	47
6.4. Parámetros utilizados para DE.	47
6.5. Parámetros utilizados para SHADE.	47
6.6. Configuraciones genéricas de reinicio consideradas para cada metaheurística.	48
6.7. Información registrada para cada evaluación.	49
6.8. Configuración inicial empleada para la evaluación de los modelos subrogados.	50
6.9. Resumen de las particiones utilizadas para construir los conjuntos de entrenamiento y validación en ambas estrategias.	55
7.1. Ranking medio de TACOLAB y número de funciones en las que cada configuración aparece entre las mejores para AGE.	68
7.2. Ranking medio de TACOLAB y número de funciones en las que cada configuración aparece entre las mejores para DE.	69
7.3. Ranking medio de TACOLAB y número de funciones en las que cada configuración aparece entre las mejores para SHADE.	69
7.4. Configuraciones de reinicio seleccionadas para las fases posteriores.	70
7.5. Comparativa de estrategias <i>offline</i> por algoritmo.	73

7.6. Comparativa de modelos para AGE bajo la estrategia no acumulativa, desglosada por bloque temporal.	75
7.7. Comparativa de modelos para DE bajo la estrategia no acumulativa, desglosada por bloque temporal.	75
7.8. Comparativa de modelos para SHADE bajo la estrategia no acumulativa, desglosada por bloque temporal.	76
7.9. Espacio de búsqueda utilizado en el ajuste fino de hiperparámetros.	77
7.10. Rank medio por kernel agregado sobre todas las configuraciones y algoritmos.	77
7.11. Cinco mejores configuraciones de RBF para AGE.	77
7.12. Cinco mejores configuraciones de RBF para DE.	78
7.13. Cinco mejores configuraciones de RBF para SHADE.	78
7.14. Configuración de RBF seleccionada para cada metaheurística	79
7.15. Comparativa entre la configuración homogénea y la configuración por algoritmo para AGE y SHADE.	79
7.16. Rendimiento <i>offline</i> de RBF con la configuración final.	80
7.17. Comparación del comportamiento <i>offline</i> de RBF ajustado sobre ejecuciones con y sin reinicio.	80
7.18. nMAE medio de RBF sobre f_{10} con y sin reinicio por bloque temporal para DE.	82
7.19. Configuración del subrogado <i>online</i> seleccionada por algoritmo.	83
7.20. Comparación entre cada metaheurística y su versión con filtro RBF.	84
B.1. Comparación TACOLAB por media del error final para AGE.	97
B.2. Comparación TACOLAB por media del error final para DE. .	98
B.3. Comparación TACOLAB por media del error final para SHA-DE.	99
C.1. Ranking global de las 36 configuraciones RBF candidatas, ordenadas por la peor posición obtenida entre las tres metaheurísticas (criterio minimax). La configuración seleccionada como homogénea aparece en negrita.	100
C.2. Top-15 configuraciones de RBF para AGE. Pos. indica la posición en el ranking completo de 36 configuraciones.	101
C.3. Top-15 configuraciones de RBF para DE. Pos. indica la posición en el ranking completo de 36 configuraciones.	101

C.4. Top-15 configuraciones de RBF para SHADE. Pos. indica la posición en el ranking completo de 36 configuraciones.	102
D.1. Ranking de configuraciones del subrogado <i>online</i> para AGE. . .	104
D.2. Ranking de configuraciones del subrogado <i>online</i> para DE. . .	105
D.3. Ranking de configuraciones del subrogado <i>online</i> para SHADE.	105
F.1. Parámetros de <code>ejecutar_metaheurísticas.py</code>	112
F.2. Parámetros de los programas de evaluación <i>offline</i>	113
F.3. Parámetros de <code>ajustar_hiperparametros.py</code>	116
F.4. Parámetros de <code>ejecutar_metaheurísticas_online.py</code>	119

Glosario

Offline: Fase en la que los modelos subrogados se entrenan y evalúan sobre datos previamente generados por las metaheurísticas, sin intervenir durante la ejecución de la búsqueda.

Online: Fase en la que el modelo subrogado se integra durante la ejecución de la metaheurística, de modo que sus predicciones pueden influir en la dinámica de la búsqueda.

Capítulo 1

Introducción

La optimización de problemas complejos continuos constituye un área de alta relevancia dentro de la informática y, en particular, dentro del marco de la inteligencia artificial y la computación evolutiva. En un gran número de aplicaciones reales, el objetivo consiste en encontrar configuraciones de alta calidad dentro de espacios de búsqueda de gran dimensión y, normalmente, difíciles de abordar mediante métodos exactos o deterministas. La dificultad no reside únicamente en el tamaño del espacio de búsqueda, sino también en la complejidad del propio problema de optimización.

En este contexto, las metaheurísticas han adquirido un papel relevante debido a su capacidad para explorar espacios de búsqueda complejos. A diferencia de otros métodos de optimización, estas técnicas permiten trabajar con problemas multimodales, no lineales o de estructura desconocida, lo que explica su utilización cuando no existen métodos exactos adecuados o cuando su aplicación no es óptima computacionalmente. Entre ellas, los algoritmos evolutivos destacan por combinar mecanismos de exploración y explotación para aproximar soluciones de alta calidad en problemas de muy diversa naturaleza.

Sin embargo, esta versatilidad tiene un coste importante: las metaheurísticas suelen necesitar muchas evaluaciones de la función objetivo para obtener soluciones competitivas. En la mayoría de algoritmos evolutivos, cada solución candidata debe evaluarse para estimar su calidad y orientar el proceso de búsqueda. Cuando dicha evaluación es sencilla, este proceso no supone un problema computacional. No obstante, en problemas reales donde la función objetivo representa simulaciones complejas, procesos físicos o procedimientos numéricos de alta carga computacional, el tiempo total de ejecución queda dominado por el coste acumulado de esas evaluaciones. En estos casos, incluso metaheurísticas bien diseñadas resultan inviables si no se dispone de mecanismos que reduzcan dicho esfuerzo computacional.

Esta limitación ha impulsado el interés por investigar estrategias que permitan mantener la capacidad exploratoria de las metaheurísticas reduciendo el número de evaluaciones reales. Entre las alternativas más utilizadas se encuentran las técnicas subrogadas, capaces de estimar el comportamiento de la función objetivo a partir de los datos observados durante el propio proceso de optimización. De este modo, la información obtenida por la metaheurística puede utilizarse no solo para actualizar la población, sino también para construir un modelo auxiliar capaz de orientar la búsqueda de forma más eficiente. En este tipo de integración, la utilidad del subrogado no depende únicamente de aproximar con precisión el valor numérico de la función objetivo, sino también de preservar correctamente las relaciones de calidad entre soluciones candidatas, ya que muchas decisiones evolutivas se basan en comparar individuos más que en conocer su valor exacto.

1.1. Motivación

Debido a que el aprendizaje automático y la computación evolutiva se han desarrollado tradicionalmente como dos comunidades científicas independientes, existe una falta de retroalimentación mutua en el diseño de algoritmos híbridos. La oportunidad de conectar ambas disciplinas motiva y define el objetivo de esta investigación. Sin embargo, integrar un modelo de aproximación en un procedimiento evolutivo plantea dificultades, ya que la distribución de las soluciones evaluadas cambian conforme la población evoluciona y dependen de la naturaleza de la propia metaheurística. Esta característica hace que no baste con entrenar un modelo de forma aislada, sino que sea necesario fundamentar la hibridación en un análisis de los datos generados por la metaheurística y cómo pueden aprovecharse.

Con el objetivo de conectar estas dos comunidades, en este Trabajo de Fin de Grado vamos a estudiar y mejorar el proceso de cómo se pueden utilizar los algoritmos de aprendizaje automático a partir del histórico de soluciones de los distintos algoritmos evolutivos para aplicarlas como técnicas de subrogado, y vamos a demostrar experimentalmente cómo se comportan. Para ello, evaluaremos el rendimiento de las metaheurísticas sobre un banco de problemas continuos a través de un enfoque metodológico dividido en dos etapas. En primer lugar, analizaremos el histórico de datos recolectado por los algoritmos para determinar con precisión qué técnicas y bajo qué configuraciones de hiperparámetros muestran mayor utilidad para ordenar soluciones candidatas. En segundo lugar, utilizaremos las conclusiones extraídas para integrar los mejores modelos detectados dentro de los algoritmos evolutivos, validando de manera dinámica su capacidad real para guiar eficazmente el proceso de optimización.

1.2. Objetivos

El objetivo principal de este Trabajo de Fin de Grado consiste en estudiar, implementar y evaluar el uso de modelos de aproximación como técnicas de subrogado para asistir a algoritmos evolutivos en problemas de optimización continua.

En resumen, los objetivos de este trabajo son:

1. Evaluar el comportamiento de las metaheurísticas seleccionadas sobre el banco de problemas considerado, incorporando estrategias de reinicio orientadas a mitigar la convergencia prematura y a favorecer la obtención de datos de búsqueda más informativos.
2. Construir y evaluar estrategias de entrenamiento y validación que respeten la estructura temporal de las muestras, con el fin de comparar distintas familias de modelos subrogados, incluyendo algoritmos de aprendizaje automático y aproximadores clásicos. Esta comparación se centra principalmente en la capacidad de ordenación de los modelos, considerando también su coste computacional.
3. Diseñar, implementar y evaluar la integración de los modelos más prometedores dentro de los algoritmos evolutivos, comparando su impacto final sobre la calidad de las soluciones, la dinámica de convergencia y el ahorro efectivo en el uso de evaluaciones reales, frente a las metaheurísticas originales.

1.3. Estructura de la memoria

El presente trabajo se organiza en varios capítulos que recogen de forma progresiva el contexto del problema, la planificación del proyecto, los fundamentos teóricos necesarios, la revisión de la literatura, el diseño experimental, el análisis de resultados y las conclusiones derivadas del estudio realizado.

En primer lugar, el Capítulo 1 introduce el problema abordado, expone la motivación del trabajo y formula los objetivos perseguidos. A continuación, el Capítulo 2 describe la planificación seguida durante el desarrollo del proyecto, así como la estimación del coste asociado a su realización.

El Capítulo 3 desarrolla los fundamentos teóricos necesarios para situar el trabajo, incluyendo los conceptos básicos de optimización, metaheurísticas, aprendizaje automático y modelos subrogados. El Capítulo 4 recoge el estado del arte, revisando los principales enfoques existentes en la literatura sobre la integración de modelos subrogados en procesos de optimización

evolutiva.

Seguidamente, el Capítulo 5 presenta las metaheurísticas, los modelos subrogados y las estrategias de referencia utilizadas durante el estudio. El Capítulo 6 describe el diseño experimental aplicado sobre dichas componentes, incluyendo el benchmark, las métricas, las estrategias de evaluación *offline* (realizada sobre datos ya generados), el ajuste de configuraciones y los detalles principales de implementación.

El Capítulo 7 recoge los resultados experimentales obtenidos y analiza el comportamiento de las metaheurísticas, la evaluación de los modelos subrogados y su posterior integración *online* (durante la ejecución de la metaheurística) en el proceso de optimización.

Por último, el Capítulo 8 resume las principales conclusiones alcanzadas y plantea las líneas de trabajo futuro. La memoria se completa con apéndices que recogen tablas complementarias utilizadas para apoyar el análisis experimental, así como los parámetros necesarios para reproducir las ejecuciones principales.

Capítulo 2

Planificación y presupuesto

En este capítulo se formaliza la organización temporal y metodológica adoptada para el desarrollo de este Trabajo de Fin de Grado, así como el desglose económico y la estimación del presupuesto financiero asociado a su ejecución real.

2.1. Tareas realizadas y planificación temporal

El proyecto se divide en distintas tareas donde, considerando únicamente la mano de obra del alumno, el tiempo dedicado se estima en 377 horas. En la Tabla 2.1 se detalla la planificación y la distribución de dicha carga de trabajo junto al tiempo de ejecución computacional.

A continuación, se detallan las tareas realizadas:

- **Comprensión del problema:** Análisis inicial de los objetivos del proyecto y estudio de los conceptos necesarios para comprender el problema de la optimización costosa, las metaheurísticas evolutivas y el uso de modelos subrogados.
- **Revisión bibliográfica:** Búsqueda y lectura de artículos, publicaciones y documentación necesaria para la realización del trabajo.
- **Implementación y adaptación de algoritmos:** Incluye la implementación y adaptación de las metaheurísticas utilizadas en el trabajo, junto con los *scripts* necesarios para automatizar ejecuciones, registrar resultados y generar tablas y gráficas de análisis.
- **Obtención y análisis de los resultados obtenidos:** Ejecución de los algoritmos de referencia sobre las distintas funciones del benchmark y análisis e interpretación de los resultados.

Tarea	Mano de obra	Cómputo	Total
Comprensión del problema	10 h	–	10 h
Revisión bibliográfica	22 h	–	22 h
Implementación y adaptación de algoritmos	40 h	–	40 h
Obtención y análisis de los resultados obtenidos	5 h	10 h	15 h
Diseño y evaluación del reinicio	25 h	35 h	60 h
Implementación de técnicas subrogadas y estrategias <i>offline</i>	40 h	–	40 h
Obtención y análisis de las estrategias <i>offline</i> y modelos subrogados	35 h	40 h	75 h
Ajuste de hiperparámetros para el mejor modelo	30 h	35 h	65 h
Integración del subrogado en las metaheurísticas	20 h	90 h	110 h
Análisis final de los resultados obtenidos	15 h	–	15 h
Realización de la memoria	120 h	–	120 h
Reuniones con el tutor	15 h	–	15 h
Total	377 h	210 h	587 h

Tabla 2.1: Desglose de mano de obra y tiempo de ejecución computacional del proyecto.

- **Diseño y evaluación del reinicio:** Recoge el estudio e implementación del criterio de reinicio, su ejecución y el análisis de su impacto sobre las metaheurísticas. Esta fase incluye también la evaluación de variantes preliminares que fueron descartadas al observar comportamientos no deseados.
- **Implementación de técnicas subrogadas y estrategias *offline*:** Implementación de los modelos subrogados considerados, definición de las estrategias de entrenamiento acumulativa y no acumulativa, construcción de los bloques temporales y cálculo de las métricas para su evaluación. Esta fase incluye también numerosas pruebas fallidas respecto a la definición de las estrategias de entrenamiento.
- **Obtención y análisis de las estrategias *offline* y modelos subrogados:** Incluye el entrenamiento de los modelos sobre las distintas estrategias y comparaciones analíticas entre ellos para seleccionar la mejor estrategia y el mejor modelo.
- **Ajuste de hiperparámetros para el mejor modelo:** Implementación de la estrategia utilizada para el ajuste de hiperparámetros y refinamiento del modelo más prometedor para su posterior integración *online*.
- **Integración del subrogado en las metaheurísticas:** Implementación de la hibridación *online*, parametrización y ejecución del modelo seleccionado y las técnicas subrogadas de referencia.
- **Análisis final de los resultados obtenidos:** Interpretación de los resultados y comparación con la variante sin subrogado.
- **Realización de la memoria:** Redacción de la memoria y revisión de la misma.
- **Reuniones con el tutor:** Preparación de reuniones de seguimiento, toma de decisiones metodológicas, contraste de resultados experimentales y reajuste de la planificación.

La planificación seguida se recoge en la Figura 2.1 mediante un diagrama de Gantt que muestra la distribución temporal de las tareas a lo largo del periodo de desarrollo del proyecto, comprendido entre el 11 de febrero y el 22 de junio de 2026.

Como se observa en la figura, algunas tareas se han llevado a cabo en paralelo, aunque, entre ellas, destaca la tarea encargada de implementar los algoritmos, que se extiende durante un periodo bastante largo debido a la incorporación posterior de una nueva metaheurística al estudio.

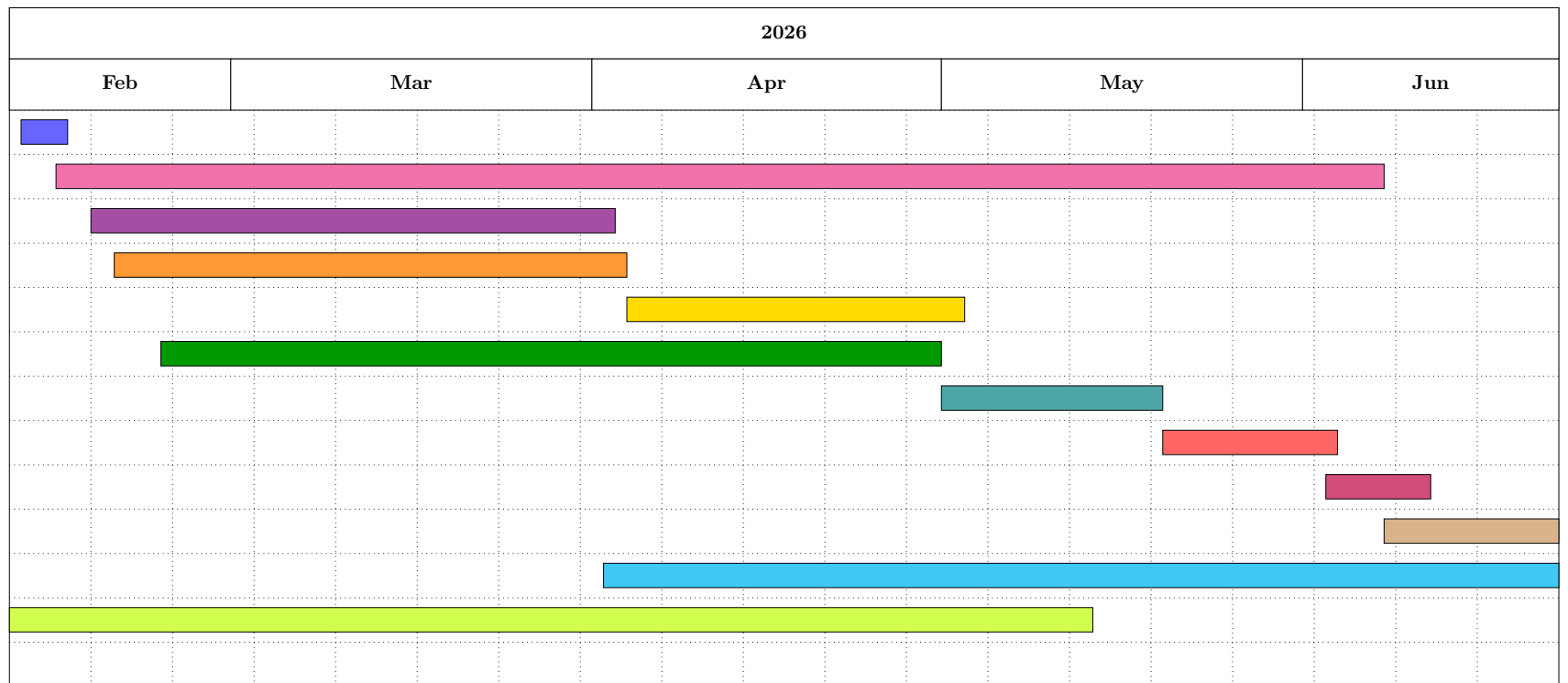


Figura 2.1: Diagrama de Gantt de la planificación temporal del proyecto.

2.2. Estimación del coste del proyecto

Para estimar el presupuesto financiero real del proyecto se consideran tres bloques de coste: costes de personal (mano de obra), costes de infraestructura física (equipamiento hardware junto a su amortización) y costes de computación externa equivalentes. Todas las herramientas y librerías empleadas son licencias de **Software Libre**, por lo que el coste respecto al software es de 0.00 €.

2.2.1. Coste de la mano de obra

El coste estimado de la mano de obra se calcula a partir del total de horas reales dedicadas al proyecto, que ascienden a 377 horas según el desglose presentado en la tabla 2.1. Considerando un precio de mano de obra estimado de 20.00 €/h correspondiente a un perfil junior en ingeniería informática que realiza el trabajo como profesional autónomo, el coste total es:

$$C_{\text{trabajo}} = 377 \text{ h} \cdot 20.00 \frac{\text{€}}{\text{h}} = 7\,540.00 \text{ €}.$$

2.2.2. Coste de equipamiento

El equipo empleado durante el desarrollo del proyecto se corresponde con un Apple MacBook Air de 13 pulgadas (chip Apple M4, 16 GB de memoria unificada y 512 GB de almacenamiento SSD) con un valor de adquisición aproximado en el mercado de 1200.00 €. No obstante, se asume una vida útil estandar para el equipamiento informático de 5 años (60 meses) y un periodo de uso intensivo para el desarrollo de este TFG de 3 meses. Aplicando el criterio de amortización, el coste es:

$$C_{\text{equipo}} = \frac{1200.00 \text{ €}}{60 \text{ meses}} \cdot 3 \text{ meses} = 60 \text{ €}.$$

2.2.3. Coste computacional equivalente

Aunque las simulaciones y el entrenamiento de los modelos se completaron de forma local aprovechando la arquitectura ARM del procesador del equipo, se calcula el coste equivalente utilizando una infraestructura en la nube y de pago. Se toma como referencia una instancia optimizada para computación en Amazon Web Services (AWS EC2), concretamente el modelo c7g.2xlarge (8 vCPU, 16 GiB RAM, arquitectura ARM Graviton).

La tarifa bajo demanda es de 0.2900 \$/h. Aplicando una tasa de cambio estimada de 1 \$ = 0.86 € e incrementando el 21 % en concepto de IVA aplicable en España, el precio final por hora es de 0.302 €/h. Para un cómputo acumulado de 210 horas de ejecución experimental, el coste computacional derivado es:

$$C_{\text{computo}} = 210 \text{ h} \cdot 0.302 \frac{\text{€}}{\text{h}} = 63.42 \text{ €}.$$

2.2.4. Presupuesto global

El coste total del proyecto (T) se calcula mediante la suma de los tres bloques analizados:

$$T = C_{\text{trabajo}} + C_{\text{equipo}} + C_{\text{computo}}$$

$$T = 7\,540.00 \text{ €} + 60.00 \text{ €} + 63.42 \text{ €} = 7\,663.42 \text{ €}.$$

En la Tabla 2.2 se presenta el desglose del presupuesto total estimado para la realización del Trabajo de Fin de Grado.

Concepto	Precio	Cant./Dur.	Coste
Mano de obra	20.00 €/h	377 h	7 540.00 €
Equipo in-formático	(1200 €/60m)	3 m	60.00 €
Tiempo computacio-nal	0.302 €/h	210 h	63.42 €
Herramientas	0.00 €/h	–	0.00 €
Total	–	–	7 663.42 €

Tabla 2.2: Resumen del presupuesto estimado del proyecto.

Capítulo 3

Fundamentos teóricos

Este capítulo recoge los fundamentos teóricos sobre los que se apoya el trabajo. Se parte de la formulación del problema de optimización continua de caja negra, se describen las metaheurísticas evolutivas como estrategia de resolución y se introduce el aprendizaje supervisado como base para construir modelos que aproximen la función objetivo. A partir de ahí se definen las técnicas subrogadas y el papel que desempeñan dentro del proceso de optimización.

3.1. Problemas de optimización continua de caja negra

De forma general, un problema de optimización puede definirse a partir de dos elementos principales: un espacio de búsqueda S y una función objetivo $f : S \rightarrow \mathbb{R}$, que asigna un valor numérico a cada solución candidata.

Entre los distintos tipos de problemas, este trabajo se centra en problemas de minimización continua, donde cada solución candidata se representa mediante un vector de variables reales $\mathbf{x} = (x_1, \dots, x_D) \in \mathbb{R}^D$, siendo D la dimensión del espacio de búsqueda. El objetivo consiste en encontrar la solución que minimiza la función objetivo f , lo que se expresa como:

$$\min_{\mathbf{x} \in S} f(\mathbf{x}). \quad (3.1)$$

Una solución $\mathbf{x}^* \in S$ se considera **óptimo global** si cumple $f(\mathbf{x}^*) \leq f(\mathbf{x})$ para todo $\mathbf{x} \in S$, y **óptimo local** si solo se satisface en una vecindad de $\mathbf{x}^*$.

Además, el espacio de búsqueda está acotado mediante restricciones de caja (*box constraints*), por lo que cada variable x_i solo puede tomar valores

dentro de un intervalo $[l_i, u_i]$. Así, el dominio queda definido como:

$$S = \prod_{i=1}^D [l_i, u_i] \subset \mathbb{R}^D. \quad (3.2)$$

Cuando el algoritmo no cuenta con información interna sobre la forma de f o no utiliza derivadas, únicamente puede evaluar soluciones candidatas y comparar valores obtenidos. En este contexto, el problema se trata como un problema de **caja negra**.

Esta situación es especialmente relevante cuando evaluar f tiene un coste elevado, ya que el número de llamadas a la función objetivo pasa a ser una limitación computacional importante. Esto motiva el uso de metaheurísticas y modelos subrogados, que se introducen en las secciones siguientes.

3.2. Metaheurísticas poblacionales

Las metaheurísticas son estrategias de búsqueda aproximadas diseñadas para encontrar soluciones de calidad en problemas donde los métodos exactos resultan inviables, ya sea por el coste computacional o por la ausencia de información suficiente para aplicar técnicas exactas o basadas en gradiente. Para ello, exploran el espacio de búsqueda bajo un presupuesto de evaluaciones limitado, sin garantizar necesariamente la obtención del óptimo global.

Dentro de este tipo de métodos, el trabajo se apoya en metaheurísticas **basadas en poblaciones**, que mantienen y actualizan simultáneamente un conjunto de soluciones candidatas. Frente a los enfoques basados en una única solución activa, trabajar con una población permite cubrir distintas regiones del espacio de búsqueda y aprovechar la información de los individuos para guiar el proceso de optimización.

3.2.1. Dinámica de búsqueda

El comportamiento de una metaheurística poblacional depende en gran medida del equilibrio entre exploración y explotación. La **exploración** orienta la búsqueda hacia regiones poco visitadas del espacio, reduciendo el riesgo de concentrarse demasiado pronto en una zona concreta. En cambio, la **explotación** utiliza la información ya obtenida para mejorar soluciones prometedoras.

Ambos procesos son necesarios: una exploración excesiva puede dificultar la convergencia, mientras que una búsqueda centrada en la explotación

puede hacer que la población quede atrapada en zonas poco competitivas, empeorando la calidad del resultado final.

El indicador más directo de este equilibrio es la **diversidad poblacional**, que mide hasta qué punto las soluciones de la población son diferentes entre sí. A medida que el algoritmo selecciona soluciones de mejor calidad, la población tiende a concentrarse en regiones prometedoras. Si esta concentración ocurre demasiado rápido, la diversidad se reduce excesivamente y el algoritmo puede entrar en una fase de **estancamiento**, en la que continúa realizando evaluaciones pero apenas obtiene mejoras relevantes.

3.2.2. Algoritmos evolutivos

Dentro de las metaheurísticas poblacionales, los **algoritmos evolutivos** constituyen la familia utilizada en este trabajo, inspirada en los mecanismos de selección natural. En estos algoritmos, cada solución candidata recibe un valor de *fitness*, que representa el valor asignado por la función objetivo f .

Aunque existen muchas variantes, estos algoritmos evolucionan una población a lo largo de sucesivas generaciones mediante cuatro operadores fundamentales.

- **Selección:** Determina qué individuos tienen prioridad para reproducirse, favoreciendo a los de mejor *fitness*.
- **Operadores de variación:** Introducen nuevas soluciones en el espacio de búsqueda.
 - **Cruce:** Genera nuevos candidatos combinando soluciones ya generadas.
 - **Mutación:** Introduce diversidad para mantener la exploración y evitar el estancamiento en óptimos locales.
- **Evaluación:** Asigna un valor de *fitness* a cada candidato.
- **Reemplazo:** Decide qué soluciones pasan a la siguiente generación entre la población actual y los nuevos descendientes.

Por tanto, las metaheurísticas poblacionales no solo producen soluciones candidatas, sino también un histórico de evaluaciones cuya distribución cambia a lo largo de la búsqueda. La diversidad, el estancamiento y la evolución temporal de la población condicionan directamente ese histórico, por lo que son conceptos necesarios para entender cómo se generan los datos utilizados en las fases posteriores del trabajo.

3.3. Aprendizaje supervisado para aproximar la función objetivo

El aprendizaje automático es un subcampo de la inteligencia artificial que agrupa técnicas para ajustar modelos a partir de datos, sin definir explícitamente todas las reglas que relacionan las entradas con las salidas. Este concepto encaja con el objetivo del presente trabajo al poder emplear las evaluaciones generadas como datos para ajustar dichos modelos.

Dentro de este ámbito, se utiliza **aprendizaje supervisado**, donde el modelo se ajusta a partir de ejemplos etiquetados. Cada ejemplo está formado por una entrada y una salida conocida, de manera que el modelo aprende a asociar ambas a partir de los datos disponibles. Una vez entrenado, puede emplearse sobre nuevas entradas no vistas durante el entrenamiento.

En el contexto considerado, las entradas son soluciones candidatas generadas por la metaheurística y las salidas son los valores obtenidos al evaluar la función objetivo. Por tanto, cada evaluación realizada durante la búsqueda proporciona una muestra de la forma:

$$(\mathbf{x}_i, f(\mathbf{x}_i)),$$

donde $\mathbf{x}_i \in \mathbb{R}^D$ representa una solución evaluada y $f(\mathbf{x}_i)$ su valor de *fitness*. A partir de un conjunto de N evaluaciones, se obtiene el conjunto de datos:

$$\mathcal{D} = \{(\mathbf{x}_i, f(\mathbf{x}_i))\}_{i=1}^N. \quad (3.3)$$

Dado que $f(\mathbf{x})$ toma valores numéricos continuos, el problema de aprendizaje se plantea como una tarea de **regresión**. En este caso, el objetivo no es asignar una clase a cada solución, sino construir un modelo $\hat{f}$ capaz de estimar el valor de la función objetivo para nuevas soluciones candidatas. Esta tarea puede realizarse mediante modelos con distinto nivel de complejidad: algunos parten de relaciones sencillas entre las variables de entrada y salida, mientras que otros permiten representar comportamientos más flexibles. En este trabajo se consideran varias alternativas dentro de este marco, que se describen en el Capítulo 5.

Para evaluar si esta aproximación generaliza correctamente, el conjunto de datos $\mathcal{D}$ se divide en dos subconjuntos con finalidades distintas. El **conjunto de entrenamiento** se utiliza para ajustar el modelo, mientras que el **conjunto de validación** se reserva para medir su comportamiento sobre muestras no utilizadas durante el entrenamiento. Esta separación permite detectar situaciones de **sobreajuste**, en las que el modelo memoriza los datos de entrenamiento pero pierde capacidad para predecir nuevas soluciones.

En este trabajo, la construcción de estos conjuntos debe tener en cuenta que las muestras no proceden de un muestreo aleatorio independiente, sino de las evaluaciones generadas durante una búsqueda evolutiva. Por ello, el orden en que aparecen las muestras y la fase de la búsqueda en la que se obtienen son aspectos relevantes para definir las estrategias de entrenamiento y validación.

3.4. Técnicas subrogadas en optimización

Las secciones anteriores muestran que, durante una búsqueda evolutiva, cada evaluación de la función objetivo aporta información sobre el problema. Cuando evaluar f tiene un coste elevado, es importante aprovechar esa información para reducir el número de llamadas a la función real. Las técnicas subrogadas parten de esta idea: construir una aproximación de menor coste de la función objetivo a partir de evaluaciones ya realizadas, de forma que pueda servir de apoyo durante la búsqueda.

Partiendo del conjunto de evaluaciones $\mathcal{D}$, definido en la Sección 3.3, el modelo $\hat{f}$ se utiliza como aproximación de la función objetivo:

$$\hat{f}(\mathbf{x}) \approx f(\mathbf{x}). \quad (3.4)$$

Cuando esta aproximación se emplea para guiar la búsqueda o para decidir qué soluciones merece la pena evaluar con la función real, el modelo actúa como subrogado. Por tanto, el término subrogado no hace referencia únicamente al tipo de modelo utilizado, sino al papel que cumple dentro del proceso de optimización. Un algoritmo de aprendizaje automático puede desempeñar este papel, aunque también pueden emplearse métodos estadísticos, de interpolación u otras técnicas de aproximación.

Aunque los modelos considerados en este trabajo se entrenan como regresores, su utilidad dentro de la búsqueda no depende únicamente de la precisión numérica de sus predicciones. En una metaheurística, el subrogado puede ser útil si ayuda a distinguir qué soluciones parecen más prometedoras, aunque no estime exactamente su valor de *fitness*. En un problema de minimización, si $f(\mathbf{x}_a) < f(\mathbf{x}_b)$, el modelo debería reflejar esa misma relación mediante $\hat{f}(\mathbf{x}_a) < \hat{f}(\mathbf{x}_b)$.

Esta idea permite diferenciar entre **fitness absoluto**, asociado al valor numérico de la función objetivo, y **fitness relativo**, asociado al orden de calidad entre soluciones [1]. Un modelo puede cometer cierto error en los valores predichos y, aun así, ordenar correctamente los candidatos. Por ello, al evaluar un subrogado resulta relevante determinar qué métricas de evaluación resultan más adecuadas según el uso del subrogado.

El principal inconveniente es el histórico de evaluaciones no uniforme que generan las metaheurísticas. El subrogado se ve afectado por la dinámica de la población durante la búsqueda, ya que se ajusta con la información disponible en un momento concreto y su fiabilidad depende de que esos datos representen adecuadamente las zonas que se quieren estimar. Si el modelo se utiliza fuera de ese contexto, puede asignar buena calidad a soluciones que después no resulten competitivas al evaluarlas con f .

Los modelos concretos utilizados en este trabajo se describen en el Capítulo 5, mientras que su evaluación experimental y su integración en las metaheurísticas se detallan en el Capítulo 6.

Capítulo 4

Revisión de la literatura: estado del arte

Dentro de los algoritmos evolutivos, el uso de modelos subrogados surge como respuesta a una limitación habitual en problemas de optimización costosa: la evaluación repetida de la función objetivo. En este contexto, los algoritmos evolutivos asistidos por subrogados (*surrogate-assisted evolutionary algorithms*, SAEA) tratan de reducir el número de evaluaciones reales necesarias mediante modelos aproximados que orientan la búsqueda hacia regiones prometedoras.

La actividad reciente del área se refleja también en la evolución del número de publicaciones indexadas en Scopus. La Figura 4.1 muestra un crecimiento claro de los trabajos relacionados con SAEA durante la última década, especialmente a partir de 2019.

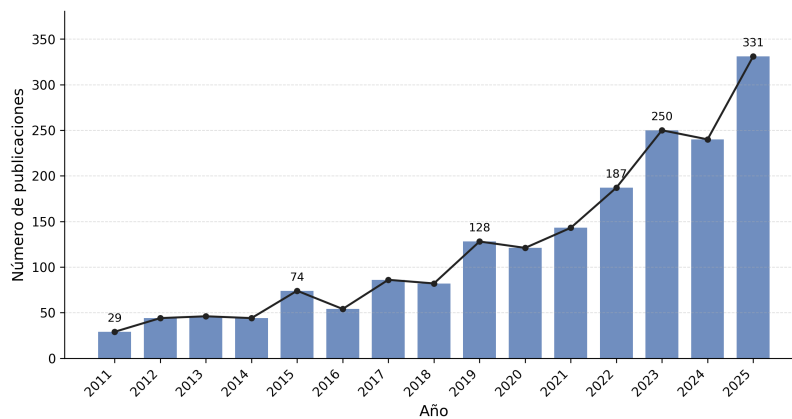


Figura 4.1: Evolución anual del número de publicaciones recuperadas en Scopus sobre algoritmos evolutivos asistidos por subrogados.

Este capítulo sitúa la propuesta del presente trabajo dentro de esta línea de investigación. A diferencia del capítulo anterior, centrado en los fundamentos teóricos necesarios para comprender las técnicas empleadas, aquí se revisa cómo la literatura ha abordado la integración de modelos subrogados en algoritmos evolutivos y qué cuestiones permanecen relevantes para el diseño experimental de este Trabajo de Fin de Grado. En particular, se presta atención a la gestión del modelo durante la búsqueda, al tipo de información que debe aportar el subrogado y a su integración en metaheurísticas poblacionales. Dentro de este marco, también se considera el uso de modelos de aprendizaje automático supervisado como subrogados.

Aunque la optimización multiobjetivo asistida por subrogados constituye una línea de investigación extensa y en auge, este trabajo se acota a la optimización continua monoobjetivo. Esta delimitación permite orientar la revisión en los trabajos más relacionados con el objetivo de esta memoria: analizar modelos subrogados sobre históricos de evaluaciones y estudiar su posterior integración *online* como mecanismo de filtrado.

4.1. Optimización evolutiva asistida por subrogados

El uso de modelos aproximados en optimización no es exclusivo de los algoritmos evolutivos. Métodos como *Efficient Global Optimization* (EGO) ya empleaban Kriging, una técnica estadística clásica de interpolación y modelado espacial, para guiar la búsqueda de óptimos en funciones de caja negra costosas [2]. Estas ideas sirvieron como base para el desarrollo posterior de la optimización asistida por subrogados.

En computación evolutiva, la incorporación de modelos subrogados se planteó inicialmente como una forma de aproximar el *fitness* de los individuos sin evaluar siempre la función objetivo real. La revisión de Jin [3] agrupó diferentes modelos de aproximación, incluyendo polinomios, Kriging y redes neuronales. Esta variedad refleja que los subrogados pueden construirse tanto mediante técnicas estadísticas clásicas como mediante modelos de aprendizaje automático supervisado. Además, este trabajo destacó que el problema no consiste solo en elegir un buen aproximador, sino también en decidir cómo combinar sus predicciones con evaluaciones reales para evitar que el subrogado dirija la búsqueda hacia regiones incorrectas.

A partir de estos fundamentos, años después Jin [4] revisó los avances recientes del área y señaló los retos pendientes, entre ellos la necesidad de estrategias de gestión del modelo, el uso de varios subrogados y la falta de estudios comparativos más sistemáticos. Estos trabajos consolidan una idea que sigue siendo importante en la literatura actual: el subrogado no debe

entenderse como un sustituto directo y permanente de la función objetivo, sino como un componente integrado en el proceso evolutivo.

Las revisiones más recientes confirman la consolidación de esta línea de investigación. He et al. [5] revisan los SAEA aplicados a problemas de optimización costosa y los organizan según los algoritmos empleados y sus escenarios de aplicación. Más recientemente, trabajos como el de Liang et al. [6] proponen taxonomías actualizadas para describir la construcción del subrogado, su actualización y su uso dentro del algoritmo evolutivo. En conjunto, estas revisiones muestran que la integración de subrogados ha pasado de ser una técnica auxiliar a constituir una línea de investigación propia dentro de la optimización evolutiva.

4.2. Gestión del modelo subrogado durante la búsqueda

Uno de los aspectos clave en los SAEA es controlar cuándo interviene el modelo subrogado durante la búsqueda. Si se utiliza sin una política de control adecuada, sus errores pueden conducir al algoritmo hacia regiones que parecen prometedoras según el modelo, pero que no lo son al evaluarlas con la función objetivo real. Por este motivo, buena parte de la literatura no se centra únicamente en mejorar la precisión del subrogado, sino también en regular su uso dentro del proceso de optimización.

Esta idea aparece ya en el marco propuesto por Jin et al. [7], donde se introducen estrategias de *evolution control* para combinar evaluaciones aproximadas y reales. Estas estrategias pueden aplicarse a nivel de individuo, decidiendo qué soluciones se evalúan con el modelo y cuáles con la función real, o a nivel de generación, alternando fases de evaluación aproximada con fases de corrección mediante evaluaciones reales.

Esta gestión también afecta al tiempo durante el cual el modelo permanece activo antes de actualizarse. Loshchilov et al. [8] proponen una variante de CMA-ES asistida por subrogados que ajusta *online* la vida útil del modelo y sus hiperparámetros. Aunque se trata de una metaheurística distinta a las consideradas en esta memoria, el trabajo muestra que la frecuencia de actualización forma parte del diseño de la integración.

La necesidad de controlar el modelo se mantiene en propuestas posteriores. Yu et al. [9] proponen una estrategia de reinicio por generaciones para un algoritmo de enjambre de partículas asistido por un modelo RBF global. En este caso, el reinicio se combina con mecanismos de gestión del subrogado a nivel de generación y de individuo, mostrando que la coordinación entre la dinámica de la población y el uso del subrogado puede mejorar la búsqueda en problemas computacionalmente costosos. Esta idea es rele-

vante para este trabajo, donde se estudia posteriormente la conveniencia de incorporar reinicios en las metaheurísticas consideradas.

Otro aspecto relacionado con la gestión del modelo es su coste computacional. Aunque los subrogados se utilizan para reducir el número de evaluaciones reales, su entrenamiento y predicción también tienen un coste. Blik [10] señala que no basta con reducir las evaluaciones costosas, sino que debe considerarse el coste total del proceso, incluyendo el aprendizaje del modelo y la optimización sobre la aproximación. Esta cuestión es relevante cuando se comparan modelos de distinta complejidad, ya que una mejora en la predicción puede no compensar si introduce un coste adicional elevado.

En conjunto, estos trabajos muestran que integrar un subrogado en una metaheurística requiere decisiones que van más allá del modelo predictivo. Es necesario determinar con qué datos se entrena, cuándo se actualiza, con qué frecuencia se consulta y cómo se limita el efecto de sus errores durante la búsqueda.

4.3. Modelos subrogados y criterios de decisión

La elección del modelo subrogado no depende solo de su familia estadística o de aprendizaje, sino también del tipo de información que se espera obtener de él. En algunos casos interesa aproximar con precisión el valor de la función objetivo. En otros, es suficiente con que el modelo permita comparar soluciones, identificar candidatos prometedores o descartar aquellos que probablemente no mejorarán la población. Esta diferencia es importante en algoritmos evolutivos, donde muchas decisiones se basan en relaciones de preferencia entre individuos más que en el valor exacto del *fitness*.

Esta cuestión aparece de forma explícita en el trabajo de Tong et al. [1], utilizado como referencia en el Capítulo 3 para distinguir entre modelos orientados al *fitness* absoluto y modelos orientados al *fitness* relativo. Más allá de la taxonomía, su estudio compara modelos de distinta naturaleza como RBF, kNN-R, RankSVM y SVC, considerando tanto la calidad predictiva como el coste de construcción y predicción y su efecto dentro de un SAEA monoobjetivo.

Díaz-Manríquez et al. [11] comparan también distintas técnicas subrogadas integradas en algoritmos evolutivos, entre ellas superficies de respuesta polinómicas, Kriging, RBF y SVR. En este caso, además del error de predicción, se considera la preservación del ranking como criterio para seleccionar modelos adecuados dentro de un proceso evolutivo. Esto refuerza la idea de que un subrogado no debe evaluarse solo por su error numérico, sino también por su capacidad para mantener relaciones de orden entre soluciones.

La incorporación de técnicas de aprendizaje automático amplía este marco más allá de los modelos estadísticos clásicos. Naharro et al. [12] comparan formulaciones basadas en regresión, orientadas a aproximar el *fitness*, con modelos pareados orientados directamente a la comparación entre soluciones. Su estudio considera distintos algoritmos de aprendizaje automático, incluyendo regresión regularizada, redes neuronales, árboles de decisión, métodos de *boosting* y bosques aleatorios.

Algunos trabajos llevan esta idea hacia modelos de clasificación, donde el subrogado separa soluciones prometedoras y no prometedoras. Sin embargo, este trabajo se centra en modelos regresivos, ya que cada solución candidata tiene asociado un valor real de *fitness*.

Por tanto, la evaluación de los modelos no debe limitarse a métricas de error absoluto. Cuando el subrogado se utiliza para decidir qué candidatos merecen una evaluación real, también es necesario medir si conserva correctamente el orden entre soluciones. Por este motivo, en el presente Trabajo de Fin de Grado se priorizan las métricas ordinales frente a las métricas de error.

4.4. Integración de subrogados en metaheurísticas poblacionales

La integración de un subrogado en una metaheurística poblacional no se limita a entrenar un modelo externo y consultar sus predicciones. En la mayoría de propuestas recientes, el modelo interviene en decisiones concretas del algoritmo, como la selección de descendientes, la identificación de regiones prometedoras o la elección de muestras que deben evaluarse con la función objetivo real. Por ello, el rendimiento final depende tanto de la calidad del modelo como de la forma en que sus predicciones modifican la dinámica de la población.

Dentro de los SAEA recientes, la evolución diferencial aparece con frecuencia como base algorítmica para integrar subrogados en problemas continuos. Wang et al. [13] siguen esta idea en ESAO, donde un modelo RBF global se utiliza para preseleccionar descendientes generados por evolución diferencial, mientras que una búsqueda local asistida por subrogado refuerza la explotación de regiones prometedoras.

Otros trabajos han extendido esta integración mediante estructuras más elaboradas. Cai et al. [14] proponen un algoritmo evolutivo asistido por subrogados para problemas costosos de alta dimensión, basado en un marco de algoritmo genético que combina búsqueda local, actualización guiada de la población y preselección de candidatos mediante un criterio de mejora esperada. Por otro lado, Wang et al. [15] combinan un modelo RBF global

y un modelo Kriging local dentro de una evolución diferencial asistida por subrogados. El primero se utiliza para capturar tendencias globales, mientras que el segundo explota regiones prometedoras teniendo en cuenta la incertidumbre del modelo. Estas propuestas muestran que el subrogado no se usa únicamente para aproximar el *fitness*, sino también para orientar qué zonas del espacio de búsqueda reciben nuevas evaluaciones reales.

La actualización del modelo también es importante en los enfoques *online*. Si el subrogado se reentrena continuamente conforme se acumulan nuevos datos, el coste de actualización puede convertirse en un cuello de botella. Para reducir este impacto, Zhan y Xing [16] estudian esquemas de aprendizaje incremental orientados a disminuir el coste de actualizar el subrogado durante la búsqueda. Esta preocupación por la eficiencia temporal es un factor clave en el presente Trabajo de Fin de Grado, donde la viabilidad de la integración *online* de los modelos de aprendizaje automático considerados depende directamente de mantener un coste de entrenamiento controlado.

La presencia de la evolución diferencial en esta línea de investigación ha dado lugar a revisiones específicas, como la de Yu et al. [17], centrada en algoritmos de evolución diferencial asistidos por subrogados. Esto justifica que parte de la comparación experimental se realice sobre algoritmos como DE y SHADE, ambos relacionados con esta familia. No obstante, el objetivo de esta memoria no es proponer una nueva variante de evolución diferencial, sino evaluar si un modelo subrogado seleccionado en una fase *offline* puede integrarse como filtro *online* en metaheurísticas poblacionales de distinta naturaleza, incluyendo también un algoritmo genético.

4.5. Posicionamiento del presente trabajo

La literatura revisada muestra que la utilidad de un modelo subrogado en optimización evolutiva no depende únicamente de su precisión predictiva. También influyen el tipo de información que aporta, el coste de construirlo y actualizarlo, y la forma en que sus predicciones intervienen dentro del algoritmo evolutivo. Esta perspectiva motiva que en este trabajo la selección del subrogado no se plantee como una comparación aislada entre modelos, sino como una evaluación progresiva: primero se caracterizan varios modelos sobre datos ya generados durante la búsqueda y, a partir de esos resultados, se estudia su integración *online* en las metaheurísticas consideradas.

Aunque existen trabajos sobre optimización evolutiva basada en datos históricos, revisados de forma general por Jin et al. [18], el uso *offline* realizado en esta memoria tiene un propósito distinto. En este caso, el subrogado no se emplea para ejecutar la búsqueda, sino para comparar modelos antes de incorporarlos al bucle de optimización.

A partir de esa caracterización, la integración *online* se aborda como un mecanismo de filtrado controlado. El subrogado no sustituye de forma definitiva a la función objetivo, sino que decide qué candidatos parecen suficientemente prometedores para consumir una evaluación real. De este modo, el trabajo se sitúa en la línea de los SAEA que combinan predicción, gestión del modelo y evaluación real, analizando si una misma estrategia de integración puede aplicarse a distintas metaheurísticas poblacionales.

Los capítulos siguientes presentan las metaheurísticas, los modelos, las estrategias de evaluación y el marco experimental utilizados.

Capítulo 5

Descripción de los algoritmos y técnicas usadas

Este capítulo describe los algoritmos y técnicas que se utilizan como base en la parte experimental del trabajo. En primer lugar, se presentan las metaheurísticas consideradas, empleadas tanto como algoritmos de referencia como para generar los históricos de evaluaciones sobre los que se entrenan y analizan los modelos subrogados.

A continuación, se introduce el mecanismo de reinicio elitista utilizado para reactivar la exploración cuando la búsqueda se estanca. Después se describen los modelos subrogados considerados en la comparativa, diferenciando entre modelos de aprendizaje automático y técnicas clásicas de referencia. Por último, se presentan las estrategias *offline* utilizadas para evaluar los modelos sobre históricos de evaluaciones y el esquema *online* mediante el que el subrogado se integra durante la búsqueda.

5.1. Metaheurísticas consideradas

Las metaheurísticas consideradas en este trabajo son las encargadas de generar las soluciones y, con ello, los históricos de evaluaciones utilizados posteriormente para analizar los modelos subrogados. Todas ellas trabajan con soluciones representadas mediante vectores reales y mantienen una población de individuos que se actualiza a lo largo de la ejecución.

Se han seleccionado tres algoritmos evolutivos que permiten cubrir distintos niveles de complejidad y adaptación: **AGE**, **DE** y **SHADE**. De este modo, se estudia si la utilidad de los modelos subrogados depende de la dinámica de búsqueda que genera los datos, y no solo del modelo utilizado.

5.1.1. Algoritmo Genético Estacionario (AGE)

Los algoritmos genéticos constituyen una de las familias clásicas dentro de los algoritmos evolutivos [19]. Estas metaheurísticas mantienen una población de soluciones candidatas y generan nuevos individuos mediante operadores inspirados en la evolución biológica como la selección, el cruce y la mutación.

Dentro de esta familia, el **Algoritmo Genético Estacionario (AGE)** se diferencia de los esquemas generacionales en la forma de actualizar la población. En lugar de reemplazar todos los individuos en cada generación, la descendencia generada compete con los individuos ya presentes y solo se incorpora si mejora la calidad de la población. Así, se conservan las mejores soluciones de la población, que van mejorando durante la búsqueda.

A continuación se presenta la variante implementada del algoritmo AGE, donde cada individuo se representa mediante un vector real $\mathbf{x}_i \in \mathbb{R}^D$, y cada componente corresponde a una variable de decisión del problema. Posteriormente se detallan cada uno de los operadores que intervienen en el pseudocódigo.

Algorithm 1 Algoritmo Genético Estacionario (AGE).

```

1:  $P \leftarrow \text{GENERARPOBLACIONINICIAL}()$ 
2:  $\text{EVALUAR}(P)$ 
3: while no se cumpla el criterio de parada do
4:    $p_1, p_2 \leftarrow \text{SELECCIONTORNEO}(P)$ 
5:   if  $\text{rand}() < p_c$  then
6:      $h_1, h_2 \leftarrow \text{CRUCEBLX\_ALPHA}(p_1, p_2)$ 
7:   else
8:      $h_1 \leftarrow p_1$ 
9:      $h_2 \leftarrow p_2$ 
10:  end if
11:   $h_1 \leftarrow \text{MUTACIONGAUSSIANA}(h_1, p_m, \sigma)$ 
12:   $h_2 \leftarrow \text{MUTACIONGAUSSIANA}(h_2, p_m, \sigma)$ 
13:   $\text{EVALUAR}(h_1, h_2)$ 
14:   $h^* \leftarrow \text{SELECCIONARMEJORHIJO}(h_1, h_2)$ 
15:   $P \leftarrow \text{REEMPLAZOESTACIONARIO}(P, h^*)$ 
16: end while
17: return mejor solución encontrada

```

En cada iteración, AGE selecciona dos progenitores, genera dos descendientes mediante cruce y mutación, evalúa los nuevos candidatos y conserva el mejor hijo como candidato a incorporarse a la población. La actualización se realiza mediante reemplazo estacionario, por lo que la población solo

cambia si el nuevo candidato mejora al peor individuo actual.

Operador de selección

La selección de progenitores se realiza mediante **selección por torneo**. En cada selección se toma un subconjunto aleatorio $\mathcal{T} \subset P$ de individuos de la población y se escoge como progenitor el individuo con menor valor de *fitness*:

$$\mathbf{p} = \arg \min_{\mathbf{x} \in \mathcal{T}} f(\mathbf{x}).$$

Este mecanismo favorece la reproducción de soluciones de mayor calidad, pero mantiene un componente aleatorio, ya que la comparación solo se realiza dentro del subconjunto seleccionado. De este manera, AGE introduce presión selectiva sin eliminar por completo la diversidad.

Operador de cruce

Una vez seleccionados los progenitores, la descendencia se genera mediante el operador de cruce **BLX- α** , habitual en algoritmos genéticos con codificación real [20]. Sean $\mathbf{p}^{(1)}$ y $\mathbf{p}^{(2)}$ dos progenitores. Para cada componente j , se definen

$$c_{\min,j} = \min\{p_j^{(1)}, p_j^{(2)}\}, \quad c_{\max,j} = \max\{p_j^{(1)}, p_j^{(2)}\},$$

y la amplitud del intervalo entre ambos valores como

$$I_j = c_{\max,j} - c_{\min,j}.$$

A partir de estos valores, la componente j del descendiente se obtiene muestreando de forma uniforme en el intervalo ampliado:

$$h_j \sim \mathcal{U}(c_{\min,j} - \alpha I_j, c_{\max,j} + \alpha I_j).$$

El parámetro α controla cuánto se amplía el intervalo definido por los progenitores. Si $\alpha = 0$, el descendiente se genera entre ambos padres. Para valores positivos, el intervalo se amplía y permite explorar regiones próximas a ellos.

Operador de mutación

Tras el cruce se aplica una **mutación gaussiana** sobre los descendientes. Su objetivo es introducir ruido en las variables de decisión para man-

tener diversidad y reducir el riesgo de convergencia prematura. Para cada componente j de un descendiente $\mathbf{h}$, la mutación se define como:

$$h'_j = \begin{cases} h_j + \sigma z_j & \text{con probabilidad } p_m, \\ h_j & \text{con probabilidad } 1 - p_m, \end{cases}$$

$$z_j \sim \mathcal{N}(0, 1).$$

El parámetro p_m controla la probabilidad de mutar cada componente, mientras que σ determina la magnitud de la perturbación introducida. Después de aplicar la mutación, los valores generados deben mantenerse dentro del dominio permitido, como se describe en la Subsección 5.1.2.

Operador de reemplazo

Finalmente, los descendientes generados se evalúan mediante la función objetivo y se conserva el mejor de ellos. Si $\mathbf{h}'_1$ y $\mathbf{h}'_2$ son los dos hijos obtenidos tras la mutación, el candidato a incorporarse a la población se define como

$$\mathbf{h}^* = \arg \min_{\mathbf{h} \in \{\mathbf{h}'_1, \mathbf{h}'_2\}} f(\mathbf{h}).$$

El esquema de actualización es **estacionario**. En lugar de reemplazar toda la población, el mejor hijo compite únicamente con el peor individuo actual. Sea

$$\mathbf{x}^{\text{peor}} = \arg \max_{\mathbf{x} \in P} f(\mathbf{x})$$

el individuo de peor calidad de la población, la nueva población se define como

$$P' = \begin{cases} (P \setminus \{\mathbf{x}^{\text{peor}}\}) \cup \{\mathbf{h}^*\}, & \text{si } f(\mathbf{h}^*) < f(\mathbf{x}^{\text{peor}}), \\ P, & \text{en otro caso.} \end{cases}$$

Así, AGE mantiene constante el tamaño de la población y la actualización se produce de forma gradual.

5.1.2. Mantenimiento del rango de búsqueda

Los operadores de cruce y mutación pueden generar valores fuera del dominio permitido por el problema. Para evitar evaluar soluciones fuera del dominio, cada candidato se acota componente a componente al intervalo de búsqueda definido.

Si l_j y u_j representan los límites inferior y superior de la variable j , respectivamente, la acotación aplicada a cada componente se define como:

$$x_j = \min\{u_j, \max\{l_j, x_j\}\}.$$

En AGE esta acotación se aplica tras el cruce BLX- α y tras la mutación gaussiana. En DE y SHADE se aplica sobre los vectores mutantes antes del cruce, de forma que los vectores de prueba evaluados pertenecen siempre al dominio permitido.

El intervalo concreto utilizado en los experimentos se define junto con el benchmark en el Capítulo 6.

5.1.3. Differential Evolution (DE)

La segunda metaheurística considerada es *Differential Evolution* (DE), introducida por Rainer Storn y Kenneth Price [21]. En este trabajo, DE se incluye como referencia diferencial básica frente al esquema genético de AGE y a la variante adaptativa representada por SHADE.

Al igual que AGE, DE mantiene una población de soluciones candidatas que evoluciona a lo largo de la ejecución. Sin embargo, la forma en la que se generan nuevos individuos es distinta. En DE, los candidatos se construyen combinando vectores de la población y diferencias entre soluciones de la propia población. Estas diferencias proporcionan información sobre la distribución actual de las soluciones y permiten generar nuevos puntos siguiendo direcciones inducidas por la propia población.

En la literatura se describen distintas variantes de DE mediante la notación DE/**x**/**y**/**z**, donde **x** indica el vector base utilizado en la mutación, y el número de diferencias empleadas y **z** el tipo de cruce aplicado. En este caso, la variante considerada es DE/**rand**/1/**bin** y cada individuo se representa mediante un vector real $\mathbf{x}_{i,g} \in \mathbb{R}^D$, donde i identifica al individuo y g la generación actual:

- **rand** indica que el vector base de la mutación se selecciona aleatoriamente,
- 1 indica que se emplea una única diferencia vectorial,
- **bin** hace referencia al uso de cruce binomial.

A continuación se presenta el esquema implementado de DE. Posteriormente se detallan los operadores que intervienen en dicho esquema.

En cada generación, DE recorre todos los individuos de la población. Para cada uno de ellos genera un vector mutante, lo combina con el individuo

Algorithm 2 *Differential Evolution* (DE).

```

1:  $P_0 \leftarrow \text{GENERARPOBLACIONINICIAL}()$ 
2:  $\text{EVALUAR}(P_0)$ 
3:  $g \leftarrow 0$ 
4: while no se cumpla el criterio de parada do
5:   for cada individuo objetivo  $\mathbf{x}_{i,g} \in P_g$  do
6:      $\mathbf{v}_{i,g} \leftarrow \text{MUTACIONDIFERENCIAL}(P_g, i, F)$ 
7:      $\mathbf{u}_{i,g} \leftarrow \text{CRUCEBINOMIAL}(\mathbf{x}_{i,g}, \mathbf{v}_{i,g}, CR)$ 
8:      $\text{EVALUAR}(\mathbf{u}_{i,g})$ 
9:      $\mathbf{x}_{i,g+1} \leftarrow \text{SELECCIÓN}(\mathbf{x}_{i,g}, \mathbf{u}_{i,g})$ 
10:   end for
11:    $P_{g+1} \leftarrow \{\mathbf{x}_{i,g+1}\}_{i=1}^N$ 
12:    $g \leftarrow g + 1$ 
13: end while
14: return mejor solución encontrada

```

objetivo mediante cruce binomial y evalúa el vector de prueba resultante. Finalmente, ambos compiten mediante selección *greedy*, conservándose la mejor de las dos soluciones.

Operador de mutación

Para cada individuo objetivo $\mathbf{x}_{i,g}$, la estrategia implementada selecciona aleatoriamente tres índices r_0 , r_1 y r_2 , distintos entre sí y distintos de i . A partir de los vectores correspondientes se construye el vector mutante:

$$\mathbf{v}_{i,g} = \mathbf{x}_{r_0,g} + F(\mathbf{x}_{r_1,g} - \mathbf{x}_{r_2,g}),$$

donde F es el factor de escala. Este parámetro controla la amplitud de la perturbación diferencial aplicada sobre el vector base $\mathbf{x}_{r_0,g}$. La diferencia $\mathbf{x}_{r_1,g} - \mathbf{x}_{r_2,g}$ define una dirección de búsqueda obtenida a partir de la distribución actual de la población.

A diferencia de la mutación gaussiana empleada en AGE, esta perturbación no procede de una distribución independiente, sino de las diferencias entre individuos. Por ello, la generación de nuevos candidatos queda ligada al estado actual de la población.

Operador de cruce

El vector mutante $\mathbf{v}_{i,g}$ se combina con el individuo objetivo $\mathbf{x}_{i,g}$ mediante un **cruce binomial**. Para cada componente j , el vector de prueba $\mathbf{u}_{i,g}$ se define como

$$u_{i,j,g} = \begin{cases} v_{i,j,g} & \text{si } \text{rand}_j \leq CR \text{ o } j = j_{\text{rand}}, \\ x_{i,j,g} & \text{en otro caso.} \end{cases}$$

donde $CR \in [0, 1]$ es la probabilidad de cruce y rand_j es una variable aleatoria uniforme en el intervalo $[0, 1]$. La posición j_{rand} garantiza que el vector de prueba herede al menos una componente del vector mutante.

Este operador controla cuánta información procedente de la mutación diferencial se incorpora a la nueva solución. Valores altos de CR favorecen una mayor presencia de componentes del vector mutante, mientras que valores bajos conservan más componentes del individuo objetivo.

Operador de selección

Finalmente, el vector de prueba compite directamente con el individuo objetivo mediante un operador de selección. Como los problemas considerados son de minimización, el individuo que pasa a la siguiente generación se determina mediante

$$\mathbf{x}_{i,g+1} = \begin{cases} \mathbf{u}_{i,g} & \text{si } f(\mathbf{u}_{i,g}) \leq f(\mathbf{x}_{i,g}), \\ \mathbf{x}_{i,g} & \text{en otro caso.} \end{cases}$$

De este modo, cada vector de prueba reemplaza al individuo objetivo únicamente cuando iguala o mejora su calidad. El procedimiento se aplica a toda la población, manteniendo constante su tamaño y generando P_{g+1} a partir de P_g .

5.1.4. Success-History based Adaptive Differential Evolution (SHADE)

La tercera y última metaheurística considerada es **SHADE**, una variante adaptativa de DE propuesta por Tanabe y Fukunaga [22]. Este algoritmo mantiene la estructura general de DE, pero adapta sus parámetros de control F y CR durante la búsqueda mediante una memoria histórica.

Además, utiliza la estrategia de mutación **current-to-pbest/1** y mantiene un archivo externo de soluciones reemplazadas. El Algoritmo 3 resume el esquema implementado.

En cada generación, SHADE genera valores específicos de F_i y CR_i para cada individuo, construye un vector mutante mediante **current-to-pbest/1**, aplica cruce binomial y decide la supervivencia mediante selección *greedy*.

A continuación, se describen con mayor detalle los componentes distintivos de **SHADE**.

Algorithm 3 *Success-History based Adaptive Differential Evolution (SHADE)*.

```

1:  $P_0 \leftarrow \text{GENERARPOBLACIONINICIAL}()$ 
2:  $\text{EVALUAR}(P_0)$ 
3:  $M_F, M_{CR} \leftarrow \text{INICIALIZARMEMORIA}(H)$ 
4:  $A \leftarrow \emptyset$ 
5:  $g \leftarrow 0$ 
6: while no se cumple el criterio de parada do
7:    $S_F, S_{CR}, S_{\Delta f} \leftarrow \emptyset$ 
8:   for cada individuo objetivo  $\mathbf{x}_{i,g} \in P_g$  do
9:      $r_i \leftarrow \text{SELECCIONARINDICEMEMORIA}(H)$ 
10:     $F_i, CR_i \leftarrow \text{GENERARPARAMETROS}(M_F[r_i], M_{CR}[r_i])$ 
11:     $p_i \leftarrow \text{GENERARPROPORCIONPBEST}(N)$ 
12:     $\mathbf{v}_{i,g} \leftarrow \text{MUTACIONCURRENTTOPBEST}(P_g, A, i, F_i, p_i)$ 
13:     $\mathbf{u}_{i,g} \leftarrow \text{CRUCEBINOMIAL}(\mathbf{x}_{i,g}, \mathbf{v}_{i,g}, CR_i)$ 
14:     $\text{EVALUAR}(\mathbf{u}_{i,g})$ 
15:   end for
16:   for cada individuo objetivo  $\mathbf{x}_{i,g} \in P_g$  do
17:      $\mathbf{x}_{i,g+1} \leftarrow \text{SELECCIÓN}(\mathbf{x}_{i,g}, \mathbf{u}_{i,g})$ 
18:     if  $f(\mathbf{u}_{i,g}) < f(\mathbf{x}_{i,g})$  then
19:        $\Delta f_i \leftarrow |f(\mathbf{u}_{i,g}) - f(\mathbf{x}_{i,g})|$ 
20:        $A \leftarrow A \cup \{\mathbf{x}_{i,g}\}$ 
21:        $\text{REGISTRAREXITO}(S_F, S_{CR}, S_{\Delta f}, F_i, CR_i, \Delta f_i)$ 
22:     end if
23:   end for
24:    $A \leftarrow \text{LIMITARCHIVO}(A)$ 
25:   if  $S_F \neq \emptyset \wedge S_{CR} \neq \emptyset$  then
26:      $M_F, M_{CR} \leftarrow \text{ACTUALIZARMEMORIA}(M_F, M_{CR}, S_F, S_{CR}, S_{\Delta f})$ 
27:   end if
28:    $P_{g+1} \leftarrow \{\mathbf{x}_{i,g+1}\}_{i=1}^N$ 
29:    $g \leftarrow g + 1$ 
30: end while
31: return mejor solución encontrada

```

Adaptación histórica de parámetros

La memoria histórica está formada por los vectores M_F y M_{CR} , cada uno con H posiciones. Inicialmente, todas las posiciones se establecen a 0.5.

Durante cada generación, cada individuo selecciona una posición aleatoria de la memoria r_i y genera sus propios valores F_i y CR_i a partir de ella:

$$F_i \sim \text{Cauchy}(M_{F,r_i}, 0.1),$$

$$CR_i \sim \mathcal{N}(M_{CR,r_i}, 0.1).$$

Después se aplican las restricciones necesarias para mantener $F_i \in (0, 1]$ y $CR_i \in [0, 1]$. Cuando un vector de prueba mejora al individuo objetivo, los valores de F_i y CR_i utilizados se almacenan en los conjuntos S_F y S_{CR} . También se registra la mejora obtenida:

$$\Delta f_i = |f(\mathbf{u}_{i,g}) - f(\mathbf{x}_{i,g})|.$$

Al finalizar la generación, los valores de F_i y CR_i que han producido mejoras se utilizan para actualizar las memorias M_F y M_{CR} . De este modo, las generaciones posteriores tienden a muestrear parámetros próximos a los que han funcionado mejor durante la búsqueda.

Operador de mutación

La mutación de SHADE utiliza la estrategia **current-to-pbest/1**. Para cada individuo objetivo $\mathbf{x}_{i,g}$, se selecciona una solución $\mathbf{x}_{pbest,g}$ entre un porcentaje de los mejores individuos de la población. El vector mutante se construye como

$$\mathbf{v}_{i,g} = \mathbf{x}_{i,g} + F_i(\mathbf{x}_{pbest,g} - \mathbf{x}_{i,g}) + F_i(\mathbf{x}_{r_1,g} - \mathbf{x}_{r_2,g}). \quad (5.1)$$

El primer término diferencial dirige la búsqueda hacia soluciones de buena calidad, mientras que el segundo mantiene una componente exploratoria basada en diferencias entre individuos, similar a la de DE. El índice r_1 se selecciona de la población actual, mientras que r_2 puede proceder de la población o del archivo externo A .

Archivo externo y operadores de cruce y selección

El archivo externo A almacena soluciones reemplazadas durante la selección y se utiliza para aumentar la diversidad de las diferencias vectoriales.

Cuando su tamaño supera el de la población, se eliminan elementos aleatoriamente.

Una vez generado el vector mutante, SHADE aplica cruce binomial con la probabilidad CR_i , obteniendo el vector de prueba $\mathbf{u}_{i,g}$. Después se aplica la misma selección *greedy* que en DE:

$$\mathbf{x}_{i,g+1} = \begin{cases} \mathbf{u}_{i,g} & \text{si } f(\mathbf{u}_{i,g}) \leq f(\mathbf{x}_{i,g}), \\ \mathbf{x}_{i,g} & \text{en otro caso.} \end{cases}$$

Cuando la mejora es estricta, la solución reemplazada se incorpora al archivo externo y los parámetros empleados se registran para actualizar la memoria. Por tanto, SHADE conserva la estructura básica de DE, pero utiliza la información obtenida durante la búsqueda para adaptar sus parámetros en generaciones posteriores.

5.2. Mecanismo de reinicio elitista

Como se explicó en la Sección 3.2.1, una pérdida excesiva de diversidad puede llevar a la población a una fase de **estancamiento**, en la que se siguen realizando evaluaciones pero apenas se obtienen mejoras relevantes. Esta situación limita tanto la capacidad exploratoria de la metaheurística como la utilidad posterior de los modelos subrogados. Para reactivar la exploración en estos casos, se incorpora un mecanismo de **reinicio elitista** común a AGE, DE y SHADE.

A diferencia de un reinicio completo, el reinicio elitista conserva la mejor solución disponible y regenera aleatoriamente el resto de la población. Así, se recupera diversidad sin descartar por completo el progreso alcanzado durante la búsqueda.

La activación del reinicio se determina a partir de la evolución del **segundo mejor individuo** de la población. Esta decisión se debe a que, tras un reinicio, el mejor individuo suele quedar muy por encima del resto de la población. Si se utilizara como referencia, podrían producirse reinicios consecutivos antes de que la población tuviera tiempo de acercarse a él o de generar nuevas mejoras. En cambio, si el segundo mejor individuo no mejora durante un número suficiente de evaluaciones, se considera que la población ha perdido capacidad de avance. La Figura 5.1 representa este procedimiento.

Sea e la evaluación actual y sea $e_{\text{mejora}}^{(2)}$ la última evaluación en la que el segundo mejor individuo experimentó una mejora relevante. El reinicio se activa cuando

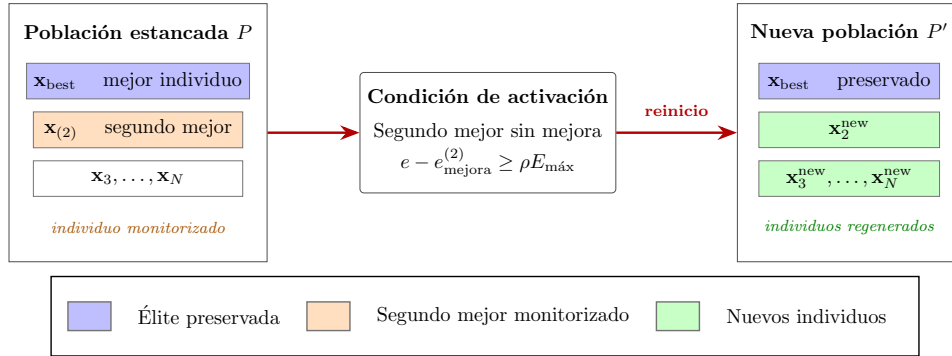


Figura 5.1: Representación esquemática del mecanismo de reinicio elitista.

$$e - e_{\text{mejora}}^{(2)} \geq \rho E_{\text{máx}}, \quad (5.2)$$

donde $E_{\text{máx}}$ representa el presupuesto total de evaluaciones y $\rho \in (0, 1)$ define la ventana de estancamiento. Para evitar activaciones debidas a pequeñas variaciones numéricas, solo se consideran mejoras superiores a una tolerancia mínima.

Cuando se activa, se conserva el mejor individuo de la población y se regenera aleatoriamente el resto dentro del dominio de búsqueda S . Después se actualizan las referencias utilizadas para detectar el estancamiento, evitando que el mecanismo vuelva a activarse inmediatamente con información de la etapa anterior. En el caso de SHADE, además, se vacía el archivo externo de soluciones reemplazadas A , ya que contiene información asociada a la población previa al reinicio.

El Algoritmo 4 resume las operaciones realizadas cuando se satisface el criterio de activación.

Algorithm 4 Mecanismo de reinicio elitista.

```

1: if  $e - e_{\text{mejora}}^{(2)} \geq \rho E_{\text{máx}}$  and  $E_{\text{máx}} - e \geq N - 1$  then
2:    $\mathbf{x}_{\text{best}} \leftarrow \text{SELECCIONARMEJORINDIVIDUO}(P)$ 
3:    $P_{\text{new}} \leftarrow \text{GENERARPOBLACIONALEATORIA}(N - 1, S)$ 
4:    $\text{EVALUAR}(P_{\text{new}})$ 
5:    $e \leftarrow e + N - 1$ 
6:    $P \leftarrow \{\mathbf{x}_{\text{best}}\} \cup P_{\text{new}}$ 
7:   if la metaheurística es SHADE then
8:      $A \leftarrow \emptyset$ 
9:   end if
10:   $\text{ACTUALIZARREFERENCIASDEESTANCAMIENTO}(P, e)$ 
11: end if

```

Las ventanas de estancamiento consideradas se detallan en el Capítulo 6.

5.3. Algoritmos considerados como técnicas de subrogado

En esta sección se describen los algoritmos que se utilizarán como técnicas de subrogado.

Se consideran aproximadores de distinta naturaleza, incluyendo algoritmos de aprendizaje automático, técnicas de interpolación y métodos estadísticos o matemáticos clásicos. Esta selección permite trabajar con diferentes niveles de flexibilidad, coste computacional y capacidad para representar relaciones no lineales. Las configuraciones de parámetros utilizadas en cada caso se concretan en el Capítulo 6.

5.3.1. Least Absolute Shrinkage and Selection Operator (LASSO)

LASSO es un modelo de regresión lineal regularizada propuesto por Tibshirani [23]. En este contexto, aproxima el valor de *fitness* como una combinación lineal de las variables de decisión:

$$\hat{f}(\mathbf{x}) = \beta_0 + \sum_{j=1}^D \beta_j x_j, \quad (5.3)$$

donde β_0 es el término independiente y β_j representa el peso asociado a la variable x_j .

LASSO utiliza una penalización L_1 como mecanismo de regularización. Esta penalización puede anular algunos coeficientes, reduciendo la complejidad del modelo. Por ello, se incluye como aproximador lineal de **bajo coste** e interpretación sencilla.

Su principal limitación aparece cuando la función objetivo presenta interacciones no lineales o una estructura multimodal difícil de representar mediante una relación lineal.

5.3.2. Response Surface Methodology (RSM)

La metodología de superficies de respuesta (*Response Surface Methodology*, RSM) consiste en aproximar la función objetivo mediante una superficie polinómica ajustada a partir de las muestras disponibles. Una superficie de grado máximo p puede expresarse como

$$\hat{f}(\mathbf{x}) = \sum_{|\alpha| \leq p} \beta_{\alpha} \mathbf{x}^{\alpha}, \quad (5.4)$$

donde α es un multiíndice que determina las potencias de las variables y β_{α} representa el coeficiente asociado a cada término.

Frente a LASSO, RSM amplía su capacidad expresiva al incorporar términos polinómicos e interacciones entre variables, manteniendo un **coste reducido**. Sin embargo, su eficacia disminuye cuando el paisaje de *fitness* es muy irregular o requiere polinomios de grado elevado.

5.3.3. Radial Basis Function (RBF)

Los modelos basados en funciones de base radial constituyen una técnica clásica de aproximación e interpolación. Su idea consiste en aproximar la función objetivo mediante una combinación ponderada de funciones radiales cuyo valor depende de la distancia entre la solución a predecir y un conjunto de centros [24]. De forma general,

$$\hat{f}(\mathbf{x}) = \sum_{i=1}^m w_i \phi(\|\mathbf{x} - \mathbf{c}_i\|), \quad (5.5)$$

donde $\mathbf{c}_i$ representa un centro, w_i su peso asociado y $\phi(\cdot)$ la función radial utilizada.

Este modelo permite capturar variaciones locales de la función objetivo a partir de la geometría de las muestras. Su comportamiento depende de la distribución de los datos, de la función radial utilizada y del número de centros considerados. Por ello, si se utilizan todos los centros disponibles, el coste computacional puede aumentar con el tamaño del conjunto de entrenamiento.

5.3.4. Support Vector Regression (SVR)

La regresión por vectores de soporte (*Support Vector Regression*, SVR) extiende las máquinas de vectores soporte a problemas de regresión [25]. Su objetivo es construir una función aproximadora que mantenga un equilibrio entre el error cometido sobre los datos de entrenamiento y la complejidad del modelo.

Una de sus características principales es la función de pérdida ε -insensible:

$$L_{\varepsilon}(y, \hat{f}(\mathbf{x})) = \max\{0, |y - \hat{f}(\mathbf{x})| - \varepsilon\}. \quad (5.6)$$

que establece un margen de tolerancia alrededor de la predicción, de modo que los errores inferiores a ε no se penalizan.

Mediante el uso de funciones *kernel*, puede representar relaciones no lineales sin transformar explícitamente el espacio de entrada. Su principal limitación es el coste de entrenamiento, que puede crecer de forma considerable con el número de muestras y dificultar su reajuste repetido durante la búsqueda.

5.3.5. Multilayer Perceptron (MLP)

El perceptrón multicapa (*Multilayer Perceptron*, MLP) es una red neuronal formada por una capa de entrada, una o varias capas ocultas y una capa de salida. Cada capa aplica una transformación sobre la representación anterior, lo que permite construir una aproximación no lineal de la función objetivo.

Para una capa oculta ℓ , la activación puede expresarse como

$$\mathbf{h}^{(\ell)} = \sigma^{(\ell)} \left(W^{(\ell)} \mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)} \right), \quad (5.7)$$

donde $W^{(\ell)}$ y $\mathbf{b}^{(\ell)}$ son los pesos y sesgos de la capa, y $\sigma^{(\ell)}$ es la función de activación. En la primera capa se toma $\mathbf{h}^{(0)} = \mathbf{x}$, mientras que la capa de salida produce la predicción $\hat{f}(\mathbf{x})$.

MLP se incluye como aproximador neuronal que no impone una forma fija sobre la función objetivo, siendo capaz de representar problemas más complejos que los modelos anteriores. A cambio de ello, su entrenamiento es sensible a la escala de los datos, al número de muestras disponibles y a la configuración de la arquitectura.

5.3.6. Random Forest (RF)

Random Forest es un modelo de ensamblado basado en combinar múltiples árboles de decisión contruidos sobre muestras y subconjuntos de variables seleccionados aleatoriamente [26].

En regresión, la predicción se obtiene promediando la salida de los T árboles que componen el ensamblado:

$$\hat{f}(\mathbf{x}) = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}), \quad (5.8)$$

donde $h_t(\mathbf{x})$ representa la predicción del árbol t .

Su principal ventaja es la robustez de las predicciones, aunque su interpretabilidad es menor que la de los modelos lineales o polinómicos. Además, su capacidad de extrapolación es limitada fuera de las regiones cubiertas por las muestras de entrenamiento.

5.3.7. Métodos de boosting basados en árboles

A diferencia de Random Forest, donde los árboles se construyen de forma independiente, los métodos de *gradient boosting* generan árboles de manera secuencial. Cada árbol nuevo intenta corregir los errores cometidos por el conjunto anterior, construyendo progresivamente un modelo aditivo.

De forma general, la predicción obtenida tras incorporar T árboles puede expresarse como

$$\hat{f}_T(\mathbf{x}) = \hat{f}_0(\mathbf{x}) + \sum_{t=1}^T \eta h_t(\mathbf{x}), \quad (5.9)$$

donde $\hat{f}_0(\mathbf{x})$ representa la estimación inicial, $h_t(\mathbf{x})$ es el árbol incorporado en la iteración t y η controla la contribución de cada nuevo estimador.

Esta familia de modelos se incluye como referencias de ensamblado secuencial frente al esquema paralelo utilizado por *Random Forest*.

Histogram-based Gradient Boosting (HGB)

HGB utiliza histogramas para agrupar los valores continuos de las variables antes de buscar puntos de división. Esto reduce el número de candidatos que deben evaluarse durante la construcción de los árboles y permite mantener un **coste de entrenamiento más reducido** cuando aumenta el número de muestras.

Extreme Gradient Boosting (XGBoost)

XGBoost es una implementación escalable de *tree boosting* propuesta por Chen y Guestrin [27]. Incorpora regularización explícita y distintas optimizaciones para acelerar el entrenamiento. Es un modelo flexible capaz de representar interacciones no lineales complejas, aunque a costa de una **mayor complejidad de configuración**.

5.3.8. Kriging

Kriging es una técnica clásica de modelado subrogado utilizada para aproximar la función objetivo a partir de un conjunto limitado de muestras evaluadas. A diferencia de otros regresores, no solo proporciona una predicción media del *fitness*, sino también una estimación de la incertidumbre asociada a dicha predicción [2].

Este método se construye mediante **procesos gaussianos**, representando la función objetivo como la suma de una tendencia global y un proceso estocástico correlacionado:

$$f(\mathbf{x}) = \mu(\mathbf{x}) + Z(\mathbf{x}). \quad (5.10)$$

En esta expresión, $\mu(\mathbf{x})$ representa la tendencia media y $Z(\mathbf{x})$ modela la variación respecto a dicha tendencia mediante una estructura de covarianza entre muestras.

Kriging se incluye como referencia clásica dentro del modelado subrogado para comparar con algoritmos modernos. Su principal limitación es el coste de ajuste, ya que requiere trabajar con matrices de covarianza cuyo tamaño aumenta con el número de muestras. Además, su comportamiento depende de la función de covarianza seleccionada y de sus parámetros.

5.4. Estrategias *offline* para la evaluación de modelos subrogados

Para evaluar el comportamiento de los modelos subrogados considerados, el primer análisis se realiza desde una perspectiva *offline*. En esta fase, las metaheurísticas se ejecutan de forma independiente y las soluciones evaluadas durante la búsqueda se almacenan junto con su valor real de *fitness*.

En este contexto, las predicciones del subrogado no intervienen todavía en la evolución de la población. El objetivo es analizar su comportamiento en condiciones controladas antes de estudiar su viabilidad *online*, donde sus estimaciones sí condicionan las decisiones de la metaheurística.

Como se explicó en el Capítulo 3, las metaheurísticas evolucionan la población a lo largo del proceso iterativo, por lo que las muestras no son independientes del tiempo al pertenecer a distintas fases de la búsqueda. Por ello, los históricos se organizan en **bloques temporales** y la validación se realiza siempre sobre muestras posteriores a las utilizadas en el entrenamiento. De este modo, el estudio no se reduce a comprobar si el modelo reproduce información ya observada, sino que permite valorar si puede anticipar el comportamiento de la búsqueda en etapas posteriores.

Bajo este marco se consideran dos estrategias: una **no acumulativa**, que utiliza un único bloque temporal como conjunto de entrenamiento, y una **acumulativa**, que incorpora progresivamente todos los bloques disponibles hasta el instante considerado.

5.4.1. Estrategia *offline* no acumulativa

En la estrategia no acumulativa, el modelo se entrena utilizando únicamente las muestras de un bloque temporal concreto, sin incorporar información de etapas anteriores.

Si B_t representa el bloque correspondiente al instante t , el conjunto de entrenamiento se define como

$$\mathcal{D}_{\text{train}}^{(t)} = B_t. \quad (5.11)$$

Con este enfoque se estudia si la información generada en una etapa concreta de la búsqueda es suficiente para estimar el comportamiento inmediatamente posterior de la metaheurística. Por tanto, el rendimiento del modelo depende de la calidad informativa del bloque utilizado para el entrenamiento.

La Figura 5.2 representa este procedimiento. En cada ajuste se utiliza un único bloque como entrenamiento y el bloque posterior se reserva para la validación.

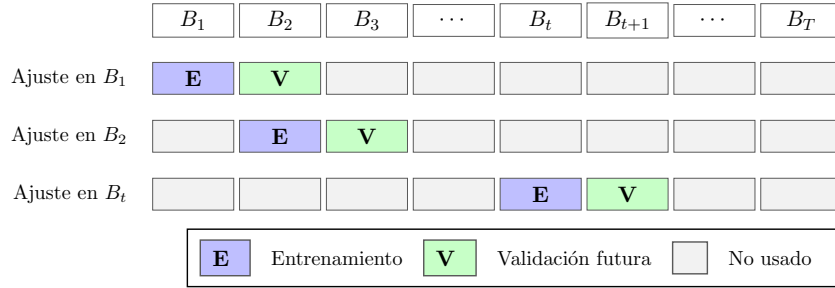


Figura 5.2: Representación esquemática de la estrategia *offline* no acumulativa.

5.4.2. Estrategia *offline* acumulativa

En la estrategia acumulativa, el modelo se entrena con todas las muestras disponibles hasta el bloque temporal considerado. A diferencia del enfoque anterior, la información previa no se descarta sino que se incorpora progresivamente al conjunto de entrenamiento.

Si $B_1, B_2, \dots, B_t$ representan los bloques disponibles hasta el instante t , el conjunto de entrenamiento se define como

$$\mathcal{D}_{\text{train}}^{(t)} = \bigcup_{k=1}^t B_k. \quad (5.12)$$

Este esquema permite disponer de un histórico más amplio para mejorar la capacidad del subrogado para anticipar la fase posterior de la búsqueda. Por contra, las muestras de fases anteriores pueden dejar de ser representativas de la etapa posterior que se quiere estimar. Además, el tamaño del conjunto de entrenamiento crece progresivamente, lo que puede aumentar el coste de ajuste de los modelos más sensibles al número de muestras disponibles.

La Figura 5.3 representa este procedimiento. En cada ajuste se incorporan todos los bloques disponibles hasta el instante considerado y el bloque posterior se reserva para la validación.

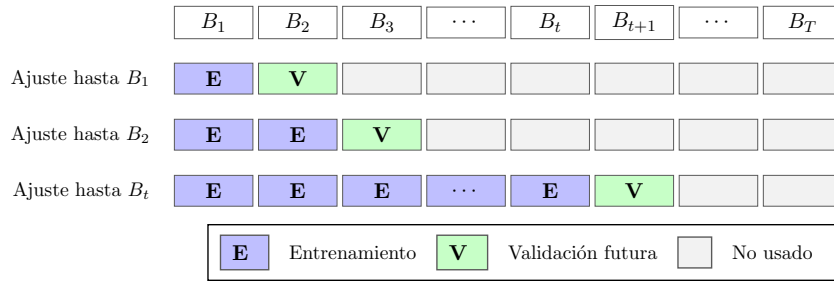


Figura 5.3: Representación esquemática de la estrategia *offline* acumulativa.

5.4.3. Validación sobre muestras futuras

En ambas estrategias, la validación se realiza sobre soluciones generadas en una etapa posterior a la utilizada para entrenar el modelo. Si B_t es el último bloque incorporado al conjunto de entrenamiento, el conjunto de validación se toma del bloque siguiente:

$$\mathcal{D}_{\text{val}}^{(t)} \subseteq B_{t+1}. \quad (5.13)$$

Así, se analiza si el subrogado es capaz de anticipar el comportamiento de la búsqueda a corto plazo. Además, simula la situación que se dará en la integración *online*, donde el modelo debe ser útil para guiar la búsqueda en las etapas posteriores de la optimización.

La forma concreta de construir estos bloques y de seleccionar las muestras de validación se especifica en el Capítulo 6.

5.5. Estrategia *online* para la integración del modelo subrogado

Una vez evaluados los modelos sobre históricos ya generados, se estudia su uso durante la propia ejecución de la metaheurística. En este escenario *online*, el subrogado deja de ser una herramienta de análisis posterior y pasa a intervenir dentro del bucle de optimización.

Dado que la evaluación de la función objetivo es el componente más costoso del proceso, la integración se plantea como un mecanismo de filtrado. Cada vez que la metaheurística genera un nuevo candidato, el subrogado estima su valor de *fitness*. Si la predicción indica que el candidato puede mejorar la población, se realiza la evaluación real. En caso contrario, se descarta sin consumir presupuesto de evaluaciones.

El filtro compara cada candidato con la solución frente a la que competiría en el mecanismo de reemplazo original. Esta referencia no es la misma en todas las metaheurísticas. En AGE, los descendientes compiten contra el peor individuo de la población; en DE y SHADE, cada vector de prueba compete contra su individuo padre.

Por tanto, sea $\mathbf{x}$ un candidato generado por la metaheurística y sea f_{ref} el *fitness* real de su solución de referencia. En un problema de minimización, el candidato se considera prometedor cuando

$$\hat{f}(\mathbf{x}) < f_{\text{ref}}. \quad (5.14)$$

Si se cumple esta condición, el candidato se evalúa con la función objetivo real. En caso contrario, se descarta antes de consumir una evaluación. De este modo, el subrogado decide qué candidatos pasan a evaluación real, y la función objetivo solo actúa como criterio definitivo sobre aquellos que superan el filtro.

La Figura 5.4 resume el uso del subrogado como filtro previo a la evaluación real.

Esta estrategia hace que el **orden relativo** entre soluciones sea especialmente relevante. El subrogado debe distinguir qué candidatos parecen mejores que la referencia, aunque no prediga exactamente su valor de *fitness*. La forma concreta de aplicar este filtro depende de decisiones experimentales como la proporción de candidatos filtrados, la disponibilidad inicial del modelo y su actualización durante la búsqueda. Estos aspectos se definen en el Capítulo 6 y se analizan en el Capítulo 7.

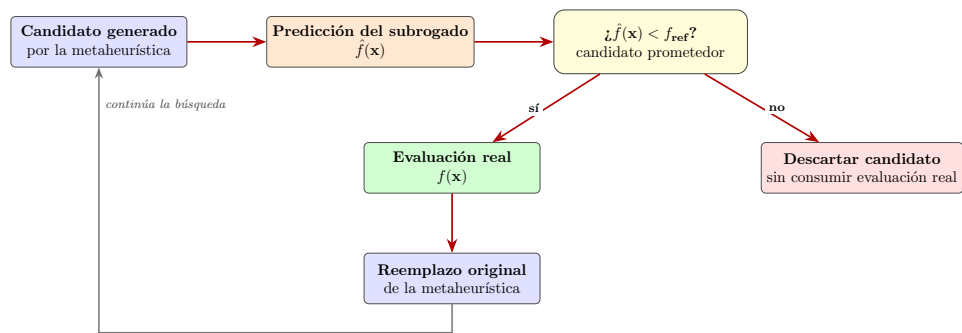


Figura 5.4: Esquema de integración *online* del modelo subrogado como filtro previo a la evaluación real.

Capítulo 6

Diseño experimental

En este capítulo se recoge el protocolo experimental seguido para evaluar las técnicas subrogadas consideradas en este trabajo y su integración en el bucle de optimización.

Para ello, se presenta en primer lugar el benchmark utilizado y se detallan las condiciones bajo las que se ejecutan las metaheurísticas. A continuación, se describen las técnicas subrogadas consideradas y las transformaciones aplicadas a los datos antes de su entrenamiento. Posteriormente, se definen las estrategias de evaluación, las métricas empleadas y el criterio seguido para comparar los modelos. Finalmente, se documentan de forma separada los detalles de implementación y el entorno necesario para reproducir las ejecuciones.

6.1. Benchmark utilizado

La evaluación experimental de una metaheurística requiere disponer de un conjunto de problemas que permita comparar su comportamiento bajo condiciones homogéneas. En este trabajo se utiliza la suite **CEC2017**, propuesta para la *Special Session and Competition on Single Objective Bound Constrained Real-Parameter Numerical Optimization* del IEEE Congress on Evolutionary Computation de 2017 [28].

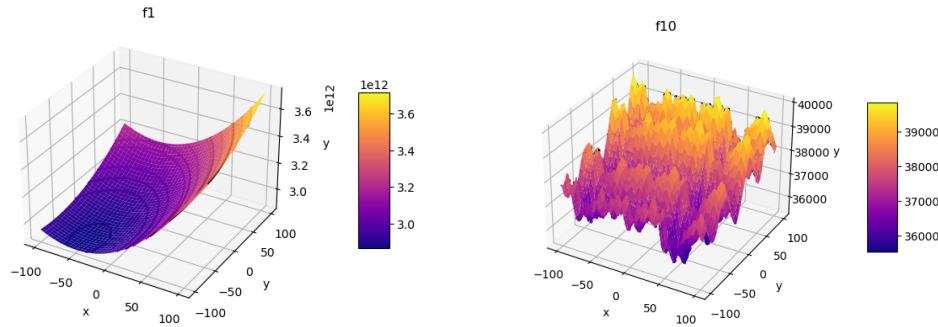
6.1.1. Suite CEC2017

CEC2017 está formada por problemas de optimización continua, monoobjetivo y con variables acotadas. Cada solución candidata se representa mediante un vector $\mathbf{x} \in [-100, 100]^D$, donde D indica la dimensión del problema.

Las funciones que la componen plantean escenarios de optimización con distintos niveles de dificultad y se agrupan en cuatro familias según las características de su paisaje de búsqueda:

- **Funciones unimodales:** permiten analizar la capacidad de convergencia de los algoritmos en problemas con una única región óptima principal.
- **Funciones multimodales:** incorporan múltiples óptimos locales, útiles para analizar la capacidad exploratoria y la robustez de las metaheurísticas frente a la convergencia prematura.
- **Funciones híbridas:** combinan diferentes componentes dentro de un mismo problema, dando lugar a paisajes más heterogéneos.
- **Funciones compuestas:** integran varias funciones y presentan una estructura de búsqueda más compleja.

La Figura 6.1 ilustra esta diferencia mediante la representación de dos funciones con características distintas.



(a) f_1 : función unimodal *Bent Cigar*.

(b) f_{10} : función multimodal *Schwefel*.

Figura 6.1: Representación ilustrativa de dos paisajes de búsqueda pertenecientes a la suite CEC2017.

La suite fue definida inicialmente con **30 funciones**, aunque revisiones posteriores eliminaron f_2 debido a su inestabilidad numérica. En este trabajo, consideramos la suite completa ya que es la expuesta por la implementación empleada.

Familia	Funciones
Unimodales	f_1-f_3
Multimodales simples	f_4-f_{10}
Híbridas	$f_{11}-f_{20}$
Compuestas	$f_{21}-f_{30}$

Tabla 6.1: Familias de funciones de CEC2017 según la numeración empleada en este trabajo.

Los detalles concretos de la integración software del benchmark y de la interfaz de evaluación empleada se recogen en la Sección 6.9.

6.2. Estrategias de resolución: Metaheurísticas

Las metaheurísticas AGE, DE y SHADE, descritas en el Capítulo 5, se ejecutan bajo las mismas condiciones con el objetivo de obtener historiales de evaluaciones comparables. Todas las pruebas se realizan en dimensión $D = 10$, sobre el dominio de búsqueda $[-100, 100]^{10}$, y utilizan un tamaño de población fijo de **50 individuos**.

El criterio de parada se establece mediante el número de evaluaciones de la función objetivo. Siguiendo la definición propia de CEC2017, cada ejecución dispone de un presupuesto máximo de $10000 \cdot D$ evaluaciones. Por tanto, al considerar $D = 10$, el límite se fija en 100000 evaluaciones por ejecución.

Dado que las metaheurísticas consideradas incorporan **componentes aleatorias**, una única ejecución no resulta suficiente para valorar su comportamiento. Por este motivo, cada combinación de función y metaheurística se ejecuta un total de **51 veces**.

Parámetro	Valor
Dimensión del problema	$D = 10$
Dominio de búsqueda	$[-100, 100]^{10}$
Presupuesto máximo de evaluaciones	100000
Tamaño de la población	50
Número de ejecuciones independientes	51

Tabla 6.2: Parámetros comunes utilizados en la ejecución de las metaheurísticas.

A partir del entorno de evaluación definido, cada metaheurística man-

tiene la configuración propia de su esquema de búsqueda. Los operadores y particularidades de los algoritmos se desarrollaron en el Capítulo 5, por lo que a continuación se recogen únicamente los valores utilizados durante la generación del histórico de evaluaciones.

1. **AGE:** para la implementación propia se utilizan los parámetros recogidos en la Tabla 6.3.

Parámetro	Valor	Parámetro	Valor
Tamaño del torneo	2	Probabilidad de cruce	0.7
Probabilidad de mutación	0.1	Parámetro α de BLX- α	0.45
Desviación típica de la mutación	0.3		

Tabla 6.3: Parámetros utilizados para AGE.

2. **DE:** se emplea la variante DE/rand/1/bin por defecto de la implementación empleada (véase en la Tabla 6.4).

Parámetro	Valor	Parámetro	Valor
Estrategia de mutación	DE/rand/1	Operador de cruce	Binomial
Factor de escala F	0.5	Probabilidad de cruce CR	0.9

Tabla 6.4: Parámetros utilizados para DE.

3. **SHADE:** se mantiene la configuración por defecto de la implementación empleada (véase en la Tabla 6.5).

Parámetro	Valor	Parámetro	Valor
Estrategia de mutación	current-to-pbest/1	Operador de cruce	Binomial
Factor de escala F	Adaptativo	Probabilidad de cruce CR	Adaptativa
Tam. de la mem. histórica	100		

Tabla 6.5: Parámetros utilizados para SHADE.

6.2.1. Criterio de reinicio elitista

Para identificar situaciones de **estancamiento**, se utilizan gráficas de convergencia y diversidad. Si se detecta este comportamiento, se incorpora el mecanismo de reinicio descrito en el Capítulo 5, parametrizado mediante una ventana de estancamiento ρ que expresa, como **proporción del presupuesto total de evaluaciones**, el tiempo sin mejoras a partir del cual se considera que el algoritmo ha convergido prematuramente.

Tal y como está definido, es necesario definir el criterio de activación, y por tanto, determinar cuándo una variación del segundo mejor individuo se corresponde con una mejora efectiva. Dado que CEC2017 considera nulos los valores por debajo de 10^{-8} , cualquier variación de *fitness* de menor orden corresponde a ruido numérico. Para descartarlas sin filtrar progresos reales, se contabilizan como mejora únicamente aquellas variaciones cuya magnitud supere una **tolerancia** de 10^{-6} .

El impacto de este mecanismo sobre la búsqueda se evalúa con un conjunto de configuraciones del criterio frente a la versión *base* sin reinicio, correspondiente a valores de ρ del 1 %, 3 %, 5 %, 7 % y 10 %. Para cada metaheurística:

Configuración	Descripción	Ventana de estancamiento
ALG	Sin reinicio	—
ALG-1	Reinicio con $\rho = 0.01$	1.000 evaluaciones
ALG-3	Reinicio con $\rho = 0.03$	3.000 evaluaciones
ALG-5	Reinicio con $\rho = 0.05$	5.000 evaluaciones
ALG-7	Reinicio con $\rho = 0.07$	7.000 evaluaciones
ALG-10	Reinicio con $\rho = 0.10$	10.000 evaluaciones

Tabla 6.6: Configuraciones genéricas de reinicio consideradas para cada metaheurística, donde **ALG** representa AGE, DE o SHADE.

El análisis se realiza de forma **independiente** por cada algoritmo ya que el objetivo no es comparar AGE, DE y SHADE entre sí, sino determinar qué configuración de cada uno resulta más adecuada para generar un historial de evaluaciones útil en la fase subrogada. Para ello, se utiliza el **ranking medio**, agregando los resultados sobre todas las funciones sin que las de mayor magnitud de error distorsionen la agregación.

6.3. Registro temporal de evaluaciones

Durante la ejecución de las metaheurísticas se registran cronológicamente las evaluaciones reales realizadas sobre la función objetivo.

En lugar de analizar únicamente el resultado final de cada ejecución, este trabajo utiliza la secuencia temporal de soluciones evaluadas para estudiar cómo evoluciona la información disponible a lo largo de la búsqueda y cómo puede aprovecharse posteriormente en la fase de evaluación subrogada sobre el histórico de datos.

Para cada evaluación se registra el vector de decisión $\mathbf{x} \in \mathbb{R}^D$, el valor real de *fitness* obtenido al evaluar la función objetivo y un **identificador**

temporal que indica el orden en el que se generó la muestra. La Tabla 6.7 resume la información registrada y su utilidad posterior.

Campo	Descripción
<code>eval_id</code>	Identificador que refleja el orden cronológico de evaluación para la posterior partición en bloques temporales.
<code>x</code>	Vector de decisión asociado a la solución candidata.
<code>fitness</code>	Valor real obtenido al evaluar la función objetivo.

Tabla 6.7: Información registrada para cada evaluación.

A partir de este histórico de evaluaciones, cada ejecución se organiza posteriormente en bloques temporales que permiten construir los conjuntos de entrenamiento y validación empleados en la evaluación posterior de los modelos subrogados.

6.4. Técnicas subrogadas: modelos candidatos y de referencia

Los modelos empleados en este trabajo se presentaron conceptualmente en el Capítulo 5. Esta sección se centra en cómo se preparan los datos antes de entrenar cada modelo y qué configuración concreta se utiliza en la Sección 7.2.2.

6.4.1. Configuración experimental de los modelos

Como se comentó en la Sección 3.4, Kriging y PCE se incorporan como técnicas clásicas de referencia, por lo que no forman parte del mismo protocolo experimental aplicado inicialmente al resto de regresores.

Para garantizar una comparación no sobreajustada a los problemas de optimización de CEC2017, metaheurística o semilla, se mantienen algunos parámetros en sus valores por defecto y se modifican los que sí merecen ser ajustados para evitar comportamientos sesgados. La configuración no predeterminada empleada en cada modelo se recoge en la Tabla 6.8.

6.4.2. Preprocesamiento de los datos

Antes de entrenar cada modelo, los datos generados por las distintas metaheurísticas requieren dos **transformaciones independientes** que afectan, respectivamente, a las variables de entrada y a la variable objetivo.

Modelo	Configuración no predeterminada
LASSO	alpha=0.01 max_iter=5000 random_state=42
RSM	PolynomialFeatures sobre regresión lineal ordinaria. Configuración por defecto
RBF	neighbors=50 smoothing=1 kernel=gaussian degree=-1 epsilon=1.0
SVR	Configuración por defecto
MLP	hidden_layer_sizes=(128, 64) alpha=1e-3 max_iter=2000 random_state=42 early_stopping=True
Random Forest	n_estimators=200 max_depth=16 min_samples_leaf=5 max_features=0.5 random_state=42 n_jobs=-1
HGB	max_iter=200 learning_rate=0.05 max_depth=4 l2_regularization=1e-3 random_state=42
XGBoost	n_estimators=400 max_depth=4 learning_rate=0.05 subsample=0.8 colsample_bytree=0.8 random_state=42 n_jobs=-1 tree_method=hist
Kriging	corr=matern52
PCE	order=3

Tabla 6.8: Configuración inicial empleada para la evaluación de los modelos subrogados.

Escalado de las variables de entrada. Para mantener un protocolo homogéneo y robusto entre modelos, **todas las técnicas** se entrenan con las variables de entrada escaladas. Esta transformación es especialmente importante en técnicas sensibles a la magnitud de las variables, como LASSO, MLP, SVR, RBF y RSM. Aunque no es estrictamente necesario para el resto, se mantiene también en estos casos para mantener la consistencia del procedimiento experimental. Dado que en **CEC2017** las variables de decisión se definen sobre el dominio $[-100, 100]^D$, todas las muestras se reescalan al intervalo $[-1, 1]^D$ mediante la transformación

$$x_j^* = \frac{x_j}{100}.$$

De este modo, se evitan diferencias de escala innecesarias entre variables y se aplica el mismo preprocesamiento a todos los modelos considerados.

Escalado de la variable objetivo. En algunos modelos, la escala de la variable objetivo puede influir directamente en el ajuste. Por este motivo, se estandariza el valor de *fitness* únicamente en aquellos modelos donde resulta necesario: LASSO, RSM, RBF, SVR y MLP. En estos casos, y a partir de las formulaciones matemáticas de cada modelo definidas en el Capítulo 5, la magnitud de y puede afectar a la regularización, al condicionamiento numérico o al proceso de optimización interno del modelo.

Para los modelos en los que sí se estandariza la variable objetivo, se utiliza la transformación

$$y^* = \frac{y - \mu_y}{\sigma_y}, \quad \mu_y = \frac{1}{n} \sum_{i=1}^n y_i, \quad \sigma_y = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \mu_y)^2}.$$

Los valores μ_y y σ_y se calculan **exclusivamente sobre el conjunto de entrenamiento**, evitando así filtraciones de información (*data leakage*) desde el conjunto de validación. Tras la predicción, el resultado se devuelve a la escala original del problema mediante la **transformación inversa**, de forma que todos los modelos se comparan sobre los valores reales de *fitness*.

6.5. Métricas empleadas

Para evaluar el rendimiento de los modelos subrogados se utilizan distintas métricas que podemos agrupar en tres familias complementarias: el **error de predicción**, la **capacidad de ordenación** y el **coste computacional**.

6.5.1. Error de predicción

La capacidad de predicción de un modelo determina su calidad como regresor. Para cuantificarla se emplean el **error absoluto medio** (MAE) y la **raíz del error cuadrático medio** (RMSE), junto con sus correspondientes versiones normalizadas.

Dado un conjunto de validación formado por n pares $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, donde y_i es el valor real del *fitness* y $\hat{f}(\mathbf{x}_i)$ la predicción del modelo, ambas métricas se definen como

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{f}(\mathbf{x}_i)|, \quad (6.1)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(\mathbf{x}_i))^2}. \quad (6.2)$$

La diferencia entre ambas es la **sensibilidad al penalizar errores de mayor magnitud**. A diferencia del MAE, el RMSE penaliza con mayor intensidad **casos atípicos**, por lo que resulta muy útil para detectar fallos puntuales relevantes.

Dado que las funciones de CEC2017 presentan rangos de *fitness* muy distintos, los valores absolutos de MAE y RMSE no resultan comparables entre funciones ni entre bloques temporales de una mismo histórico de evaluaciones. Para facilitar su interpretación, ambas se normalizan dividiéndolas por la media de los valores absolutos del *fitness* real en el conjunto de validación, denotada como $|\bar{y}| = \frac{1}{n} \sum_{i=1}^n |y_i|$:

$$\text{NMAE} = \frac{\text{MAE}}{|\bar{y}|}, \quad (6.3)$$

$$\text{NRMSE} = \frac{\text{RMSE}}{|\bar{y}|}. \quad (6.4)$$

En este caso, los valores próximos a cero indican que las predicciones se mantienen cercanas a los valores reales de *fitness*.

6.5.2. Capacidad de ordenación

Como se expuso en la Sección ??, la utilidad de un modelo subrogado dentro de una metaheurística no depende únicamente de su precisión numérica. Un modelo subrogado puede realizar predicciones que no son

precisas pero puede ser capaz de ordenar correctamente las soluciones candidatas.

Para evaluar esta propiedad se emplea el **coeficiente de correlación de Spearman**, que cuantifica el grado de concordancia ordinal entre los valores reales y los predichos.

El coeficiente se calcula como la correlación de Pearson aplicada a los rangos de ambas distribuciones. Sea R_i el rango del valor real del *fitness* de la muestra i dentro del conjunto de validación, y Q_i el rango de la predicción correspondiente. Entonces:

$$\rho_s = \frac{\sum_{i=1}^n (R_i - \bar{R})(Q_i - \bar{Q})}{\sqrt{\sum_{i=1}^n (R_i - \bar{R})^2 \cdot \sum_{i=1}^n (Q_i - \bar{Q})^2}}, \quad (6.5)$$

donde $\bar{R}$ y $\bar{Q}$ son los rangos medios.

El coeficiente toma valores en $[-1, 1]$. Un valor 1 indica concordancia perfecta de orden, -1 refleja un orden completamente invertido, y 0 señala ausencia de relación ordinal.

Cuando dos o más valores coinciden, se les asignan rangos consecutivos según su orden de aparición en el vector correspondiente. De este modo, incluso si un subrogado genera predicciones constantes, se mantiene una ordenación total determinista y el coeficiente puede calcularse de forma consistente entre ejecuciones.

6.5.3. Coste computacional

La viabilidad de un modelo subrogado para la hibridación también depende de su coste computacional:

- **Tiempo de entrenamiento:** tiempo necesario, en segundos, para ajustar el modelo sobre el conjunto de entrenamiento.
- **Tiempo de predicción:** tiempo requerido, en segundos, para evaluar el modelo sobre las muestras del conjunto de validación.

Un modelo con un tiempo de entrenamiento elevado puede no ser adecuado si debe reajustarse varias veces durante la ejecución. Del mismo modo, un tiempo de predicción alto reduce la ventaja de usar el subrogado para filtrar candidatos antes de realizar evaluaciones reales. Por tanto, ambos deben analizarse conjuntamente para identificar configuraciones cuyo coste pueda comprometer su posterior integración.

6.5.4. Criterio de comparación de los modelos

A lo largo de este trabajo, la comparación entre modelos se realiza analizando conjuntamente las métricas anteriores. Estas se presentan a continuación en orden de importancia:

1. **Coefficiente de Spearman:** permite valorar si el subrogado preserva el orden relativo de calidad entre las soluciones candidatas.
2. **Coste computacional:** analiza la viabilidad para la posterior hibridación.
 - a) **Tiempo de predicción**
 - b) **Tiempo de entrenamiento**
3. **NMAE y NRMSE:** se emplean como métricas complementarias para evaluar la calidad predictiva ya que no es el objetivo de este trabajo.

6.6. Evaluación de modelos subrogados sobre históricos de evaluaciones

Los datos de búsqueda generados por las metaheurísticas se organizan en **cinco bloques temporales consecutivos**, cada uno de ellos correspondiente al **20 % del presupuesto total de evaluaciones**. Dado que cada ejecución comprende 100 000 evaluaciones, cada bloque agrupa 20 000 muestras, ordenadas cronológicamente según el instante en que fueron generadas durante la búsqueda. Los bloques se identifican por el intervalo porcentual que ocupan dentro de las evaluaciones realizadas: **1–20 %**, **21–40 %**, **41–60 %**, **61–80 %** y **81–100 %**.

A partir de esta partición, las dos estrategias definidas en la Sección 5.4 se instancian en este trabajo mediante los conjuntos de entrenamiento recogidos en la Tabla 6.9. La **estrategia no acumulativa** utiliza un único bloque en cada ajuste, mientras que la **acumulativa** incorpora progresivamente todos los bloques disponibles hasta el instante considerado.

Bloque	No acumulativa		Acumulativa		Bloque de validación
	Intervalo	Tamaño entrenamiento	Intervalo	Tamaño entrenamiento	
B_1	1–20 %	20 000	1–20 %	20 000	B_2 (21–40 %)
B_2	21–40 %	20 000	1–40 %	40 000	B_3 (41–60 %)
B_3	41–60 %	20 000	1–60 %	60 000	B_4 (61–80 %)
B_4	61–80 %	20 000	1–80 %	80 000	B_5 (81–100 %)
B_5	81–100 %	Reservado exclusivamente para validación			

Tabla 6.9: Resumen de las particiones utilizadas para construir los conjuntos de entrenamiento y validación en ambas estrategias.

Como se observa en la tabla, el bloque 5 queda reservado exclusivamente para validación al no haber más muestras sobre las que validar el modelo si utilizase dicho bloque como entrenamiento.

Para que los resultados sean comparables y no dependan del tamaño del conjunto de entrenamiento empleado, el conjunto de validación se toma siempre del bloque temporal inmediatamente posterior al último bloque utilizado para entrenar el modelo. Como cada bloque contiene 20000 evaluaciones, se selecciona el **25 % de sus muestras**, es decir, **5000 soluciones** evaluadas por ejecución. Este criterio se aplica tanto en la estrategia acumulativa como en la no acumulativa, de forma que el tamaño del conjunto de validación no depende del número de bloques empleadas en el entrenamiento.

La selección de las muestras se realiza de forma determinista a partir de la semilla de la ejecución y del bloque temporal utilizado para validación. De esta forma, para una misma ejecución y un mismo bloque, el conjunto de validación queda fijado y es común a todos los modelos y estrategias evaluados.

Dado el coste computacional de evaluar todas las combinaciones de modelos, estrategias de entrenamiento y semillas sobre la suite completa, la primera comparación se realiza sobre un subconjunto de funciones de CEC2017. En esta fase se mantienen las 51 semillas empleadas en las ejecuciones de las metaheurísticas, pero se reduce el número de funciones para hacer viable la comparación inicial entre modelos.

El subconjunto se elige para cubrir las principales familias de funciones del benchmark:

- **Unimodal:** f_1 .
- **Multimodal:** f_4 , f_{10} .
- **Híbrida:** f_{12} , f_{18} .

- **Compuesta:** f_{22} , f_{29} .

Con esta selección se comparan los modelos en funciones con características distintas, sin elevar innecesariamente el coste experimental. Las configuraciones seleccionadas en esta fase se evalúan posteriormente sobre la suite completa.

6.7. Ajuste fino de hiperparámetros

El ajuste fino de hiperparámetros se plantea como una fase posterior a la decisión de la mejor estrategia y la comparación inicial de modelos. Su objetivo no es repetir la comparación completa entre todos los regresores, sino refinar la configuración de aquellos que hayan obtenido mejores resultados antes de evaluarlos sobre la suite completa. De esta forma, el coste experimental se concentra en el modelo o modelos con mayor probabilidad de resultar útiles en una posterior integración.

Dado que el coste de entrenamiento y predicción no es el mismo para todos los modelos, el número de semillas usado en esta fase puede variar según la técnica considerada. Cuando dicho coste sea elevado, el ajuste se realizará sobre un subconjunto fijo de semillas para que la búsqueda de hiperparámetros sea asumible. Esta reducción afecta únicamente a la fase de ajuste, ya que la configuración o configuraciones finalmente seleccionadas se evalúan posteriormente sobre las 30 funciones de CEC2017 y las 51 semillas empleadas para el histórico de evaluaciones.

El procedimiento mantiene el orden temporal de los datos generados por cada metaheurística. Para evitar que la elección de hiperparámetros utilice información posterior al bloque de entrenamiento (*data leakage*), se aplica una partición interna dentro de dicho bloque:

- El **80 % inicial** de las muestras del bloque se utiliza para el entrenamiento de las distintas configuraciones candidatas que componen el espacio de búsqueda.
- El **20 % final** de dicho bloque se reserva como conjunto de validación interna para evaluar el rendimiento de cada configuración y estimar su capacidad de generalización dentro de la misma etapa temporal.

Una vez elegida la configuración, el modelo se reentrena con todas las muestras del bloque de entrenamiento y se evalúa sobre un subconjunto del bloque temporal inmediatamente posterior (conjunto de validación). Así, las muestras futuras no intervienen en la selección de hiperparámetros, sino solo en la evaluación del modelo ya ajustado.

La Figura 6.2 representa este procedimiento para una transición concreta. En el ejemplo, el ajuste interno se realiza sobre el bloque correspondiente al intervalo 20–40 % de la búsqueda y la evaluación posterior utiliza muestras pertenecientes al intervalo 40–60 %.

Entrenamiento 20–40 %

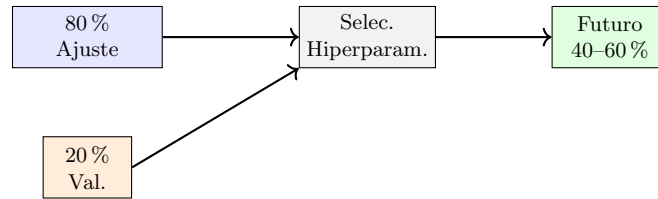


Figura 6.2: Esquema de validación interna para el ajuste de hiperparámetros con los conjuntos de ajuste y validación dispuestos en paralelo.

La búsqueda de hiperparámetros se realiza de forma exhaustiva sobre un espacio acotado de valores definido y la selección final sigue el criterio definido en la Subsección 6.5.4.

Las configuraciones consideradas para los modelos finalistas se presentan junto con sus resultados en el capítulo siguiente.

6.8. Integración del modelo subrogado durante la búsqueda

La integración del modelo subrogado durante la búsqueda se evalúa mediante el mecanismo de filtrado definido en la Sección 5.5. En esta fase, a diferencia de la evaluación sobre históricos de evaluaciones, las predicciones del subrogado pueden modificar las decisiones de la metaheurística. Por tanto, la comparación ya no se limita a medir la calidad de ordenación del modelo, sino que trata la hibridación como una variante del algoritmo original.

Para mantener acotado el coste experimental, la integración no se aplica a todos los modelos candidatos. Solo se considera el modelo y la estrategia de entrenamiento seleccionados tras la evaluación previa y el ajuste de hiperparámetros. El objetivo es comprobar si el subrogado elegido mejora el comportamiento de la metaheurística cuando interviene directamente en el proceso de búsqueda, tomando como referencia la versión original sin subrogado.

A partir de este esquema, el protocolo de integración queda definido por las siguientes decisiones experimentales:

1. **Periodo inicial de calentamiento.** El filtro subrogado no se activa desde el inicio, ya que primero es necesario disponer de evaluaciones reales para entrenar el primer modelo y que pueda generar predicciones útiles. Si $E_{\text{máx}}$ es el presupuesto total y α_{warm} la proporción reservada a esta fase, el número de evaluaciones iniciales es

$$E_{\text{warm}} = \alpha_{\text{warm}} E_{\text{máx}}.$$

En coherencia con la división temporal utilizada en la evaluación previa, este periodo inicial se fija en el 20 % del presupuesto total de evaluaciones ($\alpha_{\text{warm}} = 0.20$).

2. **Conjunto de entrenamiento.** El modelo se entrena únicamente con evaluaciones reales ya observadas. La construcción de este conjunto mantiene el orden temporal de la búsqueda y queda determinada por la estrategia de entrenamiento seleccionada previamente.
3. **Frecuencia de actualización.** Para evitar reentrenamientos excesivamente frecuentes que supongan un coste computacional excesivo o modelos desfasados, se estudian distintas frecuencias de actualización expresadas como proporción del conjunto temporal de entrenamiento. Los valores considerados son 0.25, 0.50 y 0.75.
4. **Probabilidad de uso del filtro.** El subrogado no se consulta necesariamente para todos los candidatos generados. Para cada candidato se extrae $u \sim \mathcal{U}(0, 1)$, y el filtro se aplica solo si

$$u < p_{\text{sub}},$$

donde p_{sub} controla la proporción esperada de candidatos sometidos a preevaluación subrogada. Cuando esta condición no se cumple, el candidato sigue el flujo de evaluación real de la metaheurística original. Se consideran los valores $p_{\text{sub}} \in \{0.25, 0.50, 0.75\}$.

5. **Tratamiento tras reinicio.** Después de un reinicio, la distribución de soluciones puede cambiar de forma brusca, reduciendo temporalmente la representatividad del modelo. Por ello, se evalúa un periodo de enfriamiento durante el cual el filtro permanece desactivado y solo se incorporan evaluaciones reales. Se consideran 50, 100 y 500 evaluaciones reales, equivalentes a 1, 2 y 10 poblaciones generadas para $N = 50$. No se escoge un mayor número de evaluaciones ya que podría ocurrir que el modelo se apagase durante gran parte del proceso de optimización.

Al tratarse como variantes de las metaheurísticas originales, las ejecuciones híbridas mantienen las condiciones generales definidas en la Tabla 6.2.

No obstante, para la calibración de los parámetros de integración se emplea un subconjunto reducido de funciones y semillas, distinto al utilizado en las fases previas. Esta fase no busca una optimización exhaustiva, sino tratar de seleccionar una configuración homogénea para las distintas metaheurísticas. Por ello, las decisiones experimentales se evalúan sobre las funciones f_3 , f_5 , f_9 , f_{15} , f_{19} , f_{24} y f_{27} , junto con 10 semillas independientes no utilizadas en la selección del modelo.

La comparación entre variantes sigue el criterio agregado definido en la Sección 6.2, basado en rankings medios sobre CEC2017. Además, se consideran contadores propios de la integración, como el número de reentrenamientos, el tiempo dedicado al modelo y el porcentaje de candidatos rechazados por el filtro.

Una vez seleccionada la configuración, la evaluación final se realiza sobre las 30 funciones de CEC2017 y 51 semillas nuevas, distintas tanto de las usadas en la calibración como de las empleadas en la evaluación previa. En esta fase, la variante híbrida se compara con la metaheurística original sin intervención subrogada. También se incluyen como referencia las hibridaciones basadas en Kriging y PCE, descritas en la Sección ??.

Los resultados de las distintas decisiones experimentales y las comparaciones descritas se presentan en el Capítulo 7.

6.9. Detalles de la implementación

A diferencia de las secciones anteriores, centradas en definir el protocolo experimental, esta sección recoge las principales decisiones técnicas tomadas para implementar y reproducir los experimentos.

6.9.1. Bibliotecas y herramientas empleadas

La implementación de este trabajo se ha desarrollado en Python, al tratarse de un lenguaje de programación ampliamente utilizado en computación científica, optimización numérica y aprendizaje automático. Gracias a la gran variedad de bibliotecas consolidadas en este ámbito, se homogeniza la integración de las metaheurísticas, modelos subrogados y herramientas de análisis, reduciendo la necesidad de implementar estas componentes desde cero y minimizando posibles errores en el desarrollo.

La integración del benchmark se realiza a partir del repositorio mantenido por Daniel Molina¹, basado en el código de referencia de la competición CEC2017 y escrito en C++. Sobre este código se ha construido un adapta-

¹<https://github.com/dmolina/cec2017real>

dor común en *Python*, de modo que AGE, DE y SHADE evalúen candidatos mediante una misma interfaz.

Las implementaciones de las metaheurísticas no proceden de una única biblioteca. AGE se ha desarrollado directamente en *Python* a partir del pseudocódigo descrito en el Capítulo 5. En cambio, DE y SHADE parten de las implementaciones de PyADE², que se han adaptado para utilizar la misma infraestructura común que AGE: interfaz de evaluación, registro de históricos e integración del mecanismo de reinicio. De esta forma se aprovechan implementaciones existentes de algoritmos, reduciendo el esfuerzo de desarrollo y manteniendo al mismo tiempo un protocolo homogéneo para las tres metaheurísticas consideradas.

En el caso de los modelos subrogados, la mayoría de los regresores se implementan mediante *scikit-learn*, biblioteca de aprendizaje automático que proporciona una interfaz común para entrenamiento, predicción y ajuste de hiperparámetros. Además, el escalado de la variable objetivo descrito en la Sección 6.4.2 se realiza mediante *StandardScaler*, propio de esa misma librería. El modelo RBF se construye con *RBFInterpolator* de *SciPy*, al ajustarse directamente a la formulación empleada en este trabajo. XGBoost utiliza su biblioteca oficial, ya que las consideradas no cuentan con implementación propia. Por último, Kriging y PCE se implementan mediante *Surrogate Modeling Toolbox* (SMT), una biblioteca especializada en modelado subrogado que permite incorporar estas técnicas clásicas de referencia.

Para describir el comportamiento de estos modelos, las métricas de evaluación se calculan combinando funciones de *scikit-learn*, *SciPy* y *NumPy*. En particular, *scikit-learn* se utiliza para las métricas de error, mientras que *SciPy* y *NumPy* se emplean para las operaciones necesarias en las métricas normalizadas y en el cálculo de la correlación de Spearman.

El procesamiento posterior de los resultados obtenidos por los distintos experimentos combina varias herramientas:

- Los ficheros **HDF5** se gestionan mediante *PyTables*.
- *pandas* se utiliza para agrupar ejecuciones, calcular medias y preparar las tablas del Capítulo 7.
- *Matplotlib* se emplea para generar el soporte visual de los resultados como las curvas de convergencia o de diversidad.
- DEAP se usa para el registro de estadísticas generacionales, mediante sus utilidades *Statistics* y *Logbook*.

²<https://github.com/xKuZz/pyade>

- La comparación agregada de medias y rankings medios sobre el benchmark se obtiene mediante **TACOLAB**³, una herramienta web orientada al análisis comparativo de algoritmos de optimización.

Las versiones concretas de las principales librerías empleadas en el entorno experimental para poder reproducir los experimentos son:

- Python 3.14.5
- NumPy 2.4.3
- Matplotlib 3.10.8
- SciPy 1.17.1
- scikit-learn 1.8.0
- SMT 2.14.0
- XGBoost 3.2.0
- pandas 3.0.1
- PyADE 0.0.2
- DEAP 1.4.3
- PyTables 3.11.1

Adicionalmente, la integración del benchmark CEC2017 requiere compilar el código de referencia escrito en C++. Para ello se han utilizado CMake 4.0.1 y Apple Clang 17.0.0.

6.9.2. Registro y almacenamiento de resultados

Los resultados generados por las metaheurísticas se almacenan de forma independiente para cada combinación de función, algoritmo y semilla en formato **HDF5 comprimido**, ya que permite manejar eficientemente grandes volúmenes de datos, con un tamaño reducido por la compresión sin pérdida y manteniendo separados los distintos experimentos. Además, al conservar los datos en formato numérico, se preserva la precisión asociada al tipo de dato utilizado. El contenido de este fichero se corresponde con el especificado en la Tabla 6.7.

³<https://tacolab.org/>

Además del fichero principal, durante la ejecución se generan ficheros **JSON** y **CSV** auxiliares con estadísticas descriptivas de la población, medidas de diversidad y metadatos asociados al mecanismo de reinicio. Estos archivos no se emplean como variables de entrada para los modelos subrogados, pero sí son útiles para analizar la dinámica de búsqueda y construir gráficas informativas.

El resto de experimentos también genera archivos auxiliares que recogen, para cada función, metaheurística, semilla, bloque y modelo, las métricas principales, los hiperparámetros empleados y los tiempos de entrenamiento y predicción. En la integración del modelo subrogado durante la búsqueda, se registra además información específica sobre su uso, como el número de reentrenamientos, el tiempo dedicado al modelo o la proporción de candidatos filtrados.

De este modo, los datos principales de cada ejecución se almacenan en formatos compactos y eficientes y se reservan formatos más ligeros y fáciles de interpretar para el análisis de resultados.

6.9.3. Reproducibilidad de los experimentos

El equipo utilizado para las ejecuciones se corresponde con un MacBook Air, equipado con el procesador Apple M4, que dispone de 10 núcleos distribuidos en 4 de rendimiento y 6 de eficiencia, junto con 16 GB de memoria RAM unificada y el sistema operativo macOS 26.3. Los experimentos se han lanzado de forma **secuencial**, sin paralelización entre ejecuciones, aunque algunos modelos como Random Forest y XGBoost hacen uso de paralelización interna mediante `n_jobs=-1`. Los tiempos de entrenamiento y predicción presentados son directamente comparables dentro del entorno descrito, al haberse obtenido bajo las mismas condiciones de ejecución.

Los experimentos descritos en este capítulo se organizan mediante distintos programas de ejecución. Cada uno de ellos corresponde a una fase concreta del estudio y permite modificar los parámetros necesarios sin alterar la implementación de los algoritmos.

1. `ejecutar_metaheurísticas.py`: implementa el diseño propuesto en las Secciones 6.1 y 6.2. Ejecuta las metaheurísticas sobre CEC2017 y registra los históricos de evaluaciones descritos en la Sección 6.3.
2. `ejecutar_no_acumulativo.py` y `ejecutar_acumulativo.py`: realizan la evaluación temporal de los modelos subrogados. Ambos comparten los mismos parámetros; la diferencia se encuentra en la forma de construir el conjunto de entrenamiento, tal y como se describió anteriormente.

3. `ajustar_hiperparametros.py`: lleva a cabo el ajuste fino de hiperparámetros conservando la estructura temporal e introduciendo una partición interna dentro de cada bloque de entrenamiento.
4. `ejecutar_metaheurísticas_online.py`: ejecuta las metaheurísticas consideradas sobre CEC2017, incorporando el filtro subrogado y registrando tanto el rendimiento final como los contadores asociados a la hibridación.

Los parámetros de cada programa se recogen en el Apéndice [F](#).

Capítulo 7

Experimentación y resultados

En este capítulo se presentan los resultados obtenidos siguiendo el procedimiento experimental descrito en el Capítulo 6. El análisis se plantea de forma progresiva, de manera que las decisiones tomadas en cada fase condicionan la siguiente.

En primer lugar, se estudia el comportamiento de las metaheurísticas consideradas y se evalúa el efecto del reinicio si se detectan fases de estancamiento. Seguidamente, se comparan las estrategias de evaluación *offline* y los modelos subrogados definidos. Tras refinar la técnica más prometedora, se integra durante la ejecución de las metaheurísticas para comprobar si su uso como filtro aporta una mejora real frente a la versión sin subrogado.

7.1. Estancamiento de las metaheurísticas y evaluación del reinicio elitista

Antes de evaluar los modelos subrogados, es necesario analizar el comportamiento de las metaheurísticas durante la búsqueda. Si aparecen fases de estancamiento, las evaluaciones posteriores pueden aportar poca información nueva y reducir la utilidad del histórico generado.

En esta sección se estudia si las metaheurísticas consideradas presentan este comportamiento y, en caso afirmativo, se evalúa el mecanismo de reinicio definido en el Capítulo 5. El objetivo no es optimizar el reinicio en sí, sino determinar si cada metaheurística lo necesita para generar datos más útiles en las fases posteriores sin deteriorar de forma relevante la calidad final.

7.1.1. Motivación del reinicio

Para comprobar si el estancamiento aparece en los históricos de evaluaciones generados por las metaheurísticas consideradas, se analiza la evolución del error respecto al óptimo y la distribución de las evaluaciones a lo largo del presupuesto.

Este comportamiento se aprecia en la Figura 7.1, donde se comparan tres funciones con dinámicas distintas, reflejando tanto el error medio del mejor individuo como el promedio poblacional. En f_1 y f_{29} se observa que la mejora se concentra en las primeras evaluaciones y va seguida de una fase prácticamente plana. En f_{10} , el comportamiento es más gradual para DE y SHADE, mientras que AGE se estabiliza antes. Esto demuestra que la necesidad de reactivación no tiene por qué ser la misma para todas las metaheurísticas.

El efecto de este estancamiento sobre los datos generados se observa con mayor detalle al dividir el presupuesto en bloques temporales como los definidos para la fase *offline* posterior. En la Figura 7.2 se muestra un ejemplo concreto para AGE sobre f_{12} , donde se representa la distribución del error en distintos tramos de la ejecución.

En los primeros bloques, las evaluaciones cruben distintas regiones del espacio de búsqueda. Sin embargo, a medida que aumenta el número de evaluaciones, los valores se concentran en una región mucho más estrecha. Desde el punto de vista del ajuste de subrogados, evaluaciones que aportan menos diversidad al histórico pueden limitar la capacidad del modelo para distinguir entre soluciones de distinta calidad.

Por este motivo, antes de construir los conjuntos utilizados en las fases subrogadas, se analiza si el reinicio definido permite reactivar la búsqueda sin perjudicar el resultado final de cada metaheurística.

7.1.2. Evaluación de las configuraciones de reinicio

A partir de los indicios anteriores, se comparan las configuraciones de reinicio definidas en la Subsección 6.2.1, tomando como referencia la versión sin reinicio. Los resultados anteriores demostraron que, para una misma función, las metaheurísticas consideradas no tienen por qué presentar la misma dinámica de convergencia. Por ello, el análisis se realiza por separado para cada metaheurística.

El objetivo es seleccionar, en cada caso, una ventana de estancamiento ρ que reactive la búsqueda sin deteriorar de forma relevante la calidad final de las soluciones.

Antes de comparar la calidad final, se estudia la frecuencia con la que se

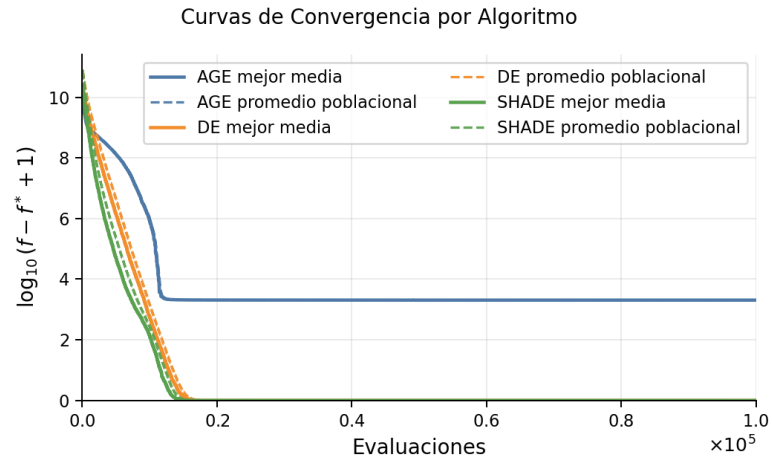
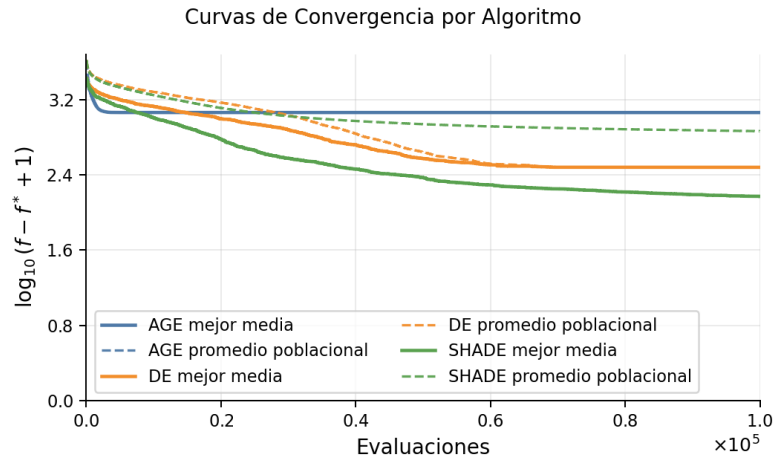
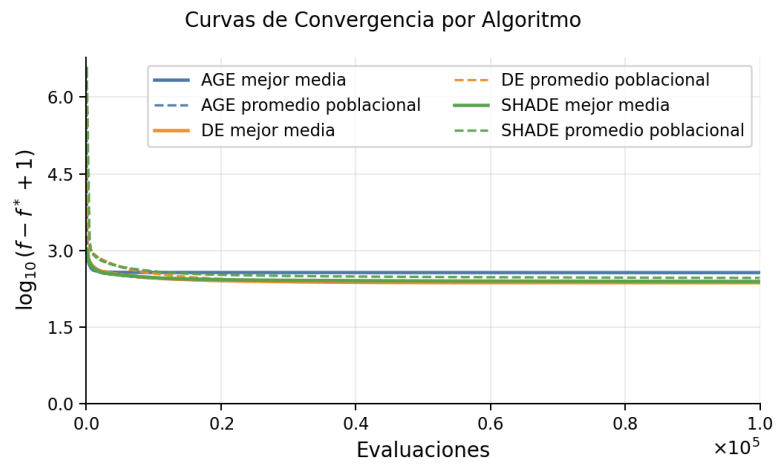
(a) f_1 (b) f_{10} (c) f_{29}

Figura 7.1: Curvas de convergencia promedio sin reinicio en tres funciones representativas de CEC2017.

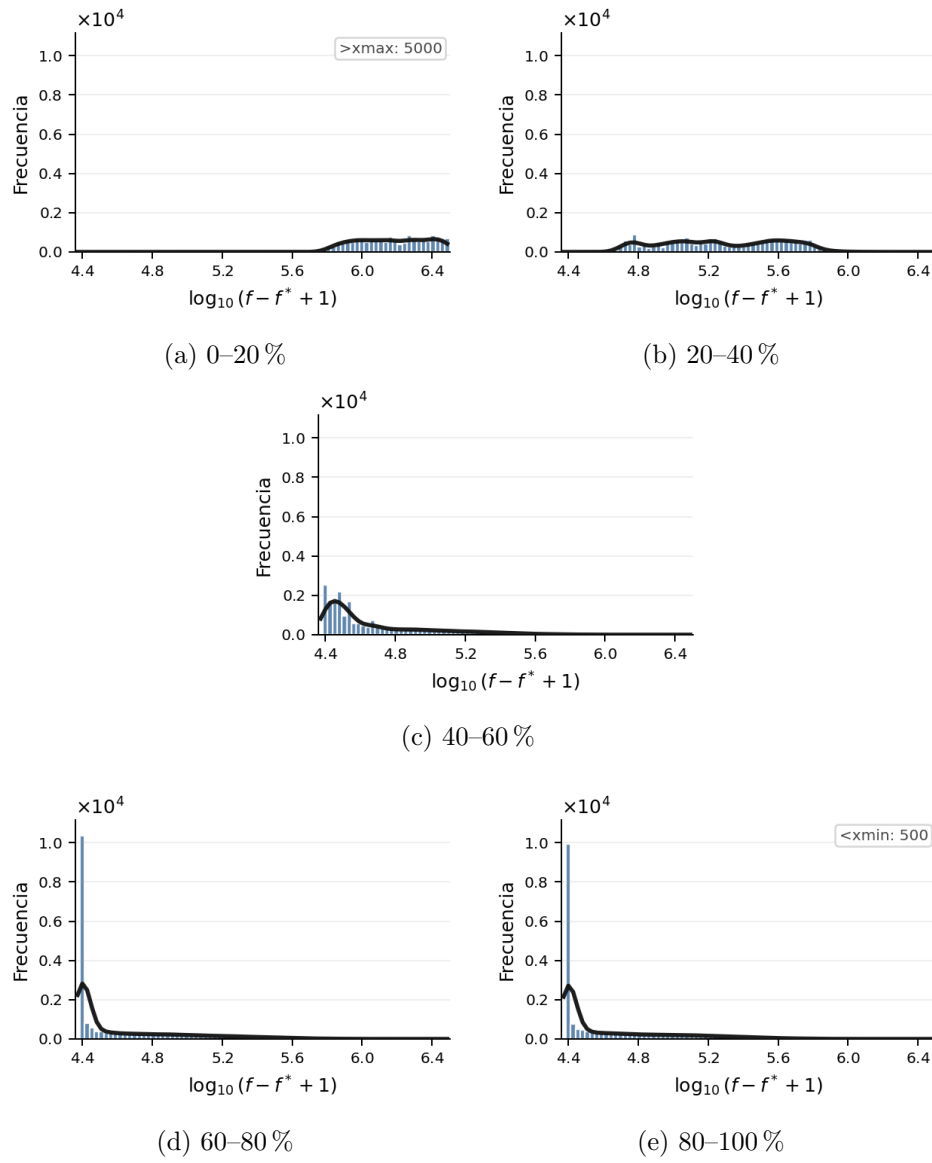


Figura 7.2: Distribución del error por bloques de evaluación: AGE sobre f_{12} sin reinicio (semilla 41).

activa el mecanismo. La Figura 7.3 muestra que las ventanas más pequeñas producen reinicios más frecuentes, como era esperable. Esta información permite interpretar la agresividad de cada configuración, pero no se utiliza por sí sola como criterio de selección.

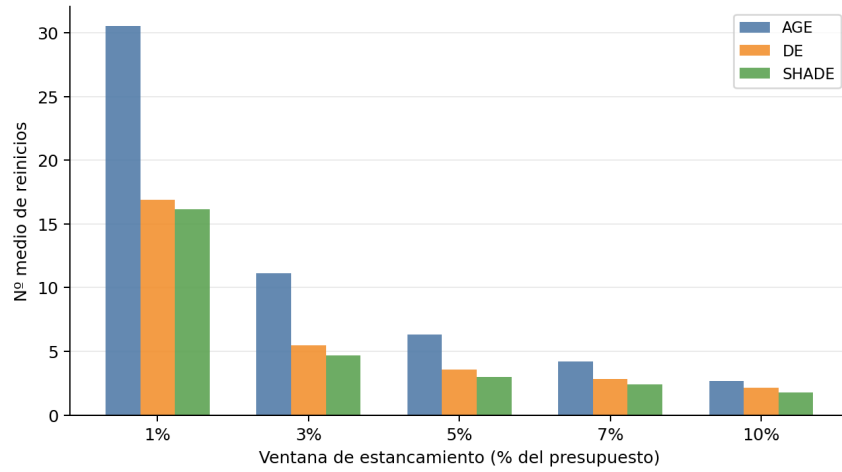


Figura 7.3: Número medio de reinicios elitistas por ejecución según la ventana de estancamiento ρ .

La comparación entre configuraciones se realiza mediante el ranking medio generado con TACOLAB, complementado por el número de funciones en las que cada configuración es mejor o empata con el resto de variantes. Las Tablas 7.1, 7.2 y 7.3 recogen estos resultados para AGE, DE y SHADE, respectivamente. En el cuerpo del capítulo se muestran únicamente estas medidas, mientras que las tablas completas generadas por TACOLAB se recogen en el Apéndice B.

Configuración	Rank medio	Func. mejor
AGE-1	1.6000	23
AGE-3	2.6500	4
AGE-5	3.3667	0
AGE-7	3.8167	0
AGE	4.6667	3
AGE-10	4.9000	1

Tabla 7.1: Ranking medio de TACOLAB y número de funciones en las que cada configuración aparece entre las mejores para AGE.

Configuración	Rank medio	Func. mejor
DE-10	2.5167	8
DE-7	3.0167	3
DE-5	3.2833	2
DE-3	3.5833	2
DE	4.0500	6
DE-1	4.5500	7

Tabla 7.2: Ranking medio de TACOLAB y número de funciones en las que cada configuración aparece entre las mejores para DE.

Configuración	Rank medio	Func. mejor
SHADE-7	2.9333	3
SHADE-10	2.9333	4
SHADE-5	3.2333	4
SHADE	3.6000	6
SHADE-3	3.7167	3
SHADE-1	4.5833	5

Tabla 7.3: Ranking medio de TACOLAB y número de funciones en las que cada configuración aparece entre las mejores para SHADE.

En AGE, el reinicio con ventana corta es el más favorable: obtiene el mejor ranking medio y aparece entre las mejores configuraciones en la mayoría de funciones. A medida que la ventana aumenta, el efecto del reinicio empeora, hasta situar a AGE-10 por detrás de la versión sin reinicio. En este caso, una reactivación frecuente ayuda a evitar que la búsqueda quede estabilizada demasiado pronto.

En cambio, en DE y SHADE el comportamiento es distinto. En DE, las mejores posiciones se obtienen con ventanas más amplias, especialmente con $\rho = 0.10$. Esto indica que el algoritmo se beneficia de conservar durante más tiempo la dinámica de la población antes de aplicar una reactivación, coherentemente con la convergencia observada en la Figura 7.1. En SHADE, las ventanas más grandes vuelven a obtener el mejor ranking medio, pero la versión sin reinicio aparece entre las mejores en más funciones. Este análisis sugiere que el reinicio puede ser útil, pero solo cuando se aplica con una ventana poco agresiva.

La Figura 7.5 muestra este efecto en una ejecución de SHADE con $\rho = 0.01$, donde los reinicios frecuentes elevan la diversidad, pero no se traducen en una mejor convergencia frente a la versión sin reinicio.

Estas diferencias se reflejan también en la Figura 7.4. AGE mejora con ventanas cortas y empeora al aumentar ρ , mientras que DE y SHADE tienden a beneficiarse de ventanas más amplias.

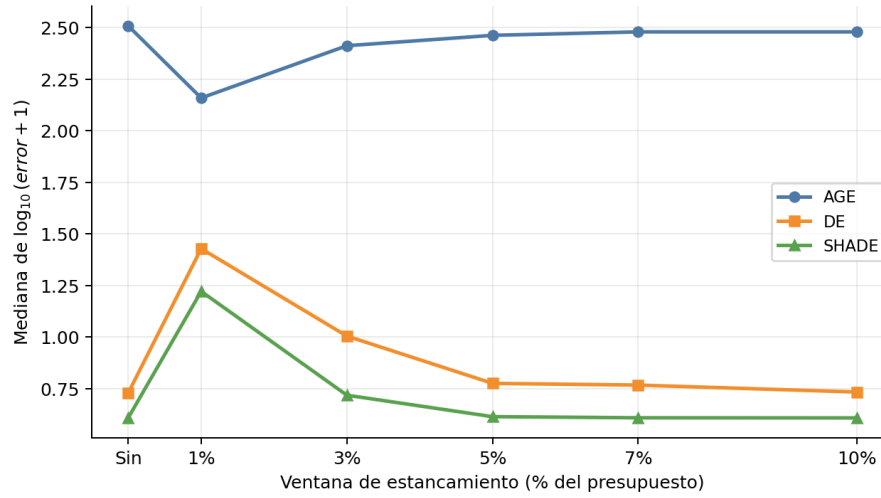


Figura 7.4: Evolución del rendimiento agregado en función del tamaño de la ventana de estancamiento ρ .

7.1.3. Conclusiones sobre la reactivación de la búsqueda

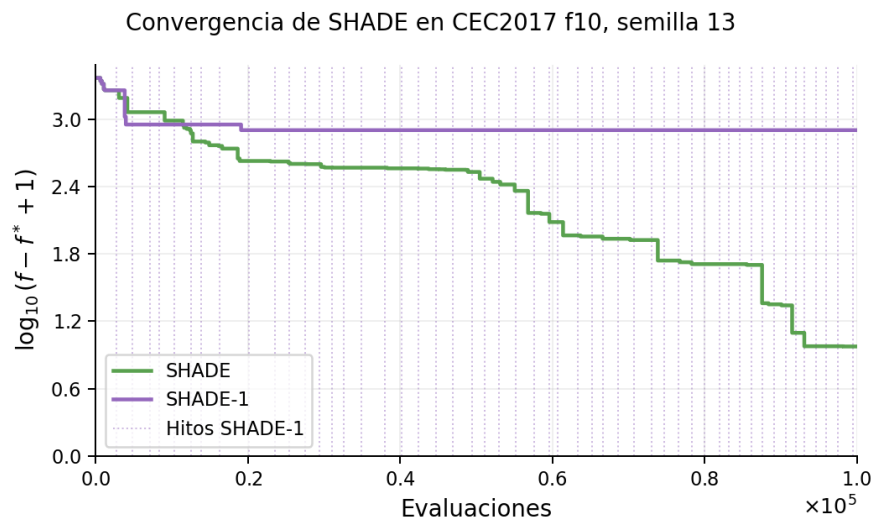
A partir del análisis anterior, no es conveniente utilizar una ventana de estancamiento común para todas las metaheurísticas. AGE se beneficia de una reactivación frecuente, mientras que DE y SHADE presentan mejores resultados con ventanas más amplias. Por ello, la configuración se fija de forma específica para cada algoritmo.

La selección es directa para AGE y DE. En SHADE, las configuraciones SHADE-7 y SHADE-10 obtienen el mismo ranking medio. Ante este empate, se selecciona SHADE-10 por ser la alternativa menos agresiva y por aparecer entre las mejores en un mayor número de funciones.

Algoritmo	Configuración	Ventana ρ
AGE	AGE-1	0.01
DE	DE-10	0.10
SHADE	SHADE-10	0.10

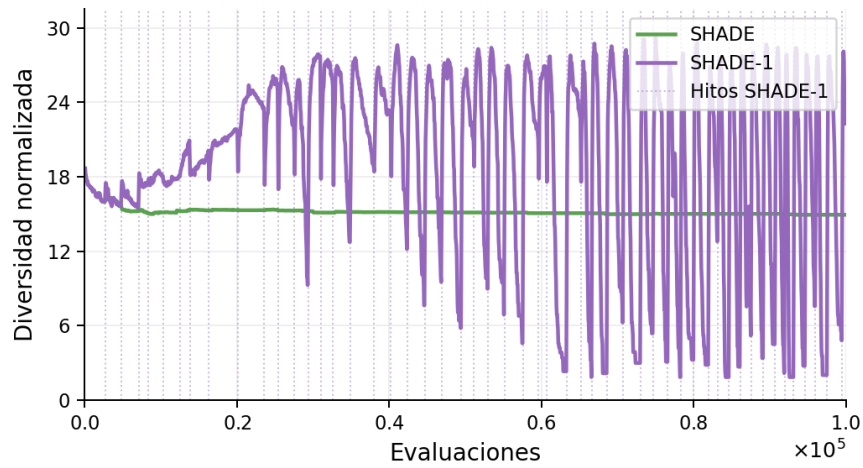
Tabla 7.4: Configuraciones de reinicio seleccionadas para las fases posteriores.

A partir de este punto, los históricos de evaluaciones empleados en las fases subrogadas se generan con estas configuraciones. De este modo, la evaluación *offline* de los modelos se realiza sobre datos menos condicionados por fases prolongadas de estancamiento.



(a) Convergencia

SHADE | CEC2017 f10 — Diversidad por variante de reinicio, semilla 13



(b) Diversidad

Figura 7.5: Ejemplo del efecto de una ventana de reinicio corta en SHADE sobre f_{10} con $\rho = 0.01$.

7.2. Estudio *offline* y ajuste de los modelos subrogados

En esta sección se estudia el comportamiento de los modelos subrogados antes de incorporarlos al proceso de optimización. Como se explicó en el Capítulo 5, esta evaluación se realiza de forma *offline*, por lo que las predicciones del modelo no intervienen todavía en la búsqueda.

Con este análisis se pretende determinar qué estrategia de entrenamiento resulta más adecuada y qué modelo ofrece una buena capacidad de ordenación, considerando también sus costes de ajuste y predicción. La configuración seleccionada será la que se utilice después en la integración *online*.

7.2.1. Selección de la estrategia de entrenamiento *offline*

Antes de comparar los modelos subrogados considerados, es necesario fijar cómo se construyen sus conjuntos de entrenamiento. Como se describió en el Capítulo 5, se consideran dos alternativas: una estrategia no acumulativa, basada únicamente en el último bloque disponible, y una estrategia acumulativa, que incorpora progresivamente todo el histórico anterior.

Esta diferencia afecta especialmente a los modelos sensibles al número de muestras. El caso más claro es SVR, ya que la estrategia acumulativa no llegó a completar todo el subconjunto de funciones considerado durante el cribado. La Figura 7.6 ilustra este comportamiento sobre f_4 , donde el coste de ajuste crece rápidamente.

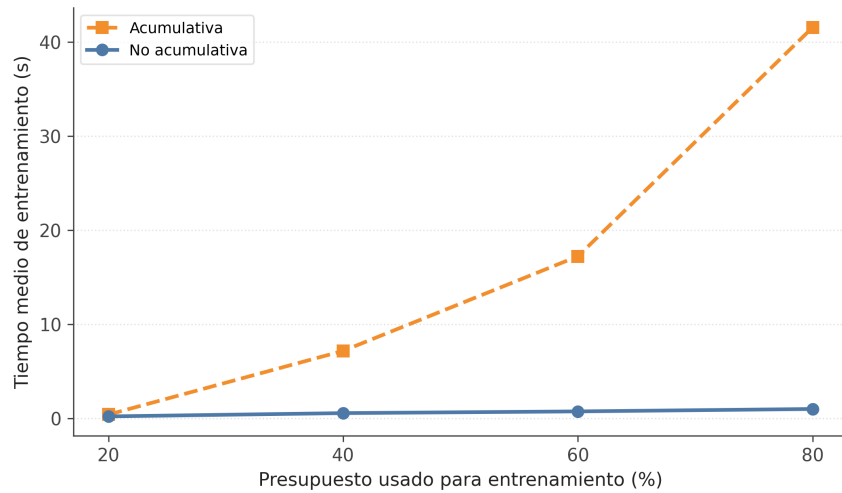


Figura 7.6: Evolución del tiempo medio de entrenamiento de SVR sobre f_4 para las estrategias acumulativa y no acumulativa.

Para garantizar una comparación justa, el análisis entre estrategias se realiza utilizando únicamente los modelos que finalizaron ambas configuraciones.

Algoritmo	Spearman		Entrenamiento (s)	
	No acum.	Acum.	No acum.	Acum.
AGE	0.3097	0.2253	0.5869	1.4696
DE	0.2226	0.1049	0.7464	1.7765
SHADE	0.4100	0.3694	0.6293	1.8712

Tabla 7.5: Comparativa de estrategias *offline* por algoritmo. Los valores se agregan sobre funciones, bloques temporales y modelos comunes a ambas estrategias, excluyendo SVR al no completar la evaluación acumulativa.

En la Tabla 7.5 se observa que la estrategia no acumulativa obtiene un Spearman medio superior en las tres metaheurísticas y requiere un menor tiempo de ajuste.

Para comprobar que esta conclusión no se debe únicamente a la media agregada, la Figura 7.7 muestra la evolución de Spearman a lo largo de los bloques temporales evaluados. En las tres metaheurísticas, la estrategia no acumulativa se mantiene por encima de la acumulativa en todos los bloques considerados.

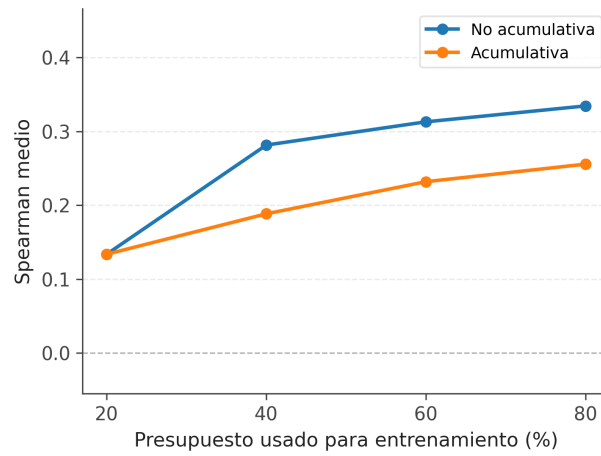
Este comportamiento era esperable, ya que las muestras generadas al inicio de la búsqueda pueden pertenecer a regiones distintas de las exploradas en etapas posteriores. Al mezclar todo el histórico, la estrategia acumulativa puede dificultar la ordenación del bloque siguiente, mientras que la no acumulativa se ajusta con información más reciente.

Por tanto, en análisis posteriores se utiliza el enfoque no acumulativo como estrategia de entrenamiento *offline* común al evaluar los modelos.

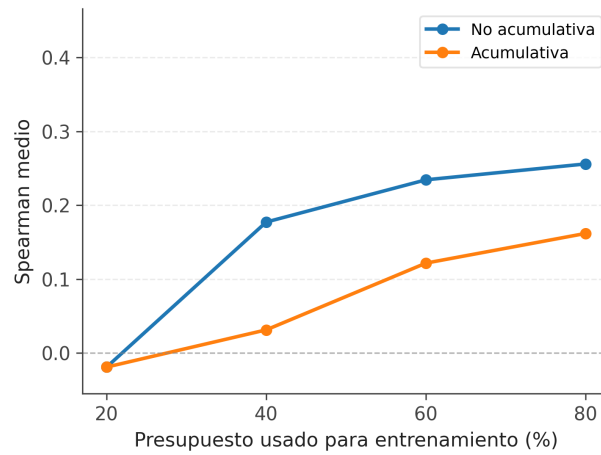
7.2.2. Comparación de modelos por bloques temporales

Fijada la estrategia no acumulativa, se comparan los modelos subrogados bajo un mismo protocolo de entrenamiento. En este análisis se vuelve a incluir SVR, ya que la comparación se realiza únicamente sobre la estrategia que sí completó. En este punto no solo interesa el rendimiento medio de cada modelo, sino también si su comportamiento depende de la metaheurística que genera los datos o del momento de la búsqueda en el que se ajusta.

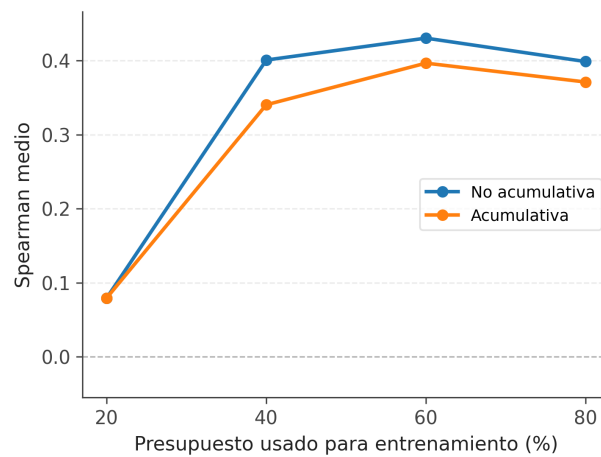
Las Tablas 7.6, 7.7 y 7.8 recogen, para cada metaheurística, el ranking medio de los modelos por bloque temporal, calculado a partir del coeficiente de Spearman. Cada columna indica el bloque utilizado para entrenar el



(a) AGE



(b) DE



(c) SHADE

Figura 7.7: Evolución de la correlación de Spearman media por bloque temporal para las estrategias acumulativa y no acumulativa, desglosada por algoritmo.

modelo. Por ejemplo, la columna 20 % corresponde al entrenamiento con el tramo 1–20 % y a la validación sobre el bloque 21–40 %.

Modelo	Rank medio por bloque				Tiempos medios	
	20 %	40 %	60 %	80 %	Entren. (s)	Pred. (s)
RBF	2.0000	2.8571	2.0000	2.0000	0.0045	0.5220
RSM	2.7143	3.8571	3.5714	4.0000	0.0191	0.0011
RF	4.4286	3.4286	3.2857	3.4286	1.7193	0.0411
SVR	5.8571	3.8571	3.8571	3.5714	0.9550	0.2627
MLP	4.2857	4.2857	4.8571	4.5714	1.3508	0.0017
XGBoost	4.4286	5.2857	5.0000	5.2857	0.5220	0.0032
HGB	6.4286	5.8571	6.7143	6.4286	0.6826	0.0164
LASSO	5.8571	6.5714	6.7143	6.7143	0.0009	0.0001

Tabla 7.6: Comparativa de modelos para AGE bajo la estrategia no acumulativa, desglosada por bloque temporal.

Modelo	Rank medio por bloque				Tiempos medios	
	20 %	40 %	60 %	80 %	Entren. (s)	Pred. (s)
RBF	2.4286	2.1429	2.4286	1.8571	0.0044	0.4253
RSM	2.7143	3.0000	2.4286	2.2857	0.0264	0.0015
SVR	3.7143	3.5714	4.2857	4.0000	1.4310	0.4751
MLP	3.5714	3.7143	3.8571	5.1429	1.7261	0.0021
RF	6.1429	4.7143	4.1429	4.4286	2.3418	0.0428
XGBoost	5.1429	5.4286	5.5714	5.0000	0.5619	0.0034
LASSO	5.5714	6.4286	6.2857	6.8571	0.0010	0.0001
HGB	6.7143	7.0000	6.8571	6.4286	0.7757	0.0184

Tabla 7.7: Comparativa de modelos para DE bajo la estrategia no acumulativa, desglosada por bloque temporal.

Los resultados muestran que RBF es el modelo con un comportamiento más regular en las tres metaheurísticas. En AGE ocupa la primera posición desde el primer bloque de entrenamiento, aunque la diferencia con RSM y Random Forest no siempre es muy marcada. En DE se observa un patrón similar, con un empate frente a RSM cuando se entrena con el bloque 41–60 %. En SHADE, RBF no es el mejor con el primer bloque de entrenamiento, pero pasa a ocupar la primera posición a partir del bloque 21–40 % y mantiene esa ventaja en los bloques posteriores, lo que indica que su rendimiento mejora cuando la búsqueda avanza y el bloque de entrenamiento contiene información más representativa.

Entre el resto de modelos, Random Forest es una alternativa competitiva en SHADE y en algunos bloques de AGE, pero no se mantiene en DE.

Modelo	Rank medio por bloque				Tiempos medios	
	20 %	40 %	60 %	80 %	Entren. (s)	Pred. (s)
RBF	3.2857	1.7143	1.7143	1.5714	0.0047	0.4237
RF	4.2857	2.5714	2.2857	2.7143	2.0612	0.0405
RSM	3.0000	4.0000	4.2857	4.5714	0.0200	0.0012
XGBoost	4.7143	3.8571	3.7143	4.0000	0.4901	0.0030
SVR	4.4286	5.4286	5.2857	5.1429	1.3476	0.4748
MLP	4.5714	5.5714	5.4286	5.0000	1.7603	0.0017
HGB	6.1429	5.5714	5.7143	5.7143	0.3510	0.0102
LASSO	5.5714	7.2857	7.5714	7.2857	0.0009	0.0001

Tabla 7.8: Comparativa de modelos para SHADE bajo la estrategia no acumulativa, desglosada por bloque temporal.

Por otro lado, LASSO y HGB ocupan las últimas posiciones para las tres metaheurísticas. También se observa que RSM aparece de forma recurrente entre las mejores posiciones, especialmente en DE. Por tanto, en este escenario las técnicas subrogadas clásicas muestran un comportamiento más competitivo que los modelos de aprendizaje automático considerados.

A partir de estos resultados, **RBF** se selecciona como el modelo más prometedor para las fases posteriores, al mostrar el comportamiento más estable entre metaheurísticas y etapas de la búsqueda. Aunque el tiempo de predicción de RBF es superior al de modelos como RSM o Random Forest, en CEC2017 cada evaluación es barata, por lo que no se espera que el uso del subrogado reduzca necesariamente el tiempo total de cómputo. Por ello, en esta fase se prioriza la capacidad de ordenación, dejando el efecto sobre las evaluaciones reales para el análisis de la integración *online*.

7.2.3. Ajuste de Hiperparámetros

Antes de incorporar RBF a la fase *online*, se refinan sus hiperparámetros. El objetivo es decidir si conviene usar una configuración común para todas las metaheurísticas o una configuración específica para cada una.

El ajuste se realiza mediante la validación interna descrita en el Capítulo 6. Dado el mayor tiempo de predicción de RBF, se mantienen las siete funciones de cribado utilizadas en los análisis anteriores, pero se emplea un subconjunto fijo de 10 semillas de las 51 disponibles para evitar que este coste domine la exploración de hiperparámetros.

Los valores concretos del espacio de búsqueda se recogen en la Tabla 7.9. A partir de estos, se evalúan todas las configuraciones posibles, cuyos rankings completos por metaheurística se incluyen en el Apéndice C.

Modelo	Hiperparámetro	Valores
RBF	kernel	multiquadric, gaussian
	epsilon	0.1, 1.0, 10.0
	smoothing	10^{-3} , 10^{-2}
	vecinos	25, 50, 100
	degree	-1

Tabla 7.9: Espacio de búsqueda utilizado en el ajuste fino de hiperparámetros.

Antes de fijar la configuración final, se analiza primero el efecto de los hiperparámetros principales para evitar que la decisión dependa de una combinación concreta.

En primer lugar, se comparan los dos kernels considerados.

Kernel	Rank AGE	Rank DE	Rank SHADE
<i>multiquadric</i>	15.675	18.359	18.491
<i>gauss</i>	21.325	18.641	18.509

Tabla 7.10: Rank medio por kernel agregado sobre todas las configuraciones y algoritmos.

En la Tabla 7.10 se observa que el kernel *multiquadric* obtiene el mejor ranking medio. La diferencia es clara en AGE, mientras que en DE y SHADE ambos kernels quedan muy próximos. Aun así, *multiquadric* mantiene la mejor posición en las tres metaheurísticas, por lo que se fija este kernel para el resto del ajuste.

Con esta decisión se reduce el conjunto de configuraciones candidatas y el análisis se centra en los parámetros ε , *smoothing* y número de vecinos.

ε	Smoothing	Vecinos	Rank medio	Pred. (s)
0.1	10^{-2}	25	11.964	0.1879
0.1	10^{-3}	25	12.954	0.1876
1.0	10^{-2}	25	13.339	0.1877
0.1	10^{-2}	50	14.089	0.2593
1.0	10^{-3}	25	14.111	0.1879

Tabla 7.11: Cinco mejores configuraciones de RBF para AGE (kernel *multiquadric*, degree = -1).

El ajuste dentro de este espacio reducido muestra que el valor de ε influye de forma distinta según la metaheurística. AGE y SHADE sitúan entre sus mejores configuraciones valores pequeños de ε , mientras que DE obtiene

ε	Smoothing	Vecinos	Rank medio	Pred. (s)
10.0	10^{-3}	100	14.807	0.4159
1.0	10^{-3}	100	15.420	0.4168
10.0	10^{-2}	100	15.739	0.4156
1.0	10^{-3}	50	16.988	0.2393
10.0	10^{-3}	50	17.141	0.2385

Tabla 7.12: Cinco mejores configuraciones de RBF para DE (kernel *multiquadric*, degree = -1).

ε	Smoothing	Vecinos	Rank medio	Pred. (s)
0.1	10^{-3}	25	14.355	0.1482
1.0	10^{-3}	25	14.984	0.1478
1.0	10^{-2}	25	15.520	0.1477
1.0	10^{-3}	50	16.054	0.2195
0.1	10^{-2}	25	16.962	0.1480

Tabla 7.13: Cinco mejores configuraciones de RBF para SHADE (kernel *multiquadric*, degree = -1).

mejores posiciones con valores mayores.

También se aprecian diferencias en el número de vecinos. AGE y SHADE favorecen configuraciones con 25 vecinos, mientras que en DE las mejores posiciones se obtienen con 100 vecinos. Este comportamiento es coherente con la dinámica más rígida de DE, ya que sus parámetros permanecen fijos durante la búsqueda y el modelo parece beneficiarse de una vecindad más amplia.

No obstante, este aumento también incrementa el tiempo de predicción, por lo que no se adopta directamente la configuración más amplia de DE sin compararla antes con una alternativa menos costosa.

En cambio, el parámetro *smoothing* no muestra una diferencia tan clara entre metaheurísticas, por lo que se fija en 10^{-3} . Con el kernel y *smoothing* fijados, se selecciona para AGE y SHADE la mejor configuración del espacio reducido. En DE, dado que las configuraciones mejor posicionadas utilizan 100 vecinos, se toma la mejor alternativa con 50 vecinos para evitar tomar la opción de mayor coste de predicción.

A partir de estos resultados, se comprueba si conviene utilizar una configuración homogénea o adaptar ε y el número de vecinos a cada algoritmo.

La Tabla 7.14 recoge la configuración seleccionada para cada metaheurística. A continuación, se compara esta selección con la mejor configuración homogénea del ranking global del Apéndice C (Tabla C.1): $\varepsilon = 1.0$,

Algoritmo	Parámetros RBF		Rank medio
	ε	Vecinos	
AGE	0.1	25	12.954
DE	1.0	50	16.988
SHADE	0.1	25	14.355

Tabla 7.14: Configuraciones seleccionadas de RBF por metaheurística. Kernel *multiquadric* y smoothing 10^{-3} fijos en todos los casos.

$smoothing = 10^{-3}$, 50 vecinos, kernel *multiquadric* y degree = -1 .

Configuración	AGE		SHADE	
	Spearman	Func. mejor	Spearman	Func. mejor
Homogénea	0.5962	5/7	0.6316	4/7
Por algoritmo	0.4725	2/7	0.6332	3/7

Tabla 7.15: Comparativa entre la configuración homogénea y la configuración por algoritmo para AGE y SHADE.

En la Tabla 7.15 se observa que la configuración homogénea obtiene mejores resultados en AGE, tanto en Spearman medio como en número de funciones ganadas. En SHADE, la configuración por algoritmo alcanza un Spearman medio ligeramente superior, aunque la homogénea aparece como mejor en más funciones. DE no se incluye, ya que la configuración seleccionada por algoritmo coincide con la homogénea.

Por tanto, aunque el ajuste interno muestra diferencias entre metaheurísticas, no se aprecia una ventaja clara al utilizar configuraciones distintas en la fase *online*. Se adopta la configuración homogénea $\varepsilon = 1.0$, $smoothing = 10^{-3}$, 50 vecinos, kernel *multiquadric* y degree = -1 , al ofrecer un comportamiento más estable y simplificar la integración posterior.

7.2.4. Resultados del modelo ajustado

Con la configuración fijada, se evalúa su comportamiento sobre la suite completa CEC2017, utilizando las 30 funciones y las 51 semillas consideradas en los experimentos finales.

Algoritmo	Spearman medio
AGE	0.5074
DE	0.3841
SHADE	0.6147

Tabla 7.16: Rendimiento *offline* de RBF con la configuración final.

En la Tabla 7.16 se observa que el comportamiento de RBF depende de la metaheurística que genera los datos. Sobre SHADE obtiene el Spearman medio más alto, seguido de AGE, mientras que en DE presenta una correlación más baja. Este resultado es coherente con los análisis anteriores, donde DE mostraba una dinámica distinta y menos favorable para el ajuste del modelo.

Para comprobar si el menor Spearman observado en DE se debe a funciones concretas o a un comportamiento más general, la Figura 7.8 desglosa el resultado por función y metaheurística. Se observa que DE presenta valores más bajos en buena parte del benchmark, especialmente frente a SHADE. Por tanto, la diferencia no parece estar provocada por una función aislada, sino por la dinámica de búsqueda de la metaheurística.

Antes de cerrar el estudio *offline*, se compara el comportamiento de RBF sobre ejecuciones con y sin reinicio para comprobar si ambos escenarios conducen a la misma interpretación de los resultados. Esta comparación permite analizar cómo se refleja el estancamiento de la búsqueda en las métricas del modelo.

Algoritmo	Escenario	Spearman	nMAE	nRMSE
AGE	Con reinicio	0.5074	3.31e+07	4.18e+07
AGE	Sin reinicio	0.6312	1.50e+11	2.03e+11
DE	Con reinicio	0.3841	4.7963	26.6350
DE	Sin reinicio	0.3284	5.38e+09	6.66e+09
SHADE	Con reinicio	0.6147	7.5981	206.5265
SHADE	Sin reinicio	0.5472	8.71e+07	5.76e+08

Tabla 7.17: Comparación del comportamiento *offline* de RBF ajustado sobre ejecuciones con y sin reinicio.

La comparación recogida en la Tabla 7.17 indica que el escenario con reinicio obtiene, en general, mejores valores de Spearman y menores errores normalizados. En este caso, las métricas de error si se analizan porque permiten observar cómo el estancamiento puede afectar a la interpretación del rendimiento del modelo.

Como se indicó en la Sección 7.1, sin mecanismos de reactivación la

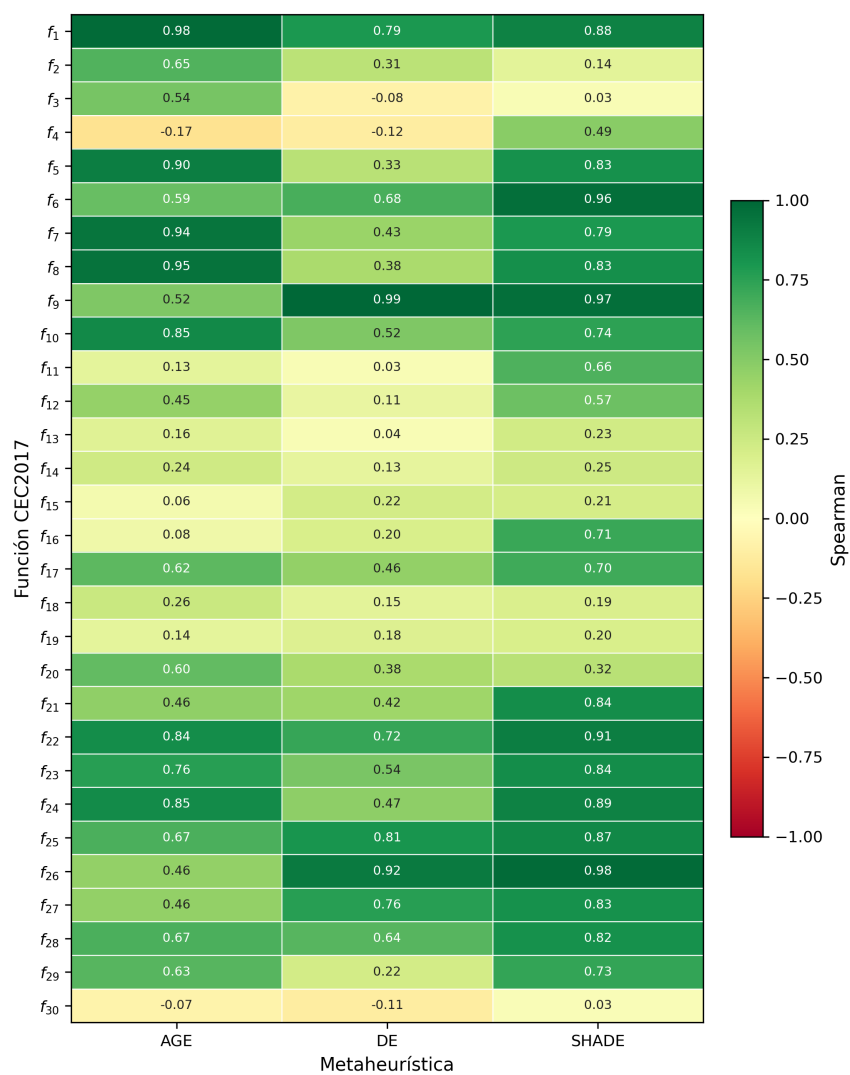


Figura 7.8: Spearman medio de RBF por función y metaheurística sobre CEC2017 completo.

búsqueda puede quedar dominada por fases de estancamiento, en las que las muestras presentan valores de *fitness* muy similares. Este efecto se aprecia con más detalle en la Tabla 7.18, donde se desglosa DE con y sin reinicio sobre f_{10} por bloques temporales.

Bloque de entrenamiento	Con reinicio	Sin reinicio
1-20 %	0.2896	0.2439
21-40 %	0.3065	0.1906
41-60 %	0.1987	0.0196
61-80 %	0.2168	2.0e-05

Tabla 7.18: nMAE medio de RBF sobre f_{10} con y sin reinicio por bloque temporal para DE.

Sin reinicio, el nMAE disminuye conforme avanza la búsqueda y alcanza valores del orden de 10^{-5} . En cambio, con reinicio el error se mantiene en valores más altos, coherente con una búsqueda que sigue generando mayor variación entre las soluciones evaluadas.

Este comportamiento no implica que el modelo haya aprendido una aproximación más útil de la función objetivo. Más bien indica que, en el escenario sin reinicio, el bloque de validación contiene valores de *fitness* muy similares. En esas condiciones, el modelo puede obtener un error muy bajo aunque su predicción aporte poca información para decidir qué candidatos deben evaluarse con la función objetivo real. Por tanto, si la búsqueda queda dominada por este tipo de estancamientos, el papel del subrogado como filtro pierde su interés para el objetivo de este trabajo.

Por este motivo, el estudio *offline* se cierra tomando como referencia las ejecuciones con reinicio, al ser un escenario más adecuado para valorar si el subrogado puede ordenar soluciones en una búsqueda que sigue generando variación.

En conjunto, los resultados muestran que, en el escenario considerado, la estrategia no acumulativa y una técnica subrogada clásica como **RBF** ofrecen el comportamiento más adecuado entre las alternativas evaluadas. Esta configuración se toma como punto de partida para la integración *online*.

7.3. Evaluación de la estrategia *online*

En esta última fase experimental, RBF se incorpora directamente al proceso de optimización. Como se definió en el Capítulo 5, el modelo actúa como un filtro que decide qué candidatos generados por la metaheurística se evalúan realmente.

El objetivo de este análisis no es comparar las metaheurísticas entre sí, sino comprobar si la hibridación mejora el comportamiento de cada una.

Para ello se consideran dos aspectos: la reducción del número de evaluaciones reales que consigue el filtro y si la calidad final de las soluciones se mantiene o mejora. Como ya se comentó en secciones anteriores, el tiempo de ejecución no se utiliza como criterio principal, ya que las funciones del benchmark utilizado tienen un coste reducido. Por este motivo, el análisis se centra en las evaluaciones reales evitadas, que es el aspecto relevante cuando la función objetivo tiene un coste elevado.

7.3.1. Calibración de los parámetros de integración

Antes de evaluar la integración final, se ajustan los parámetros que controlan la intervención del subrogado siguiendo el protocolo descrito en la Sección 6.8. Esta calibración se realiza sobre el subconjunto de funciones y semillas definido para esta fase.

Al trabajar con metaheurísticas con reinicio, también se calibra el periodo de espera tras cada reactivación.

Las combinaciones evaluadas se recogen en el Apéndice D. En este caso no se adopta una configuración homogénea, ya que los parámetros de integración determinan cómo interviene el subrogado dentro de la dinámica de cada metaheurística. AGE, DE y SHADE generan candidatos y responden al reinicio de forma distinta, por lo que una misma configuración puede tener efectos diferentes en cada algoritmo.

Esta diferencia se refleja en los rankings. AGE y DE seleccionan una probabilidad alta de consulta, aunque con distintos periodos de espera y frecuencias de reentrenamiento, mientras que SHADE prefiere una intervención más conservadora del subrogado. Por este motivo, se selecciona una configuración específica para cada metaheurística a partir del rank medio calculado sobre el error final.

Algoritmo	p	cd	rt	Rank medio
AGE-1-RBF	0.75	500	0.25	6.143
DE-10-RBF	0.75	50	0.5	7.857
SHADE-10-RBF	0.25	100	0.25	7.857

Tabla 7.19: Configuración del subrogado *online* seleccionada por algoritmo.

7.3.2. Comparación con la versión sin subrogado

En la Tabla 7.20 se comparan las versiones originales de cada metaheurística con sus variantes híbridas con RBF, evaluadas sobre las 30 funciones de CEC2017 y 51 semillas distintas a las consideradas en fases previas. Para cada caso se indica el número de funciones en las que se obtiene el menor error medio final, así como las métricas internas del filtro.

Algoritmo	Func. mejor	Consulta (%)	Rechazo (%)	Eval. evitadas (%)
AGE-1	17	–	–	–
AGE-1-RBF	13	61.22	75.32	46.11
DE-10	17	–	–	–
DE-10-RBF	17	68.42	82.30	56.31
SHADE-10	18	–	–	–
SHADE-10-RBF	18	20.73	73.55	15.25

Tabla 7.20: Comparación entre cada metaheurística y su versión con filtro RBF.

Como se puede observar, el efecto del subrogado depende de la metaheurística. En AGE, la versión con RBF evita casi la mitad de las evaluaciones reales, pero obtiene mejores resultados en menos funciones que la versión sin subrogado. Por tanto, en este caso la reducción de evaluaciones no compensa la pérdida de calidad final.

En DE ocurre lo contrario. La versión con RBF mantiene el mismo número de funciones mejores que la versión original y evita la evaluación real de más de la mitad de los candidatos generados. Este es el caso más favorable, ya que el filtro reduce de forma clara el número de evaluaciones sin empeorar el resultado final.

Por último, en SHADE también se mantiene un comportamiento equivalente a la versión sin subrogado, aunque el impacto del filtro es menor. Esto es coherente con la configuración seleccionada en la calibración, donde se escogió una intervención más conservadora del subrogado. Como consecuencia, su capacidad para reducir evaluaciones es más limitada.

La comparación anterior resume el resultado final, pero no muestra cómo evoluciona la búsqueda durante la ejecución. Como el filtro descarta candidatos antes de evaluarlos con la función objetivo real, las evaluaciones reales pueden concentrarse en soluciones que el subrogado considera más prometedoras. Por ello, se analizan también curvas de convergencia para comprobar si la versión con RBF alcanza antes valores competitivos.

En las Figuras 7.9, 7.10 y 7.11 se muestra la evolución del error medio en escala logarítmica para funciones con comportamientos distintos.

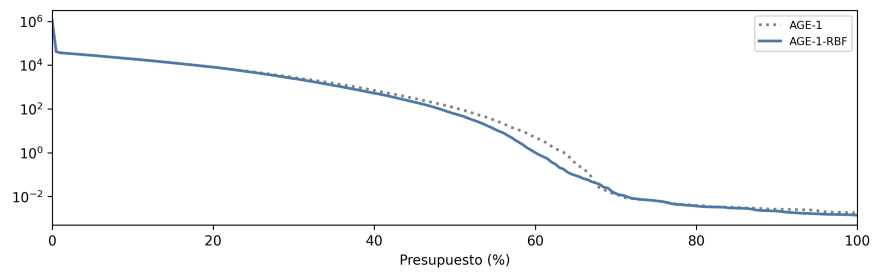
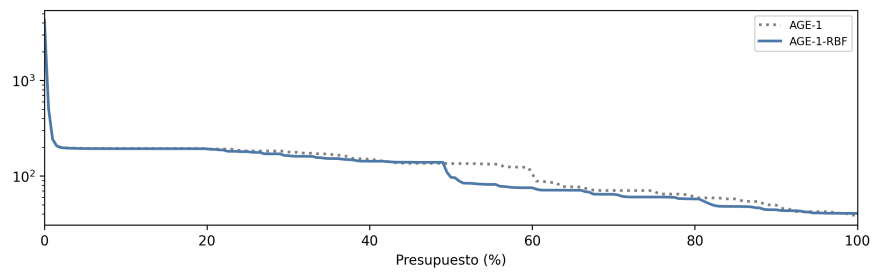
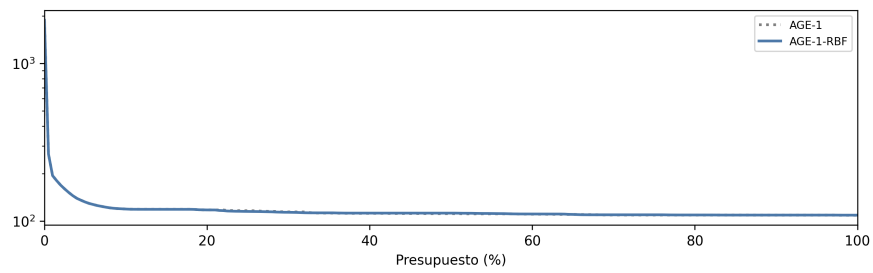
(a) f_3 (b) f_9 (c) f_{22}

Figura 7.9: Convergencia de AGE-1 frente a AGE-1-RBF.

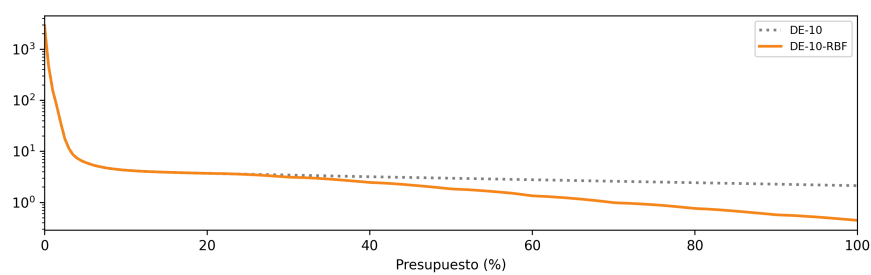
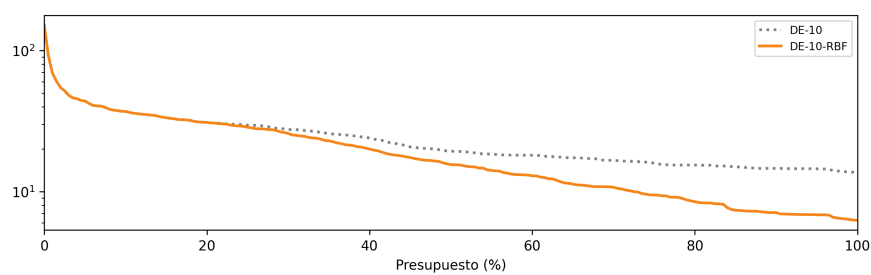
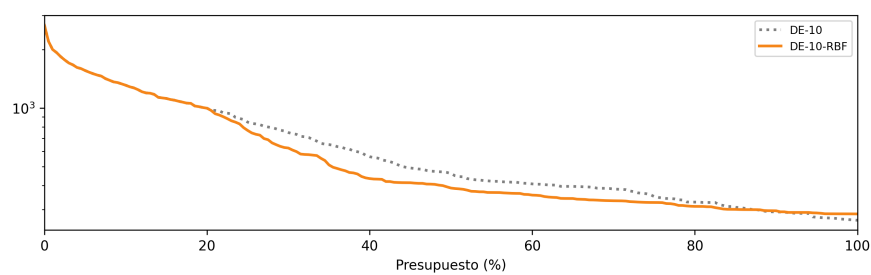
(a) f_4 (b) f_5 (c) f_{10}

Figura 7.10: Convergencia de DE-10 frente a DE-10-RBF.

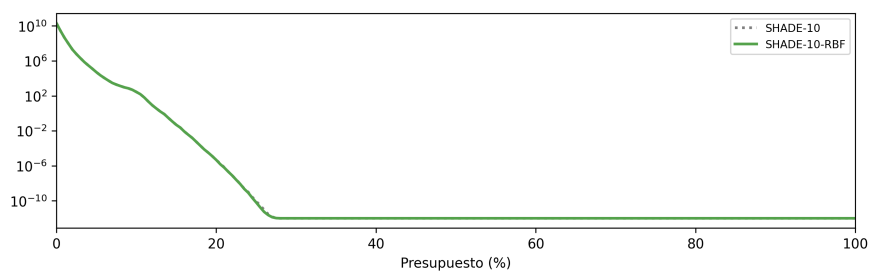
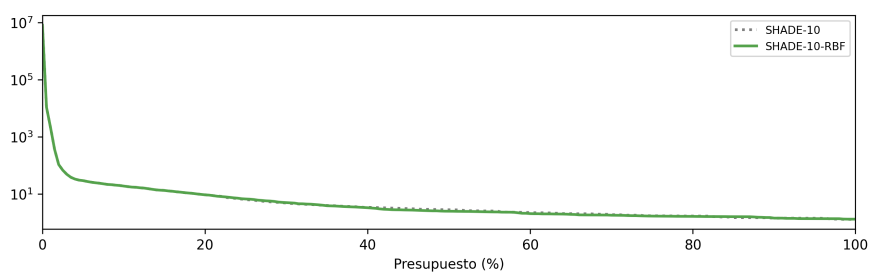
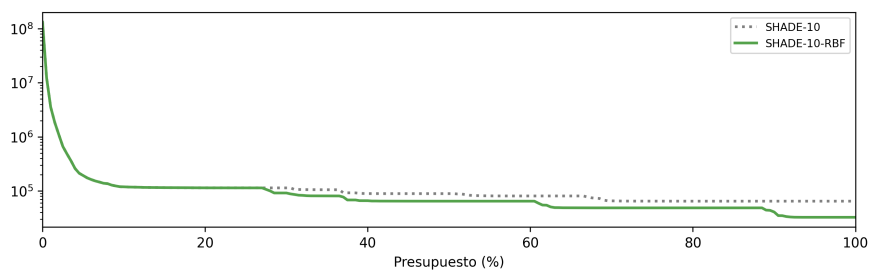
(a) f_1 (b) f_{14} (c) f_{30}

Figura 7.11: Convergencia de SHADE-10 frente a SHADE-10-RBF.

Las curvas muestran que la versión con RBF alcanza antes valores competitivos en varias de las funciones representadas, aunque esta ventaja no siempre se mantiene hasta el final del presupuesto. Por tanto, la integración del subrogado no debe valorarse solo por la calidad final, ya que también puede acelerar algunas fases de la búsqueda.

Este comportamiento es relevante si se considera un criterio de parada por estancamiento. En ese caso, alcanzar antes una solución de calidad similar podría reducir el presupuesto necesario, especialmente cuando cada evaluación de la función objetivo tiene un coste elevado. En este trabajo, sin embargo, se mantiene el presupuesto completo para comparar todas las variantes bajo las mismas condiciones.

7.3.3. Resumen de los resultados experimentales obtenidos

Los resultados anteriores muestran que la integración del subrogado no debe evaluarse únicamente por el número de evaluaciones reales que evita. El filtro debe reducir evaluaciones sin descartar candidatos que puedan contribuir a la mejora de la población. Cuando este equilibrio se mantiene, la hibridación puede ser útil. Cuando el filtrado interfiere demasiado con la dinámica de la metaheurística, la reducción de evaluaciones deja de compensar.

Por tanto, la utilidad de RBF como filtro *online* depende de la metaheurística sobre la que se integra. La fase *offline* permite seleccionar un modelo prometedor, pero no garantiza por sí sola una mejora uniforme al incorporarlo al proceso de optimización, ya que el resultado final también depende de cómo y cuándo interviene el subrogado durante la búsqueda.

Capítulo 8

Conclusiones y trabajo futuro

En este trabajo, nos hemos centrado en el estudio de técnicas de subrogado dentro de metaheurísticas evolutivas para problemas de optimización continua. Estas técnicas permiten aproximar la función objetivo a partir de soluciones ya evaluadas y son de interés cuando cada evaluación real tiene un coste elevado.

El objetivo principal ha sido analizar si estas técnicas pueden utilizarse como filtro dentro del proceso de optimización y estudiar hasta qué punto los modelos de aprendizaje automático resultan competitivos frente a técnicas subrogadas clásicas. Para ello, se ha estudiado primero el comportamiento de distintos modelos en una fase *offline* y, posteriormente, se ha evaluado la integración *online* del modelo seleccionado sobre las metaheurísticas consideradas.

El desarrollo de este trabajo ha abarcado las distintas etapas de una experimentación completa. Comenzamos introduciendo los fundamentos teóricos necesarios para comprender las metaheurísticas y los modelos subrogados. Después, se realizó una revisión de la literatura sobre algoritmos evolutivos asistidos por subrogados, que permitió situar el problema y conocer cómo se ha abordado la integración de estos modelos en trabajos previos para justificar las decisiones experimentales tomadas. A continuación, se definieron las técnicas de referencia y el protocolo experimental. A ello le siguió la generación de datos, el análisis *offline* de los modelos, el ajuste de la configuración seleccionada y su integración final en el proceso de búsqueda.

A nivel experimental, se ha observado primero que la dinámica de las metaheurísticas condiciona directamente los datos con los que se entrena el subrogado. Cuando la búsqueda se estanca, las muestras generadas presentan valores de *fitness* muy similares y el modelo puede obtener errores

numéricos muy bajos sin aportar información útil para filtrar candidatos. Por este motivo, en el estudio *offline* se han tomado como referencia las ejecuciones con reinicio, donde la búsqueda mantiene mayor variabilidad.

Sobre este escenario, se ha comprobado que la estrategia no acumulativa resulta más adecuada para entrenar los modelos subrogados, ya que evita el crecimiento del coste de ajuste y mantiene el entrenamiento centrado en muestras recientes. Entre los modelos evaluados, RBF ha sido el que ha mostrado un comportamiento más estable, especialmente en términos de capacidad de ordenación. Este resultado indica que, en el escenario considerado, una técnica subrogada clásica ha sido más competitiva que los modelos de aprendizaje automático más complejos evaluados. Por ello, RBF se ha seleccionado como modelo principal para la fase *online*.

Por último, la integración *online* ha mostrado que el uso del subrogado depende de la metaheurística sobre la que se aplica. En DE, RBF permite reducir un número importante de evaluaciones reales sin empeorar la calidad final. En SHADE, el comportamiento es más conservador, manteniendo la calidad con una reducción más moderada de evaluaciones. En AGE, sin embargo, la reducción de evaluaciones no compensa la pérdida de calidad observada.

En conjunto, podemos decir que el objetivo del trabajo se ha cumplido. Se ha caracterizado el comportamiento de distintas familias de modelos subrogados, se ha observado que las técnicas clásicas pueden ser más adecuadas en el escenario evaluado y se ha estudiado la integración *online* de RBF como filtro dentro de varias metaheurísticas. Los resultados muestran que la selección de un buen modelo en la fase *offline* no garantiza por sí sola una mejora uniforme durante la búsqueda, ya que la utilidad del subrogado depende también de cómo interviene dentro de la dinámica de cada algoritmo.

Esta interpretación debe tener en cuenta el tipo de problema evaluado. En CEC2017, las evaluaciones de la función objetivo tienen un coste reducido, por lo que la reducción de evaluaciones reales no se traduce necesariamente en una reducción del tiempo de ejecución. Sin embargo, este es el aspecto más relevante en problemas donde cada evaluación tiene un coste elevado, ya que en ese contexto reducir el número de evaluaciones reales sí puede disminuir directamente el coste total del proceso de optimización.

Por último, veremos varias líneas de extensión que podrían abordarse en investigaciones posteriores.

- **Extensión a problemas multiobjetivo.** En los últimos años ha crecido el interés por la integración de técnicas subrogadas en problemas con múltiples objetivos. Este trabajo se ha centrado en problemas de optimización continua con un único objetivo, por lo que una posible

línea futura sería adaptar el enfoque propuesto a escenarios multiobjetivo.

- **Criterios de activación adaptativos para la hibridación *online*.** En este trabajo, la intervención del subrogado durante la búsqueda se controla mediante una probabilidad fija. Como alternativa, podría estudiarse un criterio adaptativo que modifique dicha probabilidad en función del estado de la búsqueda o del comportamiento reciente del modelo. De este modo, el subrogado intervendría con mayor o menor intensidad según resulte conveniente en cada fase de la optimización.
- **Enfoques para la inicialización del modelo subrogado.** En lugar de construir el conjunto de entrenamiento del subrogado a partir de las soluciones generadas por la propia metaheurística y retrasar su disponibilidad al inicio de la búsqueda, se podría investigar una estrategia que permita integrarlo desde la primera evaluación. Por ejemplo, se pueden utilizar técnicas de muestreo aleatorio o técnicas más robustas como *Latin Hypercube Sampling*, aunque con un coste adicional en evaluaciones reales.

Bibliografía

- [1] Hao Tong, Changwu Huang, Leandro L. Minku, and Xin Yao. Surrogate models in evolutionary single-objective optimization: A new taxonomy and experimental study. *Information Sciences*, 562:414–437, 2021. doi:[10.1016/j.ins.2021.03.002](https://doi.org/10.1016/j.ins.2021.03.002).
- [2] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998. doi:[10.1023/A:1008306431147](https://doi.org/10.1023/A:1008306431147).
- [3] Yaochu Jin. A comprehensive survey of fitness approximation in evolutionary computation. *Soft Computing*, 9(1):3–12, 2005. doi:[10.1007/s00500-003-0328-5](https://doi.org/10.1007/s00500-003-0328-5).
- [4] Yaochu Jin. Surrogate-assisted evolutionary computation: Recent advances and future challenges. *Swarm and Evolutionary Computation*, 1(2):61–70, 2011. doi:[10.1016/j.swevo.2011.05.001](https://doi.org/10.1016/j.swevo.2011.05.001).
- [5] Chunlin He, Yong Zhang, Dunwei Gong, and Xinfang Ji. A review of surrogate-assisted evolutionary algorithms for expensive optimization problems. *Expert Systems with Applications*, 217:119495, 2023. doi:[10.1016/j.eswa.2022.119495](https://doi.org/10.1016/j.eswa.2022.119495).
- [6] Jing Liang, Yahang Lou, Mingyuan Yu, Ying Bi, and Kunjie Yu. A survey of surrogate-assisted evolutionary algorithms for expensive optimization. *Journal of Membrane Computing*, 7(2):108–127, 2025. doi:[10.1007/s41965-024-00165-w](https://doi.org/10.1007/s41965-024-00165-w).
- [7] Yaochu Jin, Markus Olhofer, and Bernhard Sendhoff. A framework for evolutionary optimization with approximate fitness functions. *IEEE Transactions on Evolutionary Computation*, 6(5):481–494, 2002. doi:[10.1109/TEVC.2002.800884](https://doi.org/10.1109/TEVC.2002.800884).
- [8] I. Loshchilov, M. Schoenauer, and M. Sebag. Self-adaptive surrogate-assisted covariance matrix adaptation evolution strategy. In *Proceedings of the 14th Annual Conference on Genetic and Evolutionary Computation*, 2012. doi:[10.1145/2330163.2330210](https://doi.org/10.1145/2330163.2330210).

- [9] Huapeng Yu, Ying Tan, Changhe Sun, and Jianchao Zeng. A generation-based optimal restart strategy for surrogate-assisted social learning particle swarm optimization. *Knowledge-Based Systems*, 163:14–25, 2019. doi:[10.1016/j.knosys.2018.08.010](https://doi.org/10.1016/j.knosys.2018.08.010).
- [10] Laurens Bliet. A survey on sustainable surrogate-based optimisation. *Sustainability*, 14(7):3867, 2022. doi:[10.3390/su14073867](https://doi.org/10.3390/su14073867).
- [11] A. Díaz-Manríquez, G. Toscano, and C. A. Coello Coello. Comparison of metamodeling techniques in evolutionary algorithms. *Soft Computing*, 2017. doi:[10.1007/s00500-016-2140-z](https://doi.org/10.1007/s00500-016-2140-z).
- [12] P. S. Naharro, P. Toharia, A. LaTorre, and J.-M. Peña. Comparative study of regression vs pairwise models for surrogate-based heuristic optimisation. *Swarm and Evolutionary Computation*, 2022. doi:[10.1016/j.swevo.2022.101176](https://doi.org/10.1016/j.swevo.2022.101176).
- [13] X. Wang, G. Gary Wang, B. Song, P. Wang, and Y. Wang. A novel evolutionary sampling assisted optimization method for high-dimensional expensive problems. *IEEE Transactions on Evolutionary Computation*, 2019. doi:[10.1109/TEVC.2019.2890818](https://doi.org/10.1109/TEVC.2019.2890818).
- [14] X. Cai, L. Gao, and X. Li. Efficient generalized surrogate-assisted evolutionary algorithm for high-dimensional expensive problems. *IEEE Transactions on Evolutionary Computation*, 2020. doi:[10.1109/TEVC.2019.2919762](https://doi.org/10.1109/TEVC.2019.2919762).
- [15] W. Wang, H.-L. Liu, and K. C. Tan. A surrogate-assisted differential evolution algorithm for high-dimensional expensive optimization problems. *IEEE Transactions on Cybernetics*, 2023. doi:[10.1109/TCYB.2022.3175533](https://doi.org/10.1109/TCYB.2022.3175533).
- [16] D. Zhan and H. Xing. A fast kriging-assisted evolutionary algorithm based on incremental learning. *IEEE Transactions on Evolutionary Computation*, 2021. doi:[10.1109/TEVC.2021.3067015](https://doi.org/10.1109/TEVC.2021.3067015).
- [17] Laiqi Yu, Zhenyu Meng, Lingping Kong, Vaclav Snasel, and Jeng-Shyang Pan. Surrogate-assisted differential evolution: A survey. *Swarm and Evolutionary Computation*, 94:101879, 2025. doi:[10.1016/j.swevo.2025.101879](https://doi.org/10.1016/j.swevo.2025.101879).
- [18] Yaochu Jin, Handing Wang, Tinkle Chugh, Dan Guo, and Kaisa Miettinen. Data-driven evolutionary optimization: An overview and case studies. *IEEE Transactions on Evolutionary Computation*, 23(3):442–458, 2019. doi:[10.1109/TEVC.2018.2869001](https://doi.org/10.1109/TEVC.2018.2869001).

- [19] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading, MA, 1989.
- [20] Larry J. Eshelman and J. David Schaffer. Real-coded genetic algorithms and interval-schemata. In L. Darrell Whitley, editor, *Foundations of Genetic Algorithms*, volume 2, pages 187–202. Morgan Kaufmann, San Mateo, CA, 1993. doi:[10.1016/B978-0-08-094832-4.50018-0](https://doi.org/10.1016/B978-0-08-094832-4.50018-0).
- [21] Rainer Storn and Kenneth Price. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997. doi:[10.1023/A:1008202821328](https://doi.org/10.1023/A:1008202821328).
- [22] Ryoji Tanabe and Alex S. Fukunaga. Success-history based parameter adaptation for differential evolution. In *2013 IEEE Congress on Evolutionary Computation*, pages 71–78, 2013. doi:[10.1109/CEC.2013.6557555](https://doi.org/10.1109/CEC.2013.6557555).
- [23] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi:[10.1111/j.2517-6161.1996.tb02080.x](https://doi.org/10.1111/j.2517-6161.1996.tb02080.x).
- [24] Rolland L. Hardy. Multiquadric equations of topography and other irregular surfaces. *Journal of Geophysical Research*, 76(8):1905–1915, 1971. doi:[10.1029/JB076i008p01905](https://doi.org/10.1029/JB076i008p01905).
- [25] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004. doi:[10.1023/B:STC0.0000035301.49549.88](https://doi.org/10.1023/B:STC0.0000035301.49549.88).
- [26] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi:[10.1023/A:1010933404324](https://doi.org/10.1023/A:1010933404324).
- [27] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi:[10.1145/2939672.2939785](https://doi.org/10.1145/2939672.2939785).
- [28] Noor H. Awad, M. Z. Ali, P. N. Suganthan, Jing Liang, and B. Y. Qu. Problem definitions and evaluation criteria for the cec 2017 special session and competition on single objective real-parameter numerical optimization. Technical report, Nanyang Technological University, 2016.

Apéndice A

Repositorio del proyecto

El código fuente asociado a este Trabajo Fin de Grado está disponible en el siguiente repositorio público:

<https://github.com/marinoferperez/tfg>

Este repositorio contiene la implementación desarrollada, los scripts empleados para la ejecución de los experimentos y la memoria en formato PDF.

Apéndice B

Tablas completas de comparación TACOLAB

Este apéndice recoge las tablas completas generadas con TACOLAB para la comparación de las configuraciones de reinicio. Para cada metaheurística se incluye la tabla basada en la media del error final por función, correspondiente al 100 % del presupuesto de evaluaciones. Estas tablas sirven como soporte detallado de las conclusiones presentadas en el Capítulo 7.

B.1. AGE

Tabla B.1: Comparación TACOLAB por media del error final para AGE.

	base	reinicio-1	reinicio-10	reinicio-3	reinicio-5	reinicio-7
F01	2.0373e+03	1.9782e+03	2.5359e+03	1.9269e+03	2.1499e+03	2.2942e+03
F02	1.0377e+03	6.5281e+01	1.5988e+03	4.9217e+02	5.0543e+02	4.0493e+02
F03	1.7707e-03	2.2980e-03	1.7707e-03	2.2689e-03	1.8725e-03	1.8573e-03
F04	8.8714e-01	9.2240e-01	8.8714e-01	8.8714e-01	9.0483e-01	8.8714e-01
F05	2.5755e+01	1.7983e+01	2.5562e+01	2.5494e+01	2.4874e+01	2.3200e+01
F06	7.7672e+00	2.1819e+00	6.8833e+00	2.8286e+00	5.0119e+00	5.8280e+00
F07	4.9534e+01	3.6365e+01	4.8787e+01	4.2792e+01	4.1709e+01	4.3400e+01
F08	2.3754e+01	2.0113e+01	2.2279e+01	2.1621e+01	2.0379e+01	2.0849e+01
F09	2.4554e+02	5.4311e+01	2.1010e+02	1.6246e+02	2.0548e+02	2.0799e+02
F10	1.1618e+03	9.3990e+02	1.1055e+03	9.7229e+02	1.0699e+03	1.1123e+03
F11	5.3903e+01	5.3627e+01	5.4623e+01	5.5033e+01	5.5806e+01	5.4870e+01
F12	1.1909e+04	1.0543e+04	1.2654e+04	1.1145e+04	1.2149e+04	1.3945e+04
F13	1.0142e+04	9.9611e+03	1.0300e+04	9.8447e+03	1.0652e+04	1.0296e+04
F14	5.7288e+03	9.2651e+01	4.8317e+03	2.9903e+02	1.4101e+03	2.7567e+03
F15	5.9966e+03	6.0711e+03	6.0863e+03	6.0518e+03	6.0476e+03	6.0863e+03
F16	2.2330e+02	2.2269e+02	2.3001e+02	2.1673e+02	2.2979e+02	2.3001e+02
F17	9.3425e+01	4.7159e+01	8.1236e+01	4.8581e+01	5.5784e+01	6.8300e+01
F18	1.3225e+04	1.4181e+04	1.5015e+04	1.4801e+04	1.3866e+04	1.4550e+04
F19	7.3883e+03	1.4012e+03	8.8123e+03	3.1613e+03	5.6180e+03	6.5637e+03
F20	9.8971e+01	3.2702e+01	6.1055e+01	4.8759e+01	4.4538e+01	5.0543e+01
F21	2.1566e+02	1.2155e+02	1.9687e+02	1.5203e+02	1.6193e+02	1.7358e+02
F22	1.1864e+02	1.0725e+02	1.1629e+02	1.0807e+02	1.1036e+02	1.1257e+02
F23	3.2757e+02	3.2351e+02	3.2711e+02	3.2623e+02	3.2497e+02	3.2599e+02
F24	3.5722e+02	2.9683e+02	3.5804e+02	3.3845e+02	3.3940e+02	3.3296e+02
F25	4.4208e+02	4.3356e+02	4.3939e+02	4.3849e+02	4.3698e+02	4.3925e+02
F26	7.5003e+02	4.0530e+02	6.5137e+02	4.4463e+02	5.0497e+02	5.2202e+02
F27	4.1860e+02	4.0035e+02	4.0963e+02	4.0292e+02	4.0507e+02	4.0630e+02
F28	5.1200e+02	3.9980e+02	5.0908e+02	4.6571e+02	5.0597e+02	4.5912e+02
F29	3.6464e+02	3.0625e+02	3.7241e+02	3.2310e+02	3.4096e+02	3.4660e+02
F30	2.6510e+05	2.3571e+05	2.6510e+05	2.6510e+05	2.9145e+05	2.6510e+05
Best	3	23	1	4	0	0

B.2. DE

Tabla B.2: Comparación TACOLAB por media del error final para DE.

	base	reinicio-1	reinicio-10	reinicio-3	reinicio-5	reinicio-7
F01	0.0000e+00	7.6900e-08	5.5729e-16	1.0000e-09	0.0000e+00	0.0000e+00
F02	3.4314e+00	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00
F03	0.0000e+00	8.4700e-08	0.0000e+00	1.0000e-09	0.0000e+00	0.0000e+00
F04	1.9846e+00	1.9846e+00	1.9846e+00	1.9846e+00	1.9846e+00	1.9846e+00
F05	6.6979e+00	3.2110e+01	1.4970e+01	2.7650e+01	2.5471e+01	1.9878e+01
F06	0.0000e+00	8.3890e-07	0.0000e+00	5.1900e-08	2.6000e-09	1.0000e-10
F07	1.9844e+01	4.4916e+01	3.0764e+01	3.8854e+01	3.6615e+01	3.4142e+01
F08	7.9615e+00	3.3817e+01	1.5023e+01	2.8654e+01	2.4748e+01	2.3096e+01
F09	0.0000e+00	4.0600e-08	0.0000e+00	3.0000e-10	0.0000e+00	0.0000e+00
F10	3.0274e+02	7.0308e+02	2.5019e+02	3.7422e+02	3.4628e+02	2.5725e+02
F11	8.2381e-01	8.0757e+00	7.4577e-01	7.3592e-01	7.3506e-01	7.6528e-01
F12	4.3001e+01	8.3347e+01	3.0543e+01	3.3505e+01	3.1849e+01	2.9971e+01
F13	4.0742e+00	1.2733e+01	4.0547e+00	5.8807e+00	4.1636e+00	3.9615e+00
F14	5.6576e-01	2.2127e+01	3.3165e-01	1.8751e+00	3.9073e-01	3.9018e-01
F15	3.1490e-01	3.4334e+00	9.0260e-02	9.1121e-02	1.2889e-01	1.5642e-01
F16	4.0640e+00	5.4416e+00	1.9651e+00	1.5877e+00	1.7775e+00	1.7335e+00
F17	4.4162e+00	2.7631e+01	2.1214e+00	6.8532e+00	2.2652e+00	2.9001e+00
F18	7.1684e-01	5.3399e+00	1.5231e-01	1.2815e-01	1.3736e-01	1.1089e-01
F19	1.3202e-01	4.1201e-01	3.5213e-02	5.7284e-03	4.8801e-02	3.3380e-02
F20	3.2288e-01	5.0469e+00	1.8975e-01	6.0648e-01	2.2699e-01	2.4599e-01
F21	1.8123e+02	1.0110e+02	1.4713e+02	1.0284e+02	1.1756e+02	1.4957e+02
F22	8.5808e+01	6.6807e+01	7.4191e+01	7.4898e+01	8.0549e+01	7.4910e+01
F23	3.0695e+02	3.2870e+02	3.0606e+02	3.1792e+02	3.0728e+02	3.0670e+02
F24	3.2877e+02	1.9271e+02	3.1059e+02	2.5786e+02	3.0349e+02	3.0593e+02
F25	4.1276e+02	3.9895e+02	4.0350e+02	4.0347e+02	4.0528e+02	4.0620e+02
F26	3.0551e+02	3.0000e+02	3.0000e+02	3.0000e+02	3.0000e+02	3.0000e+02
F27	3.9046e+02	3.8930e+02	3.8937e+02	3.8931e+02	3.8935e+02	3.8936e+02
F28	3.7041e+02	3.0204e+02	3.0841e+02	3.0393e+02	3.1356e+02	3.1933e+02
F29	2.3027e+02	2.6621e+02	2.3014e+02	2.3618e+02	2.3003e+02	2.3009e+02
F30	1.5136e+05	5.3808e+02	6.4156e+02	5.9290e+02	5.9011e+02	6.4653e+02
Best	6	7	8	2	2	3

B.3. SHADE

Tabla B.3: Comparación TACOLAB por media del error final para SHADE.

	base	reinicio-1	reinicio-10	reinicio-3	reinicio-5	reinicio-7
F01	0.0000e+00	7.2000e-09	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00
F02	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00
F03	0.0000e+00	2.7000e-09	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00
F04	0.0000e+00	2.7000e-08	0.0000e+00	2.0000e-10	0.0000e+00	0.0000e+00
F05	3.4452e+00	1.1575e+01	3.4123e+00	3.9694e+00	3.5219e+00	3.3196e+00
F06	0.0000e+00	2.4960e-07	0.0000e+00	2.5000e-09	0.0000e+00	0.0000e+00
F07	1.3124e+01	2.4709e+01	1.3146e+01	1.4503e+01	1.3409e+01	1.3219e+01
F08	3.0325e+00	1.1459e+01	3.0514e+00	3.5335e+00	3.2047e+00	2.9952e+00
F09	0.0000e+00	1.8000e-09	0.0000e+00	0.0000e+00	0.0000e+00	0.0000e+00
F10	1.4816e+02	6.6740e+02	1.4758e+02	2.3949e+02	1.6482e+02	1.6224e+02
F11	1.6867e-01	3.1301e+00	1.6867e-01	4.3768e-01	2.0660e-01	1.6956e-01
F12	2.0329e+02	1.4582e+02	1.4813e+02	1.4920e+02	1.5488e+02	1.5056e+02
F13	4.7045e+00	8.7646e+00	4.6967e+00	4.9920e+00	4.6572e+00	4.7399e+00
F14	2.8095e+00	1.4829e+01	2.4831e+00	6.0732e-01	4.3358e-01	1.9814e+00
F15	2.2322e-01	1.0419e+00	2.1649e-01	3.0078e-01	2.2599e-01	2.1788e-01
F16	8.1808e-01	4.6231e+00	8.1821e-01	1.3234e+00	8.8169e-01	8.2286e-01
F17	1.9698e+00	2.5044e+01	1.6216e+00	6.1595e+00	1.9554e+00	1.9426e+00
F18	3.3802e+00	3.2845e+00	2.7050e+00	1.7840e+00	1.5789e+00	2.3934e+00
F19	2.1476e-01	1.7525e+00	2.1476e-01	1.7171e-01	1.9909e-01	1.9286e-01
F20	3.9263e-01	1.6146e+01	3.9263e-01	4.5721e-01	3.9263e-01	3.9263e-01
F21	1.6349e+02	1.0010e+02	1.3787e+02	1.0006e+02	1.0619e+02	1.3063e+02
F22	9.6855e+01	9.0981e+01	9.6096e+01	9.2384e+01	9.6096e+01	9.4357e+01
F23	3.0478e+02	3.1175e+02	3.0480e+02	3.0483e+02	3.0479e+02	3.0474e+02
F24	2.9665e+02	1.4532e+02	2.7214e+02	1.6216e+02	2.3879e+02	2.3419e+02
F25	4.1745e+02	4.0328e+02	4.1698e+02	4.0880e+02	4.1423e+02	4.1608e+02
F26	3.0377e+02	3.0000e+02	3.0000e+02	3.0000e+02	3.0000e+02	3.0000e+02
F27	3.9110e+02	3.8934e+02	3.8937e+02	3.8929e+02	3.8936e+02	3.8939e+02
F28	4.3677e+02	3.0801e+02	3.3246e+02	3.1961e+02	3.3613e+02	3.3669e+02
F29	2.4490e+02	2.6686e+02	2.4556e+02	2.5053e+02	2.4637e+02	2.4552e+02
F30	1.0504e+05	1.6456e+04	1.6468e+04	4.3812e+02	4.3482e+02	3.2481e+04
Best	6	5	4	3	4	3

Apéndice C

Tablas de configuraciones de hiperparámetros

C.1. Ranking global

Kernel	ε	Vecinos	Smoothing	Pos. AGE	Pos. DE	Pos. SHADE	Peor pos.
<i>mq</i>	1.0	50	10^{-3}	7	6	9	9
<i>mq</i>	1.0	50	10^{-2}	10	14	11	14
<i>mq</i>	1.0	100	10^{-3}	13	2	15	15
<i>gauss</i>	0.1	50	10^{-2}	15	17	16	17
<i>mq</i>	0.1	50	10^{-3}	9	18	14	18
<i>mq</i>	1.0	100	10^{-2}	17	8	19	19
<i>gauss</i>	0.1	50	10^{-3}	19	10	7	19
<i>gauss</i>	0.1	25	10^{-2}	6	22	6	22
<i>mq</i>	10.0	50	10^{-3}	12	7	22	22
<i>mq</i>	10.0	100	10^{-3}	8	1	23	23
<i>gauss</i>	0.1	25	10^{-3}	11	23	1	23
<i>mq</i>	10.0	100	10^{-2}	20	3	24	24
<i>gauss</i>	1.0	50	10^{-3}	24	11	13	24
<i>gauss</i>	1.0	25	10^{-3}	21	26	3	26
<i>mq</i>	10.0	50	10^{-2}	26	12	26	26

Tabla C.1: Ranking global de las 36 configuraciones RBF candidatas, ordenadas por la peor posición obtenida entre las tres metaheurísticas (criterio minimax). La configuración seleccionada como homogénea aparece en negrita.

C.2. AGE

Kernel	ε	Vecinos	Smoothing	Rank medio	Pos.
<i>mq</i>	0.1	25	10^{-2}	11.964	1
<i>mq</i>	0.1	25	10^{-3}	12.954	2
<i>mq</i>	1.0	25	10^{-2}	13.339	3
<i>mq</i>	0.1	50	10^{-2}	14.089	4
<i>mq</i>	1.0	25	10^{-3}	14.111	5
<i>gauss</i>	0.1	25	10^{-2}	14.454	6
<i>mq</i>	1.0	50	10^{-3}	14.739	7
<i>mq</i>	10.0	100	10^{-3}	14.993	8
<i>mq</i>	0.1	50	10^{-3}	14.996	9
<i>mq</i>	1.0	50	10^{-2}	15.307	10
<i>gauss</i>	0.1	25	10^{-3}	15.65	11
<i>mq</i>	10.0	50	10^{-3}	15.704	12
<i>mq</i>	1.0	100	10^{-3}	15.757	13
<i>mq</i>	0.1	100	10^{-2}	16.089	14
<i>gauss</i>	0.1	50	10^{-2}	16.586	15

Tabla C.2: Top-15 configuraciones de RBF para AGE. Pos. indica la posición en el ranking completo de 36 configuraciones.

C.3. DE

Kernel	ε	Vecinos	Smoothing	Rank medio	Pos.
<i>mq</i>	10.0	100	10^{-3}	14.807	1
<i>mq</i>	1.0	100	10^{-3}	15.42	2
<i>mq</i>	10.0	100	10^{-2}	15.739	3
<i>gauss</i>	10.0	100	10^{-3}	16.436	4
<i>gauss</i>	1.0	100	10^{-3}	16.58	5
<i>mq</i>	1.0	50	10^{-3}	16.988	6
<i>mq</i>	10.0	50	10^{-3}	17.141	7
<i>mq</i>	1.0	100	10^{-2}	17.164	8
<i>gauss</i>	0.1	100	10^{-3}	17.171	9
<i>gauss</i>	0.1	50	10^{-3}	17.291	10
<i>gauss</i>	1.0	50	10^{-3}	17.418	11
<i>mq</i>	10.0	50	10^{-2}	17.438	12
<i>gauss</i>	10.0	50	10^{-3}	17.618	13
<i>mq</i>	1.0	50	10^{-2}	18.009	14
<i>gauss</i>	1.0	100	10^{-2}	18.077	15

Tabla C.3: Top-15 configuraciones de RBF para DE. Pos. indica la posición en el ranking completo de 36 configuraciones.

C.4. SHADE

Kernel	ε	Vecinos	Smoothing	Rank medio	Pos.
<i>gauss</i>	0.1	25	10^{-3}	12.429	1
<i>mq</i>	0.1	25	10^{-3}	14.355	2
<i>gauss</i>	1.0	25	10^{-3}	14.979	3
<i>gauss</i>	1.0	25	10^{-2}	14.982	4
<i>mq</i>	1.0	25	10^{-3}	14.984	5
<i>gauss</i>	0.1	25	10^{-2}	15.029	6
<i>gauss</i>	0.1	50	10^{-3}	15.461	7
<i>mq</i>	1.0	25	10^{-2}	15.52	8
<i>mq</i>	1.0	50	10^{-3}	16.054	9
<i>mq</i>	0.1	25	10^{-2}	16.962	10
<i>mq</i>	1.0	50	10^{-2}	16.973	11
<i>gauss</i>	1.0	50	10^{-2}	16.975	12
<i>gauss</i>	1.0	50	10^{-3}	17.193	13
<i>mq</i>	0.1	50	10^{-3}	17.507	14
<i>mq</i>	1.0	100	10^{-3}	18.557	15

Tabla C.4: Top-15 configuraciones de RBF para SHADE. Pos. indica la posición en el ranking completo de 36 configuraciones.

Apéndice D

Tablas de configuraciones de parámetros para modelar la hibridación *online*

D.1. AGE

p	cd	rt	Rank medio	Pos.
0.75	500	0.25	6.143	1
0.75	500	0.5	6.643	2
0.25	50	0.25	7.000	3
0.25	50	0.5	7.643	4
0.25	500	0.5	8.929	5
0.5	50	0.25	9.000	6
0.75	50	0.5	9.214	7
0.75	500	0.75	10.214	8
0.5	500	0.75	10.286	9
0.25	50	0.75	10.500	10
0.75	50	0.25	10.857	11
0.25	500	0.25	11.143	12
0.5	500	0.25	12.143	13
0.25	100	0.25	12.286	14
0.25	100	0.75	12.429	15
0.75	50	0.75	12.500	16
0.25	500	0.75	12.786	17
0.75	100	0.25	12.857	18
0.5	50	0.75	13.143	19
0.75	100	0.75	14.286	20

Tabla D.1: Ranking de configuraciones del subrogado *online* para AGE.

D.2. DE

p	cd	rt	Rank medio	Pos.
0.75	50	0.5	7.857	1
0.75	100	0.25	8.429	2
0.25	50	0.25	8.929	3
0.25	100	0.75	9.000	4
0.5	500	0.75	9.214	5
0.25	50	0.75	9.286	6
0.75	500	0.25	9.714	7
0.5	500	0.25	10.000	8
0.25	500	0.25	10.143	9
0.75	500	0.75	10.286	10
0.25	100	0.25	10.286	11
0.5	50	0.75	10.643	12
0.75	500	0.5	10.857	13
0.75	100	0.75	11.000	14
0.25	50	0.5	11.571	15
0.75	50	0.75	11.714	16
0.5	50	0.25	11.857	17
0.25	500	0.5	12.786	18
0.25	500	0.75	13.143	19
0.75	50	0.25	13.286	20

Tabla D.2: Ranking de configuraciones del subrogado *online* para DE.

D.3. SHADE

p	cd	rt	Rank medio	Pos.
0.25	100	0.25	7.857	1
0.5	500	0.25	7.929	2
0.25	500	0.5	8.500	3
0.5	50	0.75	8.714	4
0.25	500	0.25	8.857	5
0.75	500	0.5	9.000	6
0.5	50	0.25	9.214	7
0.5	500	0.75	9.500	8
0.75	500	0.25	9.929	9
0.75	100	0.25	9.929	10
0.25	50	0.25	10.143	11
0.75	50	0.5	11.000	12
0.25	100	0.75	11.714	13
0.25	50	0.5	11.857	14
0.75	50	0.25	12.000	15
0.75	500	0.75	12.000	16
0.25	50	0.75	12.286	17
0.25	500	0.75	13.000	18
0.75	50	0.75	13.143	19
0.75	100	0.75	13.429	20

Tabla D.3: Ranking de configuraciones del subrogado *online* para SHADE.

Apéndice E

Tablas de comparación de la hibridación respecto a la metaheurística original

E.1. AGE

	AGE-1	AGE-1-RBF
F01	2.1706e+03	2.1285e+03
F02	1.7550e+02	5.6500e+02
F03	1.7938e-03	1.3908e-03
F04	1.0312e+00	1.0084e+00
F05	1.8942e+01	2.0563e+01
F06	1.5711e+00	2.0301e+00
F07	3.5374e+01	3.7312e+01
F08	1.8713e+01	1.8534e+01
F09	3.8845e+01	4.0675e+01
F10	9.3195e+02	9.3281e+02
F11	5.5750e+01	5.2091e+01
F12	1.3012e+04	1.3025e+04
F13	1.1635e+04	1.0778e+04
F14	7.1815e+01	8.2678e+01
F15	3.9880e+03	3.6345e+03
F16	2.1880e+02	2.2340e+02
F17	4.1682e+01	4.2193e+01
F18	1.0692e+04	1.0877e+04
F19	1.0721e+03	1.1597e+03
F20	3.3668e+01	3.4317e+01
F21	1.2186e+02	1.2259e+02
F22	1.0879e+02	1.0901e+02
F23	3.2742e+02	3.2702e+02
F24	2.7769e+02	2.5145e+02
F25	4.3254e+02	4.3024e+02
F26	3.3352e+02	3.3996e+02
F27	4.0051e+02	3.9949e+02
F28	4.2414e+02	3.8448e+02
F29	3.2315e+02	3.1964e+02
F30	4.1341e+05	4.2086e+05

E.2. DE

	DE-10	DE-10-RBF
F01	7.8426e-01	2.7864e-16
F02	0.0000e+00	0.0000e+00
F03	0.0000e+00	0.0000e+00
F04	2.1187e+00	4.4316e-01
F05	1.3675e+01	6.2672e+00
F06	0.0000e+00	0.0000e+00
F07	3.0248e+01	2.7216e+01
F08	1.4920e+01	1.0094e+01
F09	0.0000e+00	0.0000e+00
F10	2.6422e+02	2.8490e+02
F11	7.1306e-01	4.4969e-01
F12	4.0043e+01	5.7382e+01
F13	4.2009e+00	4.6395e+00
F14	3.5126e-01	3.2238e-01
F15	1.8027e-01	2.0526e-01
F16	3.5059e+00	3.7371e+00
F17	1.7544e+00	1.5622e+00
F18	9.0228e-01	4.7922e-01
F19	8.1656e-03	1.6695e-02
F20	1.3466e-01	2.1041e-01
F21	1.4718e+02	1.5371e+02
F22	9.0410e+01	9.0087e+01
F23	3.0620e+02	3.0600e+02
F24	2.9274e+02	3.1597e+02
F25	4.0338e+02	4.0612e+02
F26	3.0000e+02	3.0000e+02
F27	3.8943e+02	3.8926e+02
F28	3.2099e+02	3.4536e+02
F29	2.3079e+02	2.3114e+02
F30	5.5709e+02	1.6516e+04
Best	17	17

E.3. SHADE

	SHADE-10	SHADE-10-RBF
F01	0.0000e+00	0.0000e+00
F02	0.0000e+00	0.0000e+00
F03	0.0000e+00	0.0000e+00
F04	0.0000e+00	0.0000e+00
F05	3.4567e+00	3.5831e+00
F06	0.0000e+00	0.0000e+00
F07	1.3154e+01	1.3094e+01
F08	2.9422e+00	2.9159e+00
F09	0.0000e+00	0.0000e+00
F10	1.5989e+02	1.6662e+02
F11	2.9836e-01	3.1925e-01
F12	1.7608e+02	1.9533e+02
F13	5.1150e+00	5.1634e+00
F14	1.3050e+00	1.3314e+00
F15	1.8916e-01	2.1108e-01
F16	8.3047e-01	2.5362e+00
F17	2.2985e+00	2.4185e+00
F18	6.2641e+00	7.4181e+00
F19	1.0766e-01	1.0988e-01
F20	7.8447e-01	4.4559e-01
F21	1.4257e+02	1.4111e+02
F22	9.4125e+01	9.4118e+01
F23	3.0513e+02	3.0511e+02
F24	2.6285e+02	2.6014e+02
F25	4.2228e+02	4.2164e+02
F26	3.1528e+02	2.9804e+02
F27	3.8935e+02	3.8934e+02
F28	3.2391e+02	3.4836e+02
F29	2.4689e+02	2.4645e+02
F30	6.4539e+04	3.2500e+04
Best	18	18

Apéndice F

Parámetros de ejecución de los experimentos

En este apéndice se recogen los parámetros de ejecución de los experimentos indicados en el Capítulo 6.

F.1. Parámetros de ejecución de las metaheurísticas

Parámetro	Valor por defecto	Descripción
<code>--algoritmo</code>	<code>all</code>	Metaheurística ejecutada. Valores posibles: ▪ <code>age</code>: algoritmo genético estacionario. ▪ <code>de</code>: evolución diferencial. ▪ <code>shade</code>: variante adaptativa SHADE. ▪ <code>all</code>: ejecuta las tres metaheurísticas.
<code>--cec-funcid</code>	<code>-</code>	Funciones de CEC2017 ejecutadas. Valores posibles: ▪ Lista de identificadores: 1 4 10 ▪ Lista separada por comas: 1,4,10 ▪ Suite completa: <code>all</code>
<code>--cec-dim</code>	10	Dimensión de las funciones de CEC2017. Valores posibles: ▪ 2 ▪ 5 ▪ 10 ▪ 30 ▪ 50
<code>--seed-start</code>	1	Primera semilla si no se usa <code>--seeds</code> .
<code>--n-seeds</code>	10	Semillas consecutivas desde <code>--seed-start</code> .
<code>--tam-poblacion</code>	50	Tamaño de la población.
<code>--max-evals</code>	$10000 \cdot D$	Evaluaciones máximas (D = dimensión del problema).
<code>--reinicio</code>	<code>False</code>	Activa el reinicio elitista si se incluye.

Parámetro	Valor por defecto	Descripción
<code>--reinicio-ratio</code>	-	Fracción del presupuesto total que debe transcurrir sin una mejora relevante del segundo mejor individuo antes de activar el reinicio. Valores evaluados: ▪ 0.01 ▪ 0.03 ▪ 0.05 ▪ 0.07 ▪ 0.10
<code>--outdir</code>	<code>results/</code> <code>resultados_</code> <code>benchmark_mhs</code>	Directorio utilizado para almacenar los resultados.
<code>--seeds</code>	-	Lista explícita de semillas. Si se utiliza, sustituye a <code>--seed-start</code> y <code>--n-seeds</code> . Admite valores separados por espacios o comas. Por ejemplo: ▪ <code>--seeds 1 2 3</code> ▪ <code>--seeds 1,2,3</code>
<code>--sin-metricas</code>	<code>False</code>	Desactiva el registro de métricas y trayectorias.
<code>--verbose</code>	<code>False</code>	Muestra progreso por terminal si se incluye.

Tabla F.1: Parámetros de ejecutar `metaheurísticas.py`.

F.2. Parámetros de ejecución de las técnicas subrogadas

Parámetro	Valor por defecto	Descripción
<code>--resultados-dir</code>	-	Directorio con los resultados generados por el programa de ejecución de metaheurísticas.
<code>--algoritmo</code>	<code>all</code>	Metaheurística que generó los datos. Valores posibles: ▪ <code>age</code>: algoritmo genético estacionario. ▪ <code>de</code>: evolución diferencial. ▪ <code>shade</code>: variante adaptativa SHADE. ▪ <code>all</code>: evalúa las tres metaheurísticas.
<code>--cec-funcid</code>	-	Funciones de CEC2017 ejecutadas. Valores posibles: ▪ Lista de identificadores: <code>1 4 10</code> ▪ Lista separada por comas: <code>1,4,10</code> ▪ Suite completa: <code>all</code>
<code>--modelo</code>	-	Técnica subrogada a evaluar.
<code>--seed</code>	42	Semilla utilizada por el generador pseudoaleatorio para construir el conjunto de validación.
<code>--max-seeds</code>	-	Número máximo de trayectorias. Sin límite si se omite.
<code>--modelo-params-json</code>	-	Configuración JSON de hiperparámetros del modelo. Opcional.

Tabla F.2: Parámetros de los programas de evaluación *offline* acumulativa y no acumulativa.

Parámetro	Valor por defecto	Descripción
<code>--trayectoria-dir</code>	-	Directorio con las trayectorias generadas por las metaheurísticas.
<code>--ajuste- resultados-dir</code>	<code>resultados_ benchmark_ surrogates_ offline_ajuste</code>	Subdirectorio donde se almacenan los resultados del ajuste.
<code>--algoritmo</code>	<code>all</code>	Metaheurística que generó los datos. Valores posibles: ▪ <code>age</code>: algoritmo genético estacionario. ▪ <code>de</code>: evolución diferencial. ▪ <code>shade</code>: variante adaptativa SHADE. ▪ <code>all</code>: evalúa las tres metaheurísticas.
<code>--cec-funcid</code>	-	Funciones de CEC2017 utilizadas en la ejecución. Valores posibles: ▪ Identificador individual: <code>1</code> o <code>f1</code>. ▪ Lista separada por comas: <code>1,4,10</code>. ▪ Suite completa: <code>all</code>.

Parámetro	Valor por defecto	Descripción
<code>--modelo</code>	<code>all</code>	Técnica subrogada sometida al ajuste fino. Valores posibles: ▪ <code>rbf</code> ▪ <code>svr</code> ▪ <code>mlp</code> ▪ <code>rsm</code> ▪ <code>random_forest</code> ▪ <code>hgb</code> ▪ <code>lasso</code> ▪ <code>xgboost</code> ▪ <code>all</code>
<code>--param-grid-json</code>	-	Fichero JSON opcional con el espacio de hiperparámetros. Si se omite, se usa el grid interno por defecto del modelo.
<code>--metrica-ajuste</code>	<code>spearman</code>	Métrica para seleccionar la configuración óptima. Valores posibles: ▪ <code>spearman</code> ▪ <code>mae</code> ▪ <code>rmse</code> ▪ <code>nmae</code> ▪ <code>nrmse</code>
<code>--validacion-ratio</code>	<code>0.20</code>	Fracción final del bloque para validación interna. Debe cumplir $0 < r < 0.5$.
<code>--seed</code>	<code>42</code>	Semilla empleada durante el ajuste.

Parámetro	Valor por defecto	Descripción
<code>--seed-selection-random-state</code>	42	Semilla utilizada para seleccionar el subconjunto de trayectorias cuando se limita el número de semillas con <code>--max-seeds</code> .
<code>--max-seeds</code>	-	Número máximo de trayectorias. Sin límite si se omite. Si se indica, debe ser mayor o igual que 1 y no superar las trayectorias disponibles.

Tabla F.3: Parámetros de `ajustar_hiperparametros.py`.

Los espacios de hiperparámetros concretos y las configuraciones seleccionadas se presentan junto con sus resultados en el capítulo siguiente.

F.3. Parámetros de ejecución de la hibridación

Parámetro	Valor por defecto	Descripción
<code>--algoritmo</code>	age	Metaheurística <i>online</i> ejecutada. Valores posibles: ▪ age ▪ de ▪ shade ▪ all También admite listas separadas por espacios o comas.
<code>--cec-funcid</code>	-	Funciones de CEC2017 ejecutadas. Admite identificadores individuales, listas separadas por espacios o comas, y <code>all</code> para la suite completa.
<code>--cec-dim</code>	10	Dimensión de las funciones CEC2017. Valores posibles: 2, 5, 10, 30 y 50.
<code>--seed-start</code>	1	Primera semilla si no se proporciona una lista explícita mediante <code>--seeds</code> .
<code>--n-seeds</code>	10	Número de semillas consecutivas generadas a partir de <code>--seed-start</code> .
<code>--seeds</code>	-	Lista explícita de semillas. Si se indica, sustituye a <code>--seed-start</code> y <code>--n-seeds</code> .
<code>--tam-poblacion</code>	50	Tamaño de población utilizado por las metaheurísticas.
<code>--max-evals</code>	$10000 \cdot D$	Presupuesto máximo de evaluaciones reales.
<code>--reinicio</code>	False	Activa el mecanismo de reinicio elitista en la metaheurística <i>online</i> .

Parámetro	Valor por defecto	Descripción
<code>--reinicio-ratio</code>	-	Fracción del presupuesto que debe transcurrir sin mejora antes de permitir el reinicio. Solo puede utilizarse junto con <code>--reinicio</code> .
<code>--modelo</code>	Modelo seleccionado	Modelo subrogado empleado por el filtro <i>online</i> .
<code>--modelo-params-json</code>	Config. del modelo seleccionado	Fichero JSON con los hiperparámetros del modelo subrogado.
<code>--modelo-prob</code>	0.50	Probabilidad p de aplicar el filtro subrogado a un candidato. Debe estar en $[0, 1]$. En las pruebas se consideran 0.00, 0.25, 0.50 y 0.75.
<code>--warmup-ratio</code>	0.20	Proporción inicial del presupuesto durante la cual el subrogado permanece inactivo y todas las evaluaciones aceptadas se realizan con la función real.
<code>--window-ratio</code>	0.20	Proporción del presupuesto utilizada para definir la ventana reciente de entrenamiento del subrogado.
<code>--retrain-ratio</code>	0.25	Fracción de la ventana que debe renovarse antes de reentrenar el modelo. En las pruebas se consideran 0.25, 0.50 y 0.75.
<code>--cooldown-reinicio-evals</code>	0	Número de evaluaciones reales de enfriamiento tras cada reinicio. El valor 0 desactiva este mecanismo. En las pruebas se consideran 0, 50, 100 y 500.

Parámetro	Valor por defecto	Descripción
<code>--guardar-decisiones-subrogado</code>	<code>False</code>	Guarda un fichero con las decisiones del filtro, incluyendo la predicción, la referencia real y el margen entre ambas.
<code>--outdir</code>	<code>results/ cec/ cec2017_d10 _tam50 _online</code>	Directorio donde se almacenan los resultados de la ejecución <i>online</i> .
<code>--verbose</code>	<code>False</code>	Muestra el progreso de las ejecuciones por terminal.

Tabla F.4: Parámetros de ejecutar `metaheurísticas_online.py`.

